[EQUIPO: logotipo del centro]



CICLO FORMATIVO DE GRADO SUPERIOR EN DESARROLLO DE APLICACIONES MULTIPLATAFORMA

Omar **[OMAR: apellidos]**, Álvaro Baz Rodríguez **[ÁLVARO: confirmar]** e Iván Bermejo **[BERME: apellidos]**

CURSO 2026-2027

TÍTULO: NOVA Sneakers · Diseño de Interfaces, Programación de Servicios y Procesos y Sistemas de Gestión Empresarial (Odoo)

AUTORES: Omar **[OMAR: apellidos]**, Álvaro Baz Rodríguez **[ÁLVARO: confirmar]** e Iván Bermejo **[BERME: apellidos]**

TITULACIÓN: Grado Superior en Desarrollo de Aplicaciones Multiplataforma

FECHA: **[EQUIPO: fecha de entrega]**

RESUMEN

Este proyecto intermodular presenta NOVA Sneakers, una marca de zapatillas inventada sobre la que hemos desarrollado la primera solución digital que pide el enunciado «Sneaker Factory» de 2.º DAM. Esta memoria reúne tres de los módulos del proyecto.

- **Diseño de Interfaces.** Una web responsive de la marca, hecha con HTML, CSS con metodología BEM y JavaScript sin frameworks. Tiene catálogo de 100 modelos, ficha de producto, probador conceptual y una solicitud Custom simulada. Como ampliación, una tienda conectada a un servidor Node, el panel de administración y el portal del trabajador.
- **Programación de Servicios y Procesos.** Un subsistema auxiliar en Java: un proceso padre lanza tres procesos hijos con `ProcessBuilder` para validar un pedido Custom exportado en CSV, agrupar sus tallas y generar una ficha de taller. Se comunican por la entrada y la salida estándar y controlan tiempos y códigos de salida.
- **Sistemas de Gestión Empresarial.** La configuración de Odoo 19 en Ubuntu que documentó Iván Bermejo. Recorre el pedido S00005 del Club Atletico NOVA desde la venta hasta la fabricación, la entrega y la factura.

Las tres partes se han probado con baterías automáticas y evidencias que se guardan en el repositorio. La memoria explica también, con honestidad, lo que todavía no está demostrado: la trazabilidad de un mismo pedido entre todos los sistemas, la prueba de usabilidad con personas y la copia de seguridad de Odoo.

Palabras clave: HTML5, CSS3, BEM, JavaScript, accesibilidad, Java, ProcessBuilder, procesos, IPC, Odoo, ERP, fabricación, Git.

ABSTRACT

This intermodular project presents NOVA Sneakers, a fictional sneaker brand for which we built the first digital solution required by the 2nd-year DAM «Sneaker Factory» brief. This report covers three of the project's modules.

- **Interface Design.** A responsive brand website built with HTML, CSS following the BEM methodology and framework-free JavaScript. It offers a 100-model catalog, product sheets, a conceptual fitting tool and a simulated Custom request form. As extensions, it adds an online store connected to a Node server, the administration panel and the worker portal.
- **Services and Processes Programming.** An auxiliary Java subsystem in which a parent process launches three child processes with `ProcessBuilder` to validate a Custom order exported as CSV, group its sizes and generate a workshop sheet. They communicate through standard input and output and control timeouts and exit codes.
- **Business Management Systems.** The Odoo 19 setup on Ubuntu documented by Iván Bermejo. It follows sales order S00005 for Club Atletico NOVA from sale to manufacturing, delivery and invoice.

All three parts were tested with automated test suites and evidence stored in the repository. The report also states honestly what has not been demonstrated yet: end-to-end traceability of a single order across all systems, a usability test with real users and an Odoo backup.

Keywords: HTML5, CSS3, BEM, JavaScript, accessibility, Java, ProcessBuilder, processes, IPC, Odoo, ERP, manufacturing, Git.

ÍNDICE

CAPÍTULO 1. RESUMEN DEL PROYECTO 8
1.1 Contexto y justificación 8
1.2 Planteamiento del problema 8
1.3 Objetivos del proyecto 8
1.4 Resultados obtenidos 9
Diseño de Interfaces 9
Programación de Servicios y Procesos 9
Sistemas de Gestión Empresarial 9
Integración 9
CAPÍTULO 2. ANTECEDENTES DEL PROYECTO Y ESTADO DEL ARTE 10
2.1 Estado del arte 10
Webs de marca y personalización 10
ERP para pequeñas fábricas 10
Procesos y servicios en Java 10
2.2 Contexto y justificación de nuestras decisiones 10
2.3 Planteamiento del trabajo en equipo 11
CAPÍTULO 3. DESARROLLO DEL PROYECTO 12
3.1 Diseño de Interfaces 13
3.1.1 Planificación y metodología del proyecto 13
Planificación del trabajo 13
Metodología de trabajo 14
Herramientas utilizadas 15
3.1.2 Descripción de la solución 15
Estructura de la web 15
Diseño de marca 16
HTML semántico 16
CSS con BEM y variables 17
JavaScript sin frameworks 17
Diseño responsive 18
Accesibilidad 18
La tienda conectada al servidor 18
El panel de administración (Álvaro) 19
El portal del trabajador (Berme) 19

La página «Análisis de datos» 19
3.1.3 Requisitos del proyecto 19
3.1.4 Resultados del proyecto 20
3.1.5 Pruebas 27
3.1.6 Problemas encontrados y soluciones 28
El doble pie de página 28
El cambio a BEM al final 29
El contraste del rediseño de colores 29
Otros fallos que encontraron las pruebas 29
Limitaciones que siguen abiertas 29
3.2 Programación de Servicios y Procesos (PSP) 30
3.2.1 Planificación y metodología del proyecto 30
Lo planificado: el plan operativo del equipo 30
Lo hecho según el repositorio 30
Herramientas utilizadas 32
3.2.2 Descripción de la solución 32
Arquitectura: un proceso padre y tres hijos 32
Flujo del procesamiento 33
Protocolo de cada hijo: stdin y stdout 34
La salida de error, aparte: un hilo por tubería 34
Tiempos límite: waitFor, destroy y destroyForcibly 35
Códigos de salida 35
Formato de la entrada: el CSV de 8 columnas 36
La ficha y el registro 37
Relación con PostgreSQL: el script 15 37
Relación con Odoo 37
3.2.3 Requisitos del proyecto 38
Objetivo, requisitos obligatorios y guion de la demostración 38
Anexo y criterios técnicos observables 39
3.2.4 Resultados del proyecto 40
El pedido CUSTOM-40 de principio a fin 41
La ficha auxiliar para el taller 41
Cada hijo por separado 42
Errores controlados: códigos 1, 2 y 3 42
El registro de las ejecuciones 43

La exportación desde PostgreSQL 43
3.2.5 Pruebas 44
PruebasPsp: 27 de 27 44
El analizador de Álvaro: 8 de 8 46
El comprobador rápido de Berme 47
Lo que no está probado 47
3.2.6 Problemas encontrados y soluciones 47
La codificación UTF-8 en Windows 47
El comprobador de Berme fallaba por la respuesta antigua 48
El tiempo de espera del LanzadorAnalizador no llega a actuar 49
Dos versiones del analizador 49
3.3 Sistemas de Gestión Empresarial (Odoo) 50
3.3.1 Planificación y metodología del proyecto 50
Planificación del trabajo 50
Metodología de trabajo 51
Herramientas utilizadas 51
3.3.2 Descripción de la solución 51
Entorno 51
Compañía, aplicaciones y usuarios 52
Productos, variantes y lista de materiales 52
Flujo de negocio 52
Ampliación documentada: conexión por API 53
3.3.3 Requisitos del proyecto 53
Lo que falta 55
3.3.4 Resultados del proyecto 55
Entorno e instalación 56
Base de datos, compañía y aplicaciones 58
Producto terminado y componentes 60
Fabricación de prueba: WH/MO/00001 61
Caso documentado: S00005 (Club Atletico NOVA) 63
Variantes por talla 68
Trazabilidad del stock 70
Usuarios 70
3.3.5 Pruebas 71
Lo que hay que demostrar en vivo en el Mac de Berme 71

3.3.6 Problemas encontrados y soluciones 72
Límites 74
CAPÍTULO 4. IMPLEMENTACIÓN DEL PROYECTO 76
4.1 Entorno de ejecución 76
4.2 Demostración ante el profesor 76
Comprobación de la trazabilidad entre bloques 77
4.3 Lo que no forma parte de la entrega 77
CAPÍTULO 5. CONCLUSIONES 78
5.1 Conclusiones del trabajo 78
5.2 Conclusiones personales 78
CAPÍTULO 6. ANEXOS 79
Anexo A. Enlaces y versión entregada 79
Anexo B. Documentación original de Odoo (Iván Bermejo) 79
Anexo C. Evidencias de pruebas 98
Anexo D. Equivalencias de commits 99
Anexo E. Declaración de uso de herramientas de inteligencia artificial 99
Anexo F. Requisitos y evidencias 99

CAPÍTULO 1. RESUMEN DEL PROYECTO

1.1 Contexto y justificación

El enunciado del proyecto intermodular de 2.º DAM, «Sneaker Factory» (versión 0.3), nos pide trabajar como «una pequeña consultora tecnológica». Tenemos que diseñar e implantar una primera solución digital para una fábrica de zapatillas, con tres necesidades de negocio:

- **presencia digital:** una web con catálogo y solicitudes Custom;
- **gestión empresarial:** Odoo como núcleo del flujo de ventas y fabricación;
- **automatización técnica:** un subsistema auxiliar que procese pedidos Custom con varios procesos.

Para que el trabajo sea creíble hemos creado una marca propia, **NOVA Sneakers**, con su identidad, su catálogo de 100 modelos y sus datos de prueba.

El proyecto completo incluye también Acceso a Datos (PostgreSQL y Java) y Python (análisis de datos). Cada uno tiene su propia memoria, y esta reúne solo los tres módulos que comparten el enunciado intermodular: Diseño de Interfaces, Programación de Servicios y Procesos, y Sistemas de Gestión Empresarial.

1.2 Planteamiento del problema

Una fábrica que vende a clubes deportivos recibe pedidos colectivos y personalizados. El ejemplo del enunciado es un club que encarga 40 pares con sus tallas, sus colores y una fecha objetivo. Esa fábrica necesita tres cosas:

- que el cliente conozca la marca y pueda preparar su solicitud desde cualquier dispositivo;
- que la venta quede registrada en un ERP y desde ella se encadenen la fabricación, el inventario, la entrega y la factura;
- que haya una herramienta técnica que valide el pedido, agrupe las tallas y prepare una ficha para el taller, sin sustituir al ERP.

El enunciado da como ejemplo el pedido «SO024 · Club Halcones · Runner X · 40 pares · tallas 38-46». **Es un ejemplo, no un identificador obligatorio.** Nuestro caso documentado en Odoo es el pedido **S00005 del Club Atletico NOVA**.

1.3 Objetivos del proyecto

Módulo	Objetivo
Diseño de Interfaces	Una web responsive con inicio, catálogo (precio, descripción y tallas), ficha, solicitud Custom (formulario o prototipo) y contacto, cuidando la usabilidad y la accesibilidad
Sistemas de Gestión Empresarial	Instalar, configurar y adaptar Odoo a la empresa y demostrar cómo se registra y evoluciona una venta realista: cliente, venta, fabricación, inventario, entrega y factura
Programación de Servicios y Procesos	Un subsistema auxiliar con un proceso padre y varios hijos, comunicados entre sí, con control de errores, tiempos y

	códigos de salida, y con evidencias del procesamiento
Integración	Un modelo común del pedido y un caso de uso que se pueda seguir entre bloques. El enunciado admite hacerlo con ficheros CSV/JSON y procedimientos documentados

1.4 Resultados obtenidos

Diseño de Interfaces

- **La web:** responsive, hecha con HTML semántico, CSS con BEM y JavaScript propio. En la medición del 11/11, Lighthouse dio 100 en accesibilidad y buenas prácticas, y axe no encontró violaciones. Las comprobaciones automáticas en Edge de las regresiones del 13/11 pasaron todos sus controles: entre 29 y 139 por recorrido.
- **La solicitud Custom** es una simulación declarada: no crea pedidos.
- **Ampliaciones:** una tienda con cuentas, carrito y pago simulado; el panel de administración de Álvaro, conectado a PostgreSQL; y el portal del trabajador de Berme.
- **Pendiente:** la prueba de usabilidad con personas.

Programación de Servicios y Procesos

- **Lo que hace:** `ProcesadorPedidosCustom` lanza tres hijos (validador, analizador de tallas de Álvaro y generador de la ficha) en otras máquinas virtuales de Java. Gestiona la entrada y la salida estándar, los errores, los tiempos límite y los códigos de salida 0, 1 y 2.
- **Pruebas:** 27 de 27, más las 8 del analizador; el comprobador rápido de Berme es correcto.
- **Lo que procesa:** el CSV que exporta PostgreSQL. Todavía no procesa ninguna exportación hecha desde Odoo.

Sistemas de Gestión Empresarial

- **Lo documentado:** Iván Bermejo documenta con 28 capturas Odoo 19 en Ubuntu, la Runner X con lista de materiales y 9 variantes, dos usuarios internos y el recorrido completo de S00005: venta, fabricación WH/MO/00002, entrega WH/OUT/00005 y factura INV/2026/00001 pagada.
- **Pendiente:** la copia de seguridad entregada, las exportaciones e informes de Odoo y una venta con líneas por talla. Se pueden hacer con la configuración que ya existe.

Integración

- **Lo que funciona:** cada bloque funciona y está probado por separado, y la referencia del pedido es el dato que los une.
- **Lo que falta:** la trazabilidad de un mismo pedido en Odoo, PostgreSQL, PSP y Python **no está demostrada todavía**. En el capítulo 4 describimos la ejecución que la comprobaría sin programar nada nuevo.

CAPÍTULO 2. ANTECEDENTES DEL PROYECTO Y ESTADO DEL ARTE

2.1 Estado del arte

Webs de marca y personalización

Las marcas de calzado deportivo combinan hoy tres elementos en su web:

- **un catálogo visual,** con fichas de producto y tallas;
- **la compra online;**
- **un configurador para personalizar la zapatilla.**

En pedidos colectivos, como los de un club, lo habitual es una solicitud o un presupuesto, no una compra directa. Técnicamente, estas webs se construyen con HTML semántico y CSS adaptable. Los equipos que no usan frameworks suelen organizar el CSS con una metodología de nombres como BEM, para que los estilos no se pisen.

La accesibilidad ya no es opcional. Las pautas WCAG y herramientas como Lighthouse o axe permiten medirla de forma objetiva.

ERP para pequeñas fábricas

Un ERP centraliza ventas, compras, inventario, fabricación y contabilidad. Para una pequeña empresa, Odoo es una opción frecuente: es modular y tiene una edición Community gratuita. Sus aplicaciones de Ventas, Fabricación (MRP), Inventario y Facturación cubren el flujo que pide el enunciado. Es el ERP que usamos en clase.

Procesos y servicios en Java

Cuando una tarea auxiliar se divide en pasos independientes, se puede repartir en varios procesos del sistema operativo. Esto aísla los fallos (si un paso se cuelga, se le corta) y obliga a definir bien el formato con el que se comunican. En Java, `ProcessBuilder` permite lanzar procesos hijos, redirigir su entrada y su salida estándar y esperar su código de salida. Los hilos serían la alternativa dentro de un mismo proceso; el enunciado los deja como ampliación.

2.2 Contexto y justificación de nuestras decisiones

- **Una marca propia, NOVA Sneakers,** con datos ficticios suficientes (100 modelos, clientes y pedidos de prueba), para que la demostración sea realista sin usar datos personales.
- **Web sin frameworks.** El enunciado pide HTML, CSS y, de forma opcional, JavaScript. Añadimos un servidor Node + Express solo para las ampliaciones (tienda, panel y portal), que ninguna rúbrica exige.
- **Odoo 19 Community en una máquina Ubuntu.** El enunciado pide una máquina virtual con Ubuntu y Odoo. Berme la montó con OrbStack en su ordenador.
- **PSP como subsistema auxiliar.** El enunciado insiste en que no sustituya a Ventas ni a Fabricación de Odoo. Por eso trabaja sobre una copia del pedido en CSV y produce una ficha auxiliar.

- **Integración con ficheros y procedimientos documentados,** como permite el enunciado, sin sincronización automática.

2.3 Planteamiento del trabajo en equipo

El equipo lo formamos Omar, Álvaro e Iván Bermejo (Berme). El plan de trabajo que escribimos en octubre de 2026 (`Sneaker_Factory_Reparto_Tareas_Git_Worktrees.pdf`) se basa en una regla: «Los tres tocan todo». Cada tarea tiene alguien que la empieza, pero no hay módulos exclusivos. Trabajamos con Git y GitHub, con una rama por persona y fusiones en una rama de integración.

El reparto real, contado por cada uno, está en la memoria común (`06_Intermodular/memoria/MEMORIA.md`, §9) **[EQUIPO: completar el reparto real antes de entregar]**.

CAPÍTULO 3. DESARROLLO DEL PROYECTO

En este capítulo se desarrolla cada módulo por separado, con la misma estructura en los tres:

1. planificación y metodología;
2. descripción de la solución;
3. requisitos del enunciado y cómo los cumplimos;
4. resultados con capturas;
5. pruebas;
6. problemas encontrados.

Los requisitos se citan con los identificadores de nuestro índice del enunciado oficial (por ejemplo, IM-INT-04 o IM-SGE-12), que reproducen literalmente su texto.

El estado de cada requisito se indica así:

- **cumplido:** acreditado con documentos o evidencias;
- **a demostrar en clase:** implementado, falta enseñarlo en vivo;
- **pendiente:** lo que todavía falta.

3.1 Diseño de Interfaces

3.1.1 Planificación y metodología del proyecto

El objetivo de Desarrollo de Interfaces lo marca el enunciado (IM §4.1): una web responsive que represente la identidad de NOVA Sneakers y permita recorrer su catálogo y su servicio de personalización para clubes y equipos. Sobre esa base el proyecto creció con tres piezas que no pide la rúbrica: una tienda con pago simulado, un panel de administración y un portal para el personal del almacén. Aquí separamos lo que planificamos de lo que de verdad se hizo.

Planificación del trabajo

En octubre de 2026 el equipo redactó un plan interno, «Sneaker Factory · Plan operativo del equipo y reparto de tareas» (`Sneaker_Factory_Reparto_Tareas_Git_Worktrees.pdf`). No es un enunciado ni el reparto real: es **lo planificado**. Su regla principal era que «los tres tocan todo»: una persona empieza cada tarea y las otras dos la revisan, la prueban y la cambian después.

Para Interfaces, el plan proponía seis tareas, cada una con su propio archivo:

ID	Empieza (plan)	Tarea planificada	Evidencia mínima
WEB-01	Álvaro	`index.html` y `base.css`: inicio, menú y cabecera	Inicio navegable
WEB-02	Omar	`catalogo.html` y `catalogo.css`: modelos, precios, tallas e imágenes	Catálogo coherente
WEB-03	Berme	`producto.html` y `producto.css`: ficha con variantes y descripción	Ficha legible
WEB-04	Álvaro	`custom.html` y `custom.css`: formulario para clubes	Solicitud completa
WEB-05	Omar	`contacto.html` y `responsive.css`: contacto y revisión móvil	Responsive probado
WEB-06	Berme	`custom.js`: validación y cálculo orientativo, si procede	Mejora opcional

El plan fijaba además seis hitos sin fechas (H1 Base, H2 Datos, H3 Aplicaciones, H4 Integración, H5 Seguridad y H6 Defensa), y para la web pedía «inicio, catálogo, ficha de producto, Custom y contacto; móvil y escritorio».

Lo que se hizo, según el repositorio y Git, coincide en el contenido pero no en la forma:

Tarea	Lo que hay en el repositorio	Commits (autor en Git)
WEB-01	Una sola página, `01_Interfaces/cliente/index.html`, y un CSS principal, `css/inicio.css`	`30e9d8d` (omar1676)
WEB-02	Catálogo de 100 modelos en la sección `#catalogo-completo`, con los datos en `js/modelos.js`	`1c536d1`, `b222eea` y `1121a1a` (omar1676); colores, `202c1d0` (Alvarobaz)

WEB-03	La ficha es un `<dialog>` de la misma página	`b222eea` y `f8611f9` (omar1676)
WEB-04	Sección `#custom` con el formulario simulado	`f8611f9` (omar1676)
WEB-05	Pie único `<footer id="contacto">`; las media queries, dentro de `inicio.css`	`f8611f9` y `27a7bc5` (omar1676)
WEB-06	`js/custom.js`: total de pares y envío simulado	`f8611f9` (omar1676)

Además se hicieron partes que el plan no recogía: la tienda (`f0b9985`), el panel de Álvaro (de `410a793` a `320e381`), el portal de Berme (`b8e5fe1`), el login único del personal (`6dc8343`), la página «Análisis de datos» (`5b37ff7`) y, al final, el paso de la web a BEM (`e0d2694`). Por el orden de los commits, primero se hizo la web básica, después el panel en la rama de Álvaro, luego la integración con el servidor y la tienda, más tarde el portal y, por último, la limpieza, el pie único y BEM. **[EQUIPO: explicar por qué se eligió una sola página en lugar de los seis archivos del plan; no consta en el repositorio.]**

> Las fechas de commit no sirven para el calendario. El 13/11/2026 la historia publicada en GitHub se reescribió con un *force-push*: se mantuvieron archivos, autores y mensajes, pero cambiaron las fechas y los identificadores (`06_Intermodular/EQUIVALENCIAS_COMMITS.md`). Por eso solo usamos el **orden** de los commits. **[EQUIPO: si se quiere un calendario por semanas, reconstruirlo con otra fuente.]**

Metodología de trabajo

El plan proponía un ciclo de cuatro momentos para cada tarea: **crear**, **revisar**, **cambiar** (otra persona mejora el trabajo) y **defender**. También proponía una rama y una *pull request* pequeña por tarea, un *worktree* por tarea activa y mensajes como `feat(web): …`.

El historial muestra un flujo algo distinto:

- **Ramas por persona** (`feature/omar`, `feature/alvaro` y `feature/berme`), fusionadas por `omar1676` (por ejemplo, `492c4e8`, «Merge Alvaro's branch…»). Los commits del portal de Berme se trajeron con `git cherry-pick -x`, que conserva su autor.
- **Tres* pull requests *a `main`**: la estructura inicial, la limpieza y la documentación del caso de Odoo.
- **Mensajes de commit**: los primeros siguen el formato del plan (`feat(web): build catalog and virtual try-on`); los siguientes, en inglés y sin prefijo.
- **Revisión cruzada**: cuando se tocó un archivo de otro, el cambio quedó apuntado para su autor en `servidor-admin/README.md` («Cambios en los archivos de Álvaro (los tiene que revisar él)» y lo mismo para Berme). **[ÁLVARO: confirmar si da por buenos esos cambios.] [BERME: lo mismo.]**
- **Pruebas tras cada cambio**, con los resultados guardados en carpetas de evidencias (apartado 3.1.5).

Un aviso: **el autor de un commit no demuestra quién hizo el trabajo**. Las fusiones las firmó `omar1676` y parte del trabajo pudo hacerse en pareja. El reparto real está sin completar en `06_Intermodular/memoria/MEMORIA.md` §9.2. **[EQUIPO: indicar quién empezó, quién revisó y quién defiende Interfaces.]**

Herramientas utilizadas

Herramienta	Versión	Para qué la usamos
HTML5, CSS3 y JavaScript	Sin librerías ni compilación	La web, la tienda, el panel y el portal
Visual Studio Code	**[EQUIPO: confirmar el editor y su versión; no consta en el repositorio]**	Escribir el código
Git y GitHub	—	Versiones, ramas, fusiones y *pull requests*
Microsoft Edge y sus DevTools	157	Navegador de pruebas y modo dispositivo (F12) para ver la web a 390 px
Playwright	1.65.0	Controlar Edge, recorrer la web y hacer las capturas
axe-core y Lighthouse	4.12.1 y 12.8.2	Accesibilidad (WCAG 2.1 A y AA) y buenas prácticas
Node.js, Express, pg y Nodemailer	24.15.0, 5.3.0, 8.23.1 y 10.5.0	Servidor de la tienda, el panel y el portal (`servidor-admin/`)
Lucide y Chart.js	1.62.0 y 4.4.1	Iconos y gráficos del panel y del portal, desde una CDN con `integrity`
Python y Streamlit	3.13.5 y 1.67.0	Servir la web estática (`http.server`) y el dashboard de «Análisis de datos»
`scripts/comprobar_web.py`	—	Comprobar que existen los recursos que enlaza la web (de Berme, `e352e0d`)

Playwright, axe-core y Lighthouse no son dependencias del proyecto, y sus scripts **no están en el repositorio**: solo sus resultados y capturas. En las sesiones de Omar se usaron Claude Code (Anthropic) y Codex (OpenAI) para escribir, revisar y probar código y documentación (`03_PSP/README.md`, «Autoría», y `06_Intermodular/ESTADO_ENTREGA_FINAL.md`). **[ÁLVARO: declarar si usó IA y para qué.] [BERME: lo mismo.]**

3.1.2 Descripción de la solución

La solución tiene dos partes. La primera es la **web pública de la marca** (`01_Interfaces/cliente/`), que es lo que pide el enunciado y funciona sin servidor: basta con abrir `index.html`. La segunda son las **pantallas que usan la base de datos** a través del servidor Node del proyecto (`servidor-admin/`): la tienda, el panel, el portal y «Análisis de datos». Son ampliaciones y no sustituyen a nada obligatorio.

Estructura de la web

La web es una sola página. Cada parte que pide el IM (IM-INT-04) es una sección con su ancla:

Parte que pide el IM	Dónde está (`index.html`)	Qué enseña
Inicio	Cabecera y «El escaparate» (líneas 24-152)	Logo, menú y hasta ocho zapatillas, con filtros por colección y buscador
Catálogo	`#catalogo-completo` (líneas 153-249)	Las 100 tarjetas con foto, nombre, precio y, con servidor, disponibilidad
Ficha de producto	`<dialog id="detalle">` (desde la	Colección, nombre, foto, descripción,

	línea 410)	precio, tallas y acceso al probador
Sección corporativa	`#historia`, «Nuestra visión» (líneas 250-266)	El mensaje de la marca
Servicio Custom	`#custom` (líneas 267-368)	Los cuatro pasos del proceso y el formulario simulado
Contacto	`<footer id="contacto">` (líneas 372-409)	Correo, dirección y horario ficticios, y «Volver arriba»

Cada archivo tiene una función: `css/inicio.css` (aspecto y móvil), `js/modelos.js` (datos), `js/inicio.js` (escaparate, catálogo, filtros, ficha y probador) y `js/custom.js` (formulario). La tienda añade `js/tienda.js` y `css/tienda.css`. Los datos del catálogo están en un solo sitio: `modelos.js` guarda el nombre, la colección y las imágenes de cada modelo, y su objeto `datosColeccion`, el precio orientativo, las tallas y la descripción de las seis colecciones. El **nombre** une cada modelo con su fila de PostgreSQL (`21_catalogo_web.sql`).

Diseño de marca

La identidad se basa en un fondo casi negro (`#171719`), texto claro (`--tinta: #f4f0ea`) y un acento anaranjado (`--brillo: #e5a168`), que marca el punto final de los títulos («El escaparate.», «Hablemos.»), las etiquetas y el foco. El catálogo y Custom invierten el esquema, con fondo claro (`#eeece6`), para que se note el cambio de zona. Cada título lleva encima una etiqueta numerada («04 / NOVA CUSTOM», «05 / CONTACTO») y los títulos crecen con la pantalla gracias a `clamp(2.4rem, 4.5vw, 5rem)`. El logo es una copia del que usa el panel de Álvaro, y al cargar la página aparece con una animación que se omite si el navegador pide movimiento reducido. La letra es Arial, a 14 px con interlineado 1,5. **[EQUIPO: si hubo una guía de estilo o un boceto previo, citarlo; no consta en el repositorio.]**

HTML semántico

La página usa la etiqueta adecuada para cada parte: `lang="es"` y un enlace «Saltar al escaparate» para el teclado; `<header>`, `<nav>`, `<main>`, `<section>` con `aria-labelledby` y un único `<footer>`; `<dialog>` para la ficha, el probador, el acceso, el carrito y el pedido; `<label>` en cada campo, `<fieldset>` con `<legend>` para las tallas, `<output>` para el total y `<dl>` para el contacto; y `role="status"`, `role="alert"` o `aria-live` en los mensajes que cambian.

Este fragmento es la cabecera (`01_Interfaces/cliente/index.html`, líneas 23-40). El logo es un enlace con nombre accesible y el menú lleva a las cuatro secciones:

```
<a class="saltar" href="#catalogo">Saltar al escaparate</a>
<header class="cabecera">
  <a
    class="marca marca--logo"
    href="#inicio"
    aria-label="NOVA Sneakers, inicio"
    ><img src="images/logo-nova.png" alt=""
  /></a>
  <nav aria-label="Navegación principal">
    <a href="#catalogo-completo">Catálogo</a
    ><a href="#historia">Nuestra visión</a
    ><a href="#custom">Custom</a
    ><a href="#contacto">Contacto</a>
  </nav>
  <a class="cabecera__enlace" href="#catalogo"
    >EXPLORAR <span aria-hidden="true">↗</span></a
```

```
  >
</header>
```

CSS con BEM y variables

Las clases de la web siguen **BEM**: `bloque`, `bloque__elemento` y `bloque--modificador`; por ejemplo, `catalogo`, `catalogo__tarjeta` y `catalogo__tarjeta--street`. El cambio se hizo al final, en `e0d2694` («Rename the shop web classes to BEM»); antes había nombres sueltos con guiones. El panel y el portal **conservan** sus clases con prefijo `adm-`.

Los colores van en **variables de CSS**. En `:root` están los tres generales, y cada colección define su pareja de tonos en su modificador, de modo que los mismos estilos sirven para las seis. Fragmento de `css/inicio.css` (líneas 1472-1481 y la regla de la línea 1502, recortada):

```
.catalogo__tarjeta--street,
.catalogo__filtro--street {
  --col-claro: #3d5a73;
  --col-oscuro: #16222d;
}
.catalogo__tarjeta--luxe,
.catalogo__filtro--luxe {
  --col-claro: #4a3d1f;
  --col-oscuro: #0e0d0a;
}
/* … casual, hombre, mujer y unisex, igual … */
.catalogo__tarjeta {
  border: 1px solid var(--col-oscuro);
  background: var(--col-oscuro);
  color: #fff;
}
```

JavaScript sin frameworks

El JavaScript es el del navegador, sin React ni otras librerías, cargado con `defer`. El IM lo plantea como evolución opcional (IM-INT-09). `mostrarCatalogo()` (`js/inicio.js`) crea cada tarjeta con `document.createElement` y escribe los textos con `textContent`; cada tarjeta es un `<button>` cuyo `aria-label` incluye nombre, precio y disponibilidad («Ver NOVA Street Vortex, 109,90 €, disponible»). `filtrarModelos()` sirve al escaparate y al catálogo, y `abrirDetalle()` abre la ficha con `showModal()` y, al cerrarla, devuelve el foco a la tarjeta.

En el formulario Custom, el navegador comprueba primero los campos obligatorios, el email y que las cantidades sean enteros no negativos (`min="0"` y `step="1"`). Después actúa `js/custom.js` (líneas 31-52), que cancela el envío real, exige al menos un par y escribe el mensaje de simulación:

```
formularioCustom.addEventListener("submit", (evento) => {
  evento.preventDefault(); // no hay servidor: el formulario no se envía
  const total = contarPares();
  if (total === 0) {
    resultadoCustom.textContent =
      "Indica cuántos pares quieres de al menos una talla.";
    camposTalla[0].focus();
    return;
  }
  const tallas = [];
  camposTalla.forEach((campo) => {
    if (Number(campo.value) > 0) {
      tallas.push(`${campo.dataset.talla}: ${campo.value}`);
    }
  });
```

```
  const club = formularioCustom.elements.club.value;
  const modelo = formularioCustom.elements.modelo.value;
  resultadoCustom.textContent =
    `SIMULACIÓN · Solicitud preparada para ${club}: ${modelo}, ${total} pares ` +
    `(tallas ${tallas.join(", ")}). No se ha enviado ningún dato ni se ha ` +
    "registrado ningún pedido.";
});
```

El formulario es un **prototipo**: el botón dice «SIMULAR ENVÍO», no sale ninguna petición de red y no se crea ningún pedido, ni en PostgreSQL ni en Odoo. SO024 (Club Halcones, Runner X, 40 pares) aparece solo como ejemplo, porque es el del enunciado.

Diseño responsive

`index.html` declara el `viewport` y `css/inicio.css` usa dos puntos de corte: **760/761 px**, entre móvil y escritorio, y **370 px**, para pantallas muy estrechas. En el código final hay 12 media queries con `max-width` (10 a 760 px y 2 a 370 px), 5 con `min-width: 761px` y 2 de `prefers-reduced-motion`; `tienda.css` añade otras 2. Con ellas:

- el catálogo y la estantería pasan de 4 columnas a 2 (CSS Grid), las secciones de dos columnas se apilan y las nueve tallas del formulario pasan a 3 por fila;
- el menú se queda en tres enlaces con zona táctil de 36 px; «Nuestra visión» se alcanza bajando;
- a 370 px o menos, el logo baja a 95 px.

También se usan Flexbox (cabecera, menú y botones), `clamp()`, `aspect-ratio` y `object-fit`. El panel y el portal tienen sus propios cortes, a 900 y 480 px.

> La memoria técnica común (`06_Intermodular/memoria/MEMORIA.md` §3.3) da otras cifras («8 con `min-width`», estantería de 5 columnas) de una versión anterior. **[EQUIPO: corregir esa tabla o citar estas, que salen del código final.]**

Accesibilidad

La referencia es **WCAG 2.1 nivel AA**, el estándar del W3C que miden axe-core y Lighthouse (`01_Interfaces/memoria/ESTRATEGIA_PRUEBAS_INTERFACES.md` §2). Además de lo dicho, todas las imágenes tienen `alt` (vacío en las decorativas); el foco se ve siempre, con un doble contorno en las tarjetas; la ficha se cierra con Escape; los errores salen junto a su campo y se anuncian, y si faltan pares el foco va a la primera talla; y los radios de color y talla de la tienda se manejan con las flechas del teclado.

La tienda conectada al servidor

La tienda es la misma web servida por `servidor-admin` en `http://127.0.0.1:3000/tienda/`; sin servidor, la ficha avisa de que no está disponible. `js/tienda.js` pide el catálogo real y añade una barra con «Iniciar sesión» y «Carrito (n)», un bloque de compra en la ficha (color, tallas con las agotadas tachadas, cantidad y «Añadir al carrito»), un carrito en `localStorage` con **solo el SKU y la cantidad**, cuentas de cliente, un pedido en tres pasos (entrega, **pago simulado** y resultado) y «Mis pedidos». El inicio de sesión es el diálogo de Álvaro (`74da21a`), al que solo se le cambiaron las dos últimas líneas del envío y se le añadió «Crear una cuenta».

El precio, el total y el stock los calcula PostgreSQL. El pago tiene dos botones, «Simular pago aceptado» y «Simular pago rechazado», y no pide datos bancarios. Al aceptar, el servidor manda un recibo con Resend o, en desarrollo, con Ethereal, un buzón de prueba que no entrega nada.

El panel de administración (Álvaro)

El panel (`01_Interfaces/admin/`) es, según Git, de `Alvarobaz`: estructura, páginas, gráficos y diseño responsive, desde `410a793` hasta `320e381`. Tiene 11 páginas suyas (Dashboard, Pedidos, Productos, Inventario, Clientes, Trabajadores, Sugerencias, Mejoras, Estadísticas, Auditoría y Configuración); Sugerencias, Mejoras y Configuración son **prototipos** con aviso. La conexión con PostgreSQL la hizo `omar1676` (`83a8567`). El panel solo lee, salvo el botón «Marcar en reparto» (`334cbc6`), que pasa un pedido confirmado a `enviado` mediante una función de la base. Con `git blame -C`, `admin/styles.css` tiene 748 líneas de Alvarobaz y 82 de omar1676; `admin/script.js`, 97 y 385, sobre todo por las llamadas a la API.

El portal del trabajador (Berme)

El portal (`01_Interfaces/trabajador/`) tiene cuatro páginas: Resumen, Pedidos, Inventario y Personalizados. Según Git, las páginas y sus rutas son de `Ivan Bermejo` (`b8e5fe1`); con `git blame -C`, `trabajador/script.js` tiene 300 líneas suyas y 29 de omar1676. Después, `omar1676` lo puso detrás del **login único del personal** (`6dc8343`, `1582689` y `12af0b4`): desde `login.html`, el administrador va al panel, el trabajador al portal y el comercial recibe «Acceso denegado». El trabajador avanza pedidos y corrige stock solo mediante dos funciones de la base que dejan la auditoría a su nombre. Su diseño no se cambió: se añadieron «Cerrar sesión» y nombres accesibles.

La página «Análisis de datos»

Es la duodécima entrada del menú del panel (`analisisAdmin.html` y `analisis.js`, `5b37ff7`, de omar1676). Muestra el dashboard de Python (`04_Python/app_dashboard.py`, con Streamlit) en un `<iframe>` con título. Antes, `analisis.js` comprueba que Streamlit responde en el puerto 8501 y, si no, explica cómo arrancarlo. El marco usa `?embed=true&embed_options=dark_theme`, sin la barra de Streamlit y con los colores del panel. Solo entra el administrador.

3.1.3 Requisitos del proyecto

Los requisitos salen del enunciado (IM §4.1) y de los criterios del módulo 0488 (IM p. 8, RA4 y RA8). Usamos los ID del índice del enunciado (`rubrica_IM_RT.md`) y el estado de la matriz de auditoría (`13_IM_ABCD.md` §1.1): **Cumplido**, con evidencia en el repositorio; **Para la demo**, hecho y por enseñar en clase; **Pendiente**, sin hacer.

ID	Requisito, en breve	Cómo se cumple	Estado
IM-INT-01	Mostrar empresa, catálogo y solicitud Custom	Web de una página con todas las secciones (`index.html`, líneas 51-368)	Cumplido
IM-INT-02	El cliente llega a Custom	Enlace en el menú; «menú → Custom» comprobado en Edge	Cumplido
IM-INT-03	Web responsive con identidad	Paleta, logo y etiquetas comunes; capturas a 1440 y 390 px	Cumplido
IM-INT-04	Inicio, catálogo, ficha, Custom y corporativa/contacto	Las seis partes del apartado 3.1.2	Cumplido
IM-INT-05	Responsive en móvil y escritorio	Media queries, Grid, Flexbox y `clamp()`; sin desplazamiento horizontal de 320 a 1440 px	Cumplido

IM-INT-06	Nombre, imagen, precio, descripción y tallas	100 modelos; precio, tallas y descripción por colección	Cumplido
IM-INT-07	Proceso Custom y formulario o prototipo	Cuatro pasos y formulario con pares de la 38 a la 46; envío simulado	Cumplido
IM-INT-08	Calidad de la interfaz	Componentes repetidos, BEM y variables; Lighthouse 100 y axe sin violaciones (antes de BEM)	Cumplido
IM-INT-09	JavaScript opcional	Filtros, buscador, total en vivo y validación	Cumplido
IM-INT-10	Demo en escritorio y móvil con una solicitud Custom	Con el modo dispositivo de Edge	Para la demo
IM-INT-11	¿Navegable, responsive y representativa?	Igual que IM-INT-04 y 05	Cumplido
IM-INT-12	¿Propuesta clara para pedidos Custom?	Igual que IM-INT-07	Cumplido
IM-INT-A1	Sin frameworks como React en el mínimo	HTML, CSS y JavaScript sin librerías	Cumplido
RA4 a)-b)	Estándares de usabilidad y accesibilidad	WCAG 2.1 AA, elegido y justificado	Cumplido
RA4 c)-g)	Menús, controles y legibilidad	Menú único, etiquetas, `fieldset`, foco visible y zonas táctiles de 36 px	Cumplido
RA4 h)-i)	Mensajes claros y pruebas de accesibilidad	Errores junto al campo; axe y Lighthouse	Cumplido
RA4 h)-i)	Pruebas de usabilidad con personas	Solo procedimiento y plantillas vacías (`PROCEDIMIENTO_USABILIDAD.md`)	**Pendiente**
RA8 a)	Estrategia de pruebas	`ESTRATEGIA_PRUEBAS_INTERFACES.md` §1-§5	Cumplido
RA8 b)	Pruebas de integración	Panel 23/23, tienda 39/39 y portal 24/24 con el servidor y PostgreSQL	Cumplido
RA8 g)	Documentar estrategia y resultados	Estrategia, carpetas de evidencias y este capítulo	Cumplido

Pendiente: la prueba de usabilidad con 3 a 5 personas ajenas al equipo. El procedimiento define tareas, métricas, cuestionario SUS y consentimiento, pero no hay ningún resultado. **[EQUIPO: hacer la prueba y añadir aquí sus resultados.]**

La auditoría propone además tres refuerzos: repetir Lighthouse sobre el código final, guardar en el repositorio los scripts de navegador y corregir las citas al commit antiguo `8cdc08c` (hoy `1bcbb10`).

3.1.4 Resultados del proyecto

Enseñamos la interfaz con capturas reales del repositorio. Las de `06_Intermodular/evidencias/cierre-entrega-20261113/` son del **código final** (`4f09742`); las demás, de la regresión del 13/11 (`1bcbb10`), **anterior a BEM**. Según el commit `e0d2694`, BEM no cambió el aspecto, pero esas capturas no se repitieron.

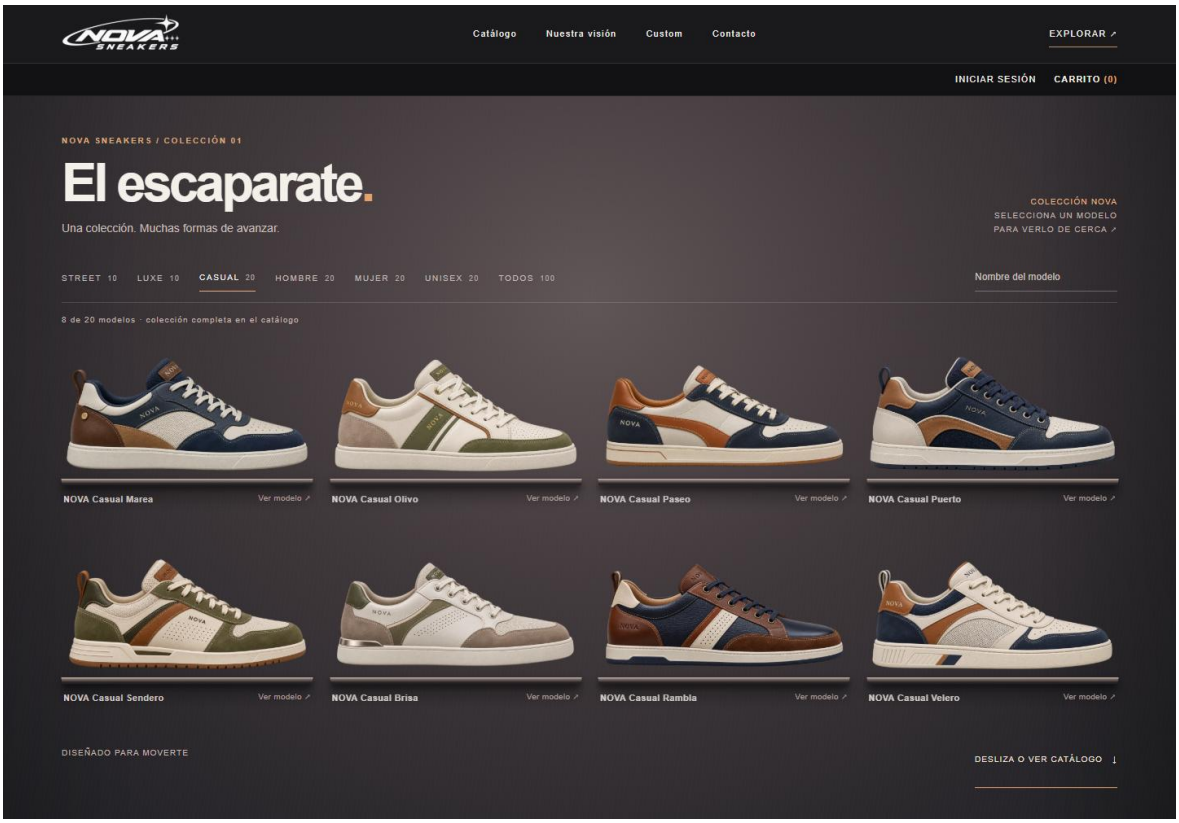


Figura 1. *Web de NOVA Sneakers en escritorio, a 1440 px (código final)*

La portada en escritorio: cabecera con logo, menú y «Explorar»; barra de la tienda con «Iniciar sesión» y «Carrito (0)»; y el escaparate con los filtros por colección, el buscador y ocho zapatillas Casual, con el aviso de que el resto está en el catálogo. Fuente: `cierre-entrega-20261113/web-1440.png`.

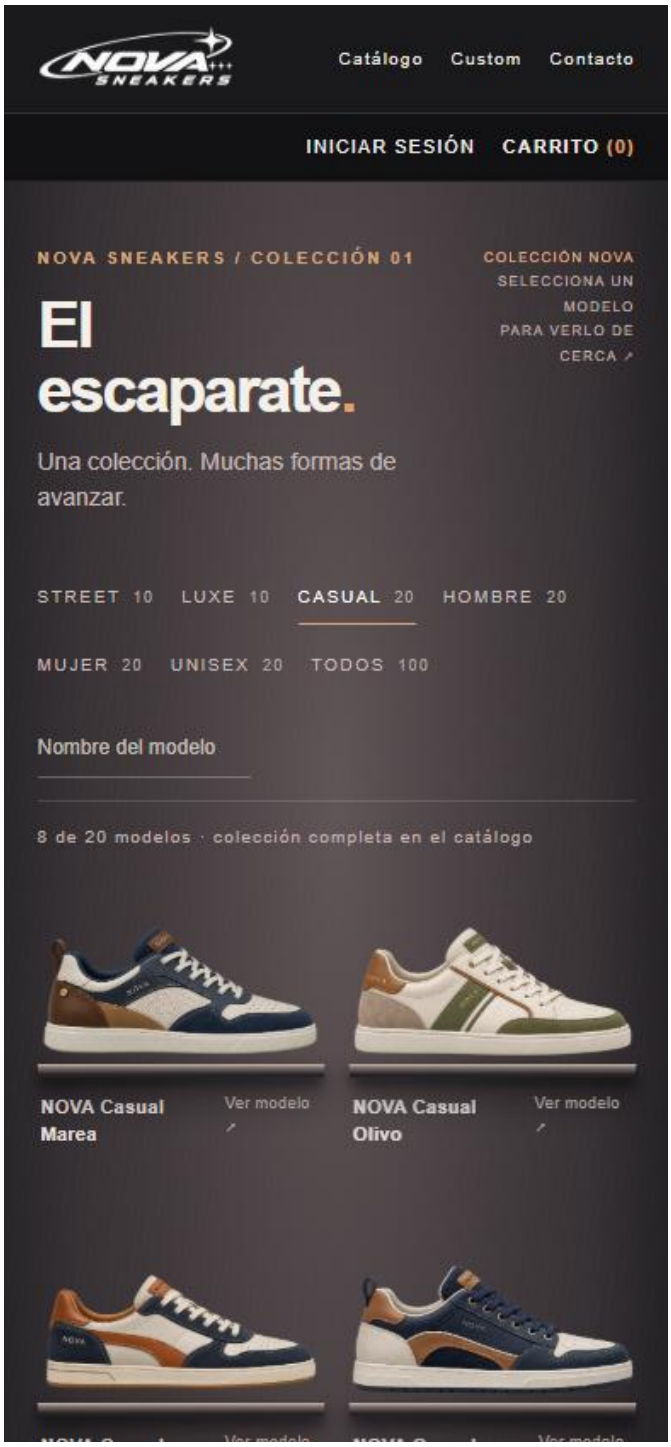


Figura 2. *La misma portada en el móvil, a 390 px (código final)*

A 390 px, el menú se queda en tres enlaces sin partirse, los filtros ocupan dos filas, el buscador baja y la estantería pasa a dos columnas, sin desplazamiento horizontal. Fuente: `cierre-entrega-20261113/web-390.png`.



Figura 3. *Catálogo con las tarjetas de la colección Hombre*

Cada tarjeta lleva el color de su colección, la etiqueta en una pastilla oscura y una banda opaca con nombre, precio y disponibilidad. Fuente: `regresion-final/05_web/capturas_interfaces/escritorio_01_catalogo_con_precios.png`.

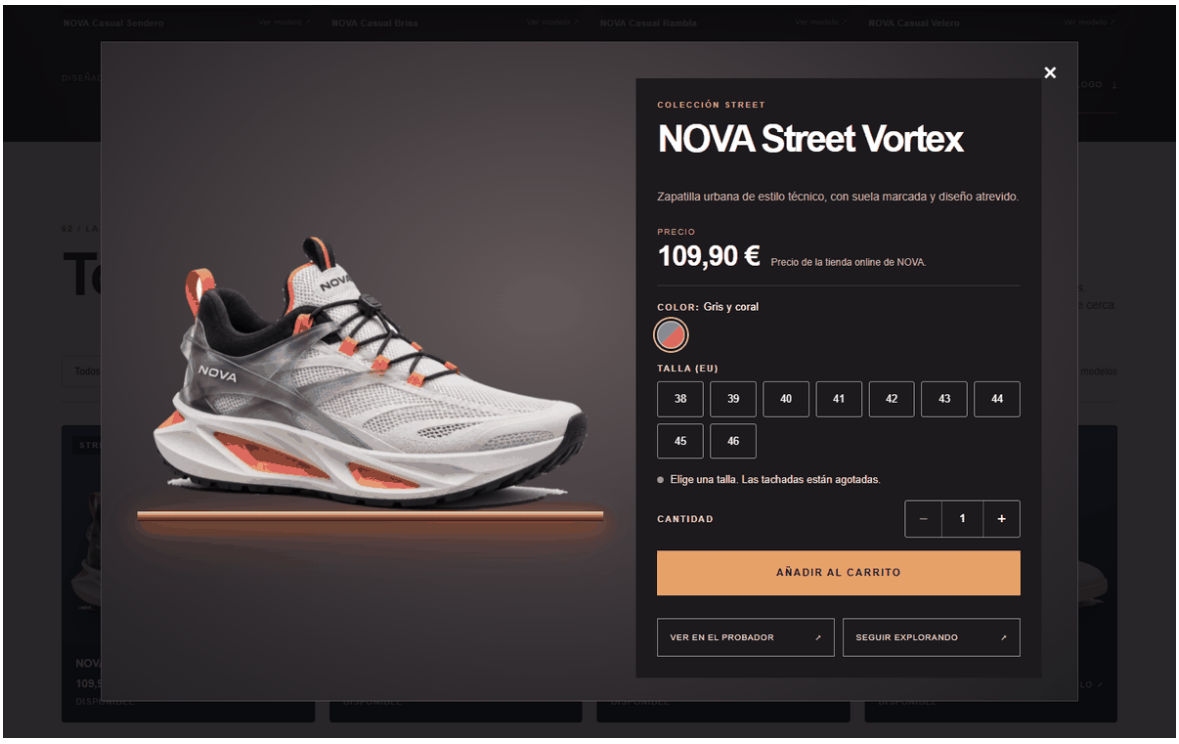


Figura 4. *Ficha de producto con el bloque de compra*

La ficha de NOVA Street Vortex: colección, nombre, descripción, precio, color, cuadrícula de tallas, cantidad, «Añadir al carrito», «Ver en el probador» y «Seguir explorando». Fuente: `regresion-final/05_web/capturas_interfaces/escritorio_02_ficha_street.png`.



Figura 5. *Formulario Custom después de simular el envío*

A la izquierda, los cuatro pasos del proceso y el ejemplo SO024; a la derecha, el aviso de prototipo, los campos con etiqueta, las tallas en un `fieldset`, el total (40 pares) y el mensaje «SIMULACIÓN · Solicitud preparada…». Los datos son de prueba. Fuente: `regresion-final/05_web/capturas_interfaces/escritorio_04_custom_simulado.png`.

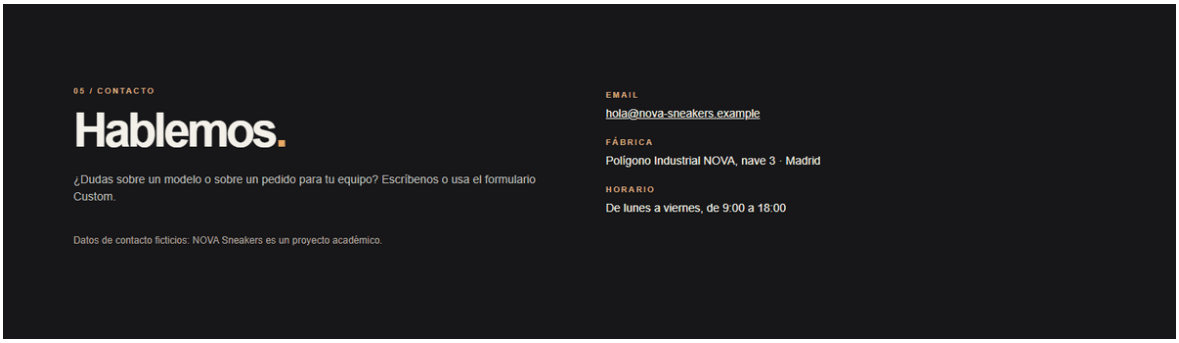


Figura 6. *Contacto dentro del único pie de página*

El contacto ya forma parte del único `<footer>`: correo, fábrica y horario ficticios y el aviso de proyecto académico. Fuente: `regresion-final/05_web/capturas_interfaces/escritorio_03_contacto.png`.

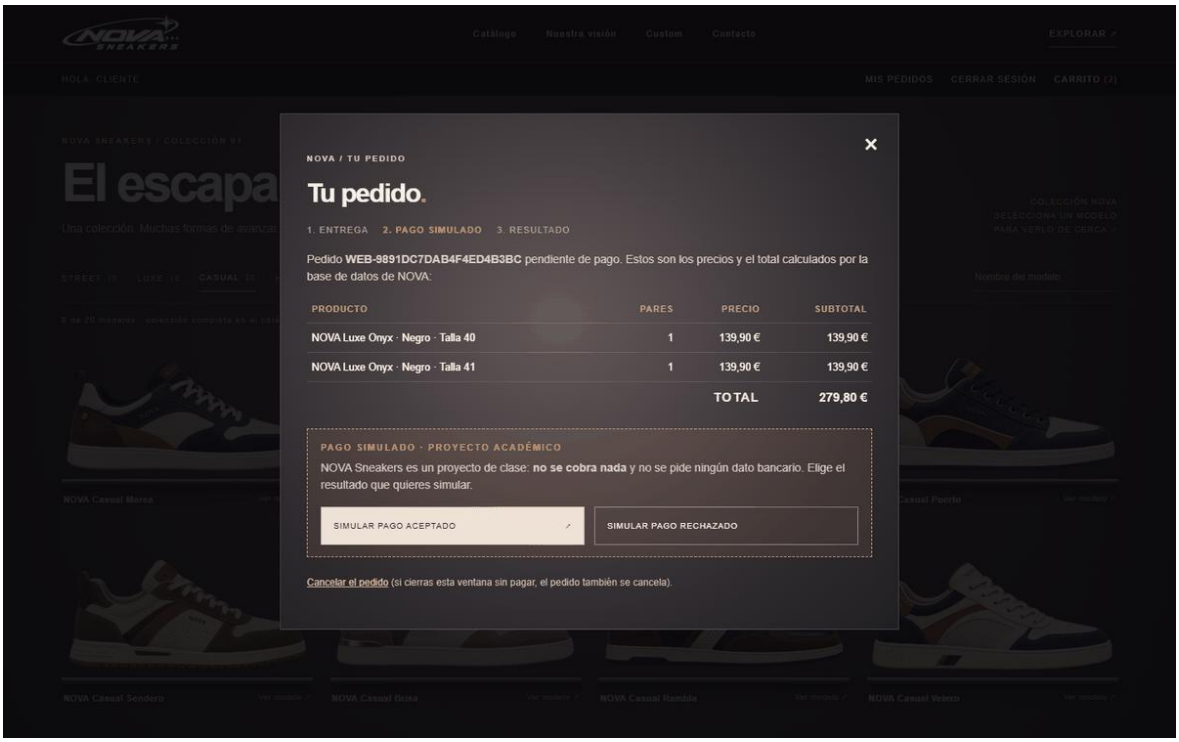


Figura 7. *Tienda: paso 2 del pedido, con el pago simulado*

El pedido `WEB-…` ya existe como pendiente, con las líneas y el total que calculó PostgreSQL (279,80 €). El recuadro «Pago simulado · Proyecto académico» explica que no se cobra nada y ofrece los dos botones. Fuente: `01_Interfaces/evidencias/tienda/capturas-2026-11-13/escritorio_08_checkout_pago_simulado.png`.

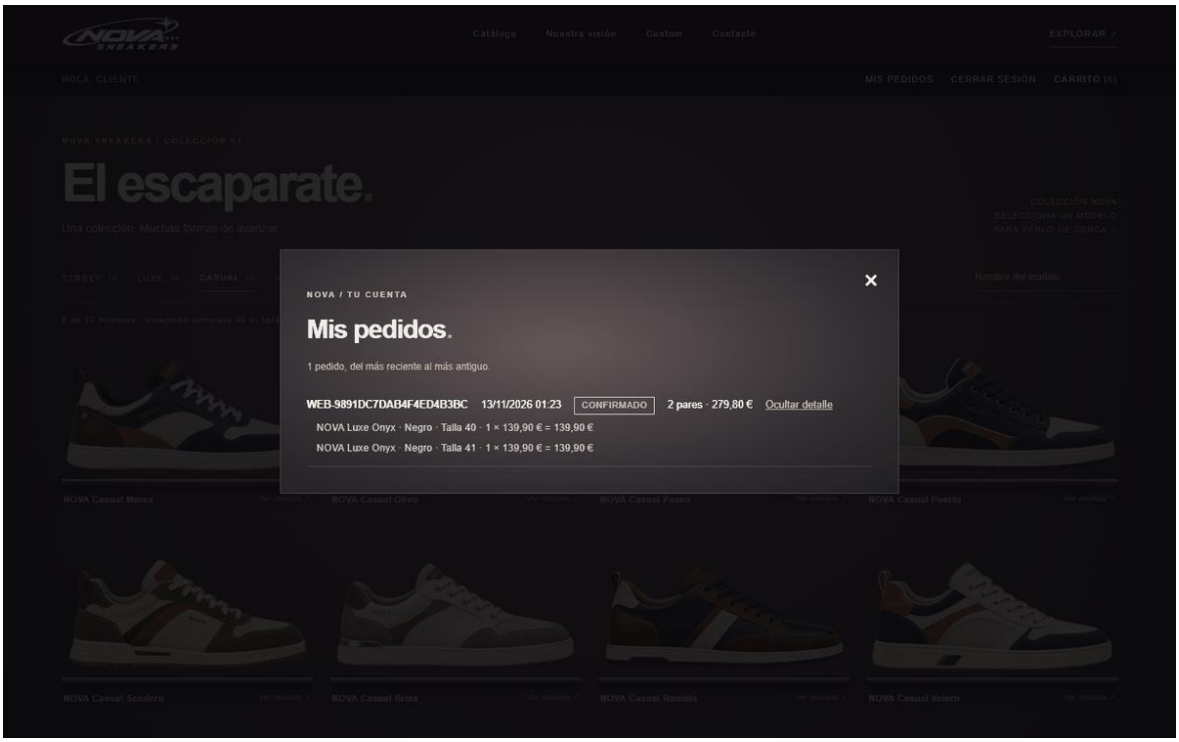


Figura 8. *Tienda: «Mis pedidos» con el pedido confirmado*

Tras aceptar el pago simulado, el pedido aparece «Confirmado» con su fecha, pares, total y detalle. Fuente: `tienda/capturas-2026-11-13/escritorio_10_mis_pedidos.png`.

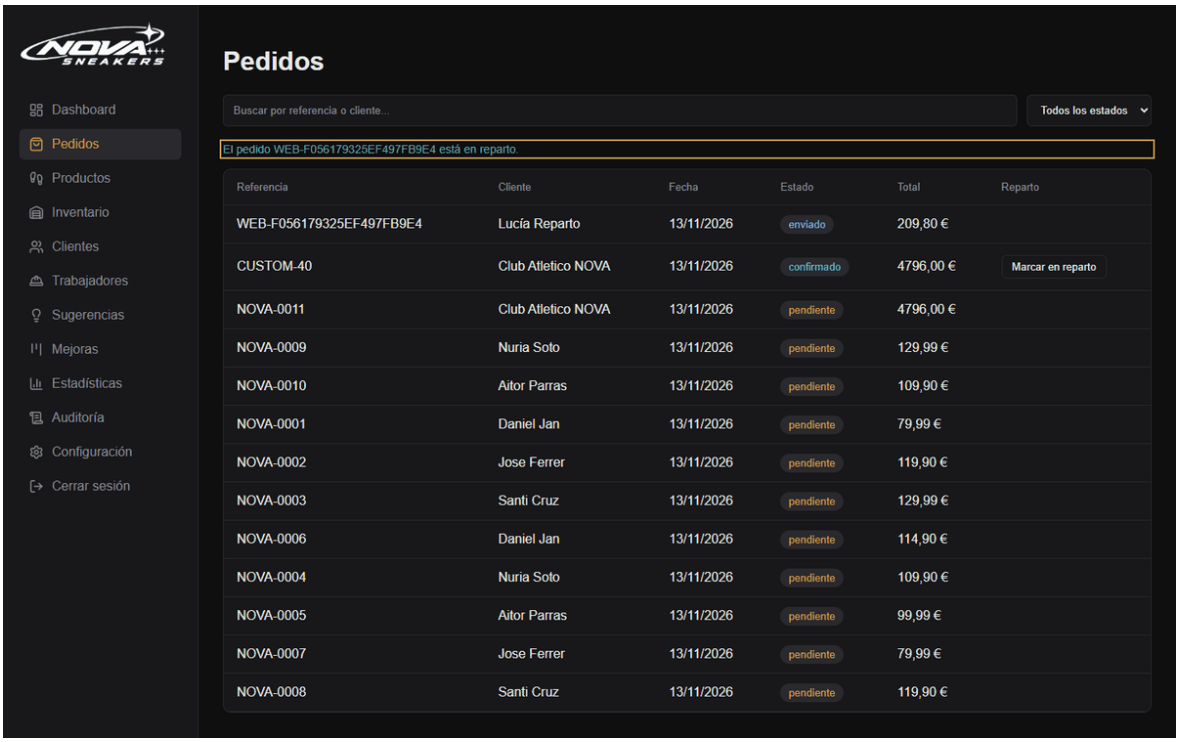


Figura 9. *Panel de administración: Pedidos, después de «Marcar en reparto»*

La página Pedidos del panel de Álvaro con una base de prueba. El aviso confirma que el pedido web pasó a «enviado»; el botón solo aparece en los pedidos confirmados, como CUSTOM-40. Fuente: `01_Interfaces/evidencias/reparto-y-correo/capturas/panel_tras_marcar_en_reparto.png`.

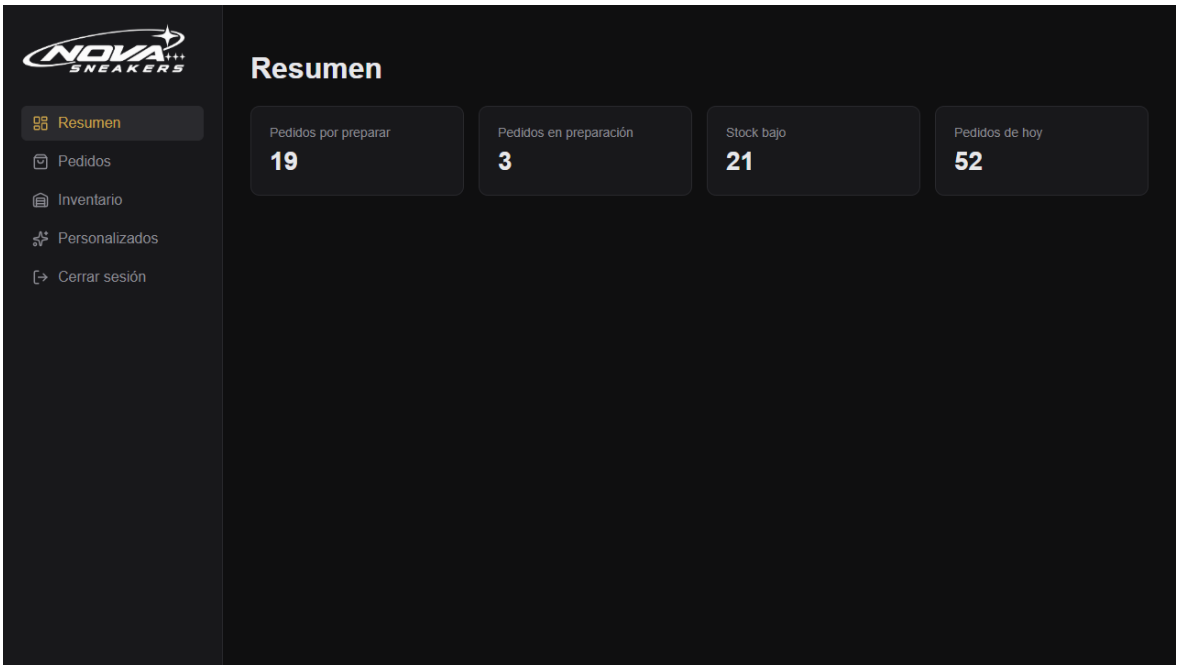


Figura 10. *Portal del trabajador: Resumen (código final)*

El resumen del portal de Berme con la cuenta de prueba del trabajador: pedidos por preparar y en preparación, stock bajo y pedidos de hoy, con el menú de sus cuatro páginas. Fuente: `cierre-entrega-20261113/portal-trabajador.png`.

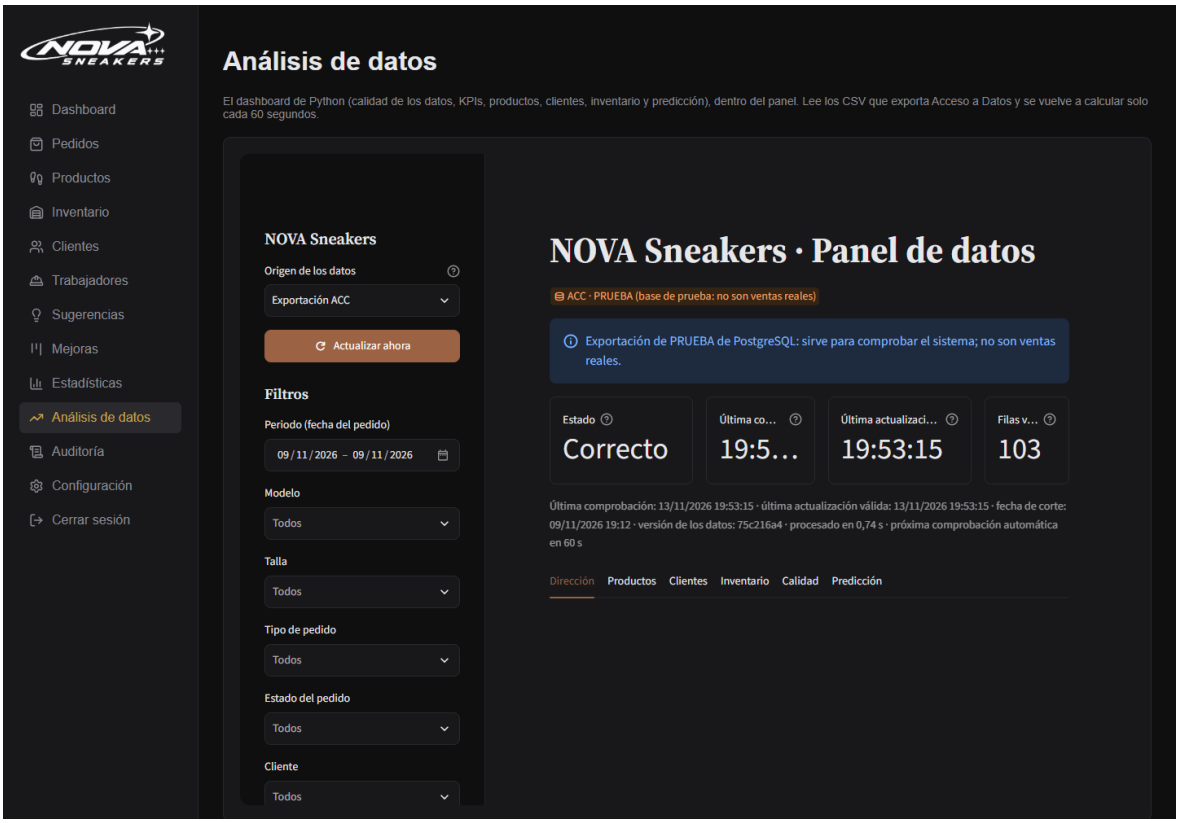


Figura 11. *«Análisis de datos»: el dashboard de Python dentro del panel (código final)*

El dashboard de Streamlit dentro del panel, con tema oscuro: filtros, el aviso de que los datos son una exportación de PRUEBA, el estado de la última comprobación y las seis pestañas. Fuente: `cierre-entrega-20261113/dashboard-integrado-1440.png`.

No hay captura de Lighthouse: su resultado está guardado como informe HTML (`01_Interfaces/evidencias/web/lighthouse/`), y una captura hecha ahora no sería la de la medición del 11/11. **[EQUIPO: si se repite Lighthouse sobre el código final, añadir su captura.]**

3.1.5 Pruebas

La estrategia está en `01_Interfaces/memoria/ESTRATEGIA_PRUEBAS_INTERFACES.md`, que fija para cada requisito qué se comprueba y su criterio de aceptación (por ejemplo, «0 violaciones de axe» para IM-INT-08). Hay pruebas funcionales en Edge con Playwright, en 4 anchos (1440, 820, 390 y 360 px) y en 11 anchos de 320 a 1440 px para la cabecera; de accesibilidad, con axe-core, Lighthouse y el teclado; de integración, con la web, el servidor y PostgreSQL; y de regresión, repitiéndolo todo tras cada cambio.

Hay que distinguir sobre qué código se hizo cada prueba, porque BEM llegó al final:

Prueba	Código y fecha	Resultado	Evidencia
Edge: catálogo de 100 modelos, BEM y ancho, login del administrador e iframe, a 1440 y 390 px; login y datos del portal	**Final** (`4f09742`), 13/11	9 de 9	`cierre-entrega-20261113/web-navegador.json`
Recursos locales	**Final**, 13/11	21 páginas HTML	`cierre-entrega-20261113/web_recursos.txt`

(`comprobar_web.py`)		y 155 recursos	
Node: panel, tienda y portal	**Final**, 13/11	23/23, 39/39 y 24/24	`cierre-entrega-20261113/node_*.txt`
Edge: mejoras, Interfaces y web completa	Antes de BEM (`1bcbb10`), 13/11	81/81, 136/136 y 114/114	`06_Intermodular/evidencias/regresion-final/05_web/`
Edge: compra, reparto y portal	Antes de BEM, 13/11	139/139, 29/29 y 82/82	Ídem
Las mismas, en la rama de limpieza (`ec8837a`)	Antes de BEM, 13/11	Los mismos recuentos	`06_Intermodular/evidencias/regresion-limpieza/`
«Análisis de datos», con y sin Streamlit	Rama de limpieza, 13/11	43/43 y 2/2	`01_Interfaces/evidencias/panel-admin/analisis-datos/`
axe en 5 estados de la web	11/11 y 13/11, antes de BEM	0 violaciones	`evidencias/web/resultados/` y `02_interfaces.txt`
Lighthouse 12.8.2, escritorio y móvil	11/11, antes de BEM	Accesibilidad 100 y buenas prácticas 100	`01_Interfaces/evidencias/web/lighthouse/`
Contraste de cada texto en Edge	12/11	212 de 212 cumplen AA	Informe de integración (fuera del repositorio)

Extracto de `06_Intermodular/evidencias/regresion-final/05_web/02_interfaces.txt` (líneas 128-136), con axe en los estados de la web, anterior a BEM:

```
OK    [axe] nueva_escritorio: ninguna violación WCAG A/AA nueva respecto a la web anterior ->
ninguna
OK    [axe] nueva_movil: ninguna violación WCAG A/AA nueva respecto a la web anterior -> ninguna
OK    [axe] nueva_escritorio_ficha_abierta: ninguna violación WCAG A/AA nueva respecto a la web
anterior -> ninguna
OK    [axe] nueva_escritorio_formulario_enviado: 0 violaciones WCAG 2.1 A/AA -> ninguna
OK    [axe] nueva_escritorio_ficha_abierta: 0 violaciones WCAG 2.1 A/AA -> ninguna
OK    [axe] nueva_escritorio_probador_abierto: 0 violaciones WCAG 2.1 A/AA -> ninguna
OK    [axe] nueva_movil: 0 violaciones WCAG 2.1 A/AA -> ninguna
OK    [axe] nueva_escritorio: 0 violaciones WCAG 2.1 A/AA -> ninguna
OK    [axe] formulario Custom y contacto (con el resultado visible): ninguna violación WCAG A/AA
-> ninguna
```

El mensaje de `e0d2694` dice que, tras BEM, los estilos calculados no cambiaron y las pruebas dieron los mismos recuentos (81, 136, 114, 139, 29 y 82), pero esos resultados **no están guardados**. Sobre el código final solo constan los 9 controles de Edge del cierre.

No probado o no repetido: Lighthouse y la medida de contraste tras BEM; la usabilidad con personas (**pendiente**); un lector de pantalla y un móvil real (el «móvil» es Edge a 390 px). Además, los scripts de Playwright no están en el repositorio.

3.1.6 Problemas encontrados y soluciones

El doble pie de página

La sección de contacto y el pie eran dos bloques oscuros seguidos, y los dos decían que el proyecto es académico: parecían dos *footers*. En `27a7bc5` se unieron en un solo `<footer id="contacto">` con los datos de contacto y una línea final con la marca, el lema, «Volver arriba» y el aviso de datos ficticios. El ancla `#contacto` siguió funcionando (figura 6).

El cambio a BEM al final

Pasar las clases a BEM tan tarde (`e0d2694`) podía romper estilos o scripts que buscan clases. Por eso se mantuvieron los `id`, el diseño y el comportamiento, y se compararon los estilos calculados antes y después. Quedan dos cosas: el panel y el portal siguen con sus clases `adm-`, y algunas clases renombradas vienen del diálogo y los colores de Álvaro. **[ÁLVARO: revisar esos nombres.]**

El contraste del rediseño de colores

El rediseño de Álvaro (`202c1d0`) ponía texto blanco sobre degradados. Medido en Edge, la etiqueta se quedaba en 2,9:1, el precio entre 1,0 y 1,8:1 y el foco en 1,3:1, cuando AA pide 4,5:1 para el texto y 3:1 para el foco. En `1121a1a` se mantuvo su paleta, pero la etiqueta pasó a una pastilla oscura, la banda del precio se hizo opaca y el foco ganó un contorno oscuro. Después, las 212 medidas cumplieron AA. **[ÁLVARO: confirmar esta adaptación.]**

Otros fallos que encontraron las pruebas

- **Probador:** las tres complexiones estaban ocultas y el peso no hacía nada; Playwright se quedaba esperando una opción oculta. Se arregló en `1121a1a`.
- **CSS que nunca se aplicaba:** 52 declaraciones anuladas por reglas posteriores y cuatro capas de reglas del probador. Se quitaron en `252f5fc`, sin cambios en los estilos calculados de 126 estados de la página.
- **Accesibilidad del portal:** axe halló 940 campos sin etiqueta y un desplegable sin nombre en el original; con nombres accesibles, en escritorio quedó sin violaciones. En el móvil, «Personalizados» y algunas tablas del panel mantienen avisos `scrollable-region-focusable`. **[ÁLVARO y BERME: hacer enfocables esas tablas.]**
- **Etiqueta del acceso a la tienda:** decía «Usuario o correo», pero solo acepta correo. Se cambió a «Correo electrónico» como corrección de texto posterior al código probado. **[ÁLVARO: dar el visto bueno.]**
- **Puerto 8501 ocupado en el cierre:** Streamlit se arrancó en otro puerto y solo el navegador de la prueba redirigió el 8501, sin tocar el código.

Limitaciones que siguen abiertas

- El formulario Custom **simula** el envío y no crea pedidos.
- Precios, tallas y stock de los 100 modelos son **provisionales** y hay que acordarlos con Odoo.
- El pago de la tienda es **simulado** y el recibo nunca se ha mandado por el Resend real.
- Sin internet, el panel y el portal se quedan sin iconos ni gráficos (CDN).
- El portal considera stock bajo menos de 5 pares y el panel, menos de 10.
- `imagenCatalogo` y `catalogo__tarjeta--con-fondo` siguen en el código sin usarse.
- Faltan la prueba con personas, repetir Lighthouse y guardar los scripts de navegador.

3.2 Programación de Servicios y Procesos (PSP)

En este apartado describimos el subsistema auxiliar de PSP: una aplicación Java de consola que recibe la exportación CSV de un pedido Custom y la procesa con tres procesos hijos. El código está en la carpeta `03_PSP/` del repositorio, y la versión que describimos es la que se probó en la regresión de cierre del 13/11, sobre el commit `4f09742`.

3.2.1 Planificación y metodología del proyecto

Lo planificado: el plan operativo del equipo

En octubre de 2026 preparamos un plan de trabajo común, `Sneaker_Factory_Reparto_Tareas_Git_Worktrees.pdf`. No es un enunciado, sino una propuesta del equipo («el reparto y los nombres de clases son una propuesta organizativa»), así que aquí lo tratamos como lo planificado, no como el reparto real.

Su regla general era que «los tres tocan todo». Cada tarea la empieza una persona, los otros dos la revisan, una persona distinta introduce después una mejora y los tres la defienden. Para PSP, el ejemplo de rotación era este: «Berme prepara el proceso principal. Omar modifica gestión de errores; Álvaro prueba IPC y añade casos». El plan dividía PSP en seis tareas (p. 5):

ID	Empieza	Tarea y clase	Evidencia mínima
PSP-01	Berme	`ProcesadorPedidosCustom.java`: entrada y proceso coordinador	Padre ejecutable
PSP-02	Omar	`ValidadorPedidoCustom.java`: campos, tallas y cantidades	Entrada controlada
PSP-03	Álvaro	`AnalizadorTallas.java`: agrupación por talla/variante	Resumen correcto
PSP-04	Berme	`GeneradorFichaTaller.java`: ficha auxiliar para taller	Archivo generado
PSP-05	Omar	`GestorProcesos.java`: ProcessBuilder, stdin/stdout, códigos, tiempos	IPC comprobada
PSP-06	Álvaro	`ResultadoProceso.java` y pruebas negativas: errores y retornos	Fallos controlados

Los hitos del plan (p. 8) situaban «PSP básicos» en el H3, con cada bloque funcionando de forma aislada. En el H4 pedían que «SO024 recorre los módulos con la misma referencia», y en el H5, «pruebas positivas y negativas con evidencia». Una tarea solo estaba terminada cuando funcionaba, tenía prueba y evidencia, la habían revisado los compañeros y los tres sabían explicarla. SO024 es el ejemplo de pedido del enunciado, no un identificador obligatorio, y no existe ninguna venta SO024 en Odoo.

Lo hecho según el repositorio

Las fechas de los commits no sirven para reconstruir el calendario. El 13/11 la historia publicada en GitHub se reescribió y cambiaron las fechas y los identificadores (`06_Intermodular/EQUIVALENCIAS_COMMITS.md`). Por eso contamos el trabajo por su contenido, con los identificadores publicados que cita `03_PSP/README.md` en su apartado «Autoría»:

7. **Primera versión, con dos hijos** (commit `f5426cb`, de `omar1676`): `ValidadorPedidoCustom`, `LectorCsv`, `LanzadorValidacion` y un `AnalizadorTallas` en la raíz de `03_PSP/`, que solo agrupa por talla.
8. **El paquete de Álvaro** (commit `e71947e`, de `Alvarobaz`): la carpeta `03_PSP/src/Psp/`, con su analizador, su lanzador, `ResultadoProceso` y sus pruebas. Su rama se fusionó en `main` con el commit `492c4e8`, y los archivos son idénticos a los de su commit.
9. **El coordinador con tres hijos** (commit `9dc66ac`, de `omar1676`): `ProcesadorPedidosCustom`, `GestorProcesos`, `GeneradorFichaTaller`, las pruebas, los datos de ejemplo, las evidencias y los cambios del validador (datos normalizados y salida en UTF-8). Según el README, «se prepararon en la sesión de Omar con ayuda de Claude».

Berme subió el comprobador `scripts/comprobar_psp.py` (commit `f1983d2`), el CSV sintético multilínea `03_PSP/pruebas/custom40.csv` (`70bfc8d`) y `06_Intermodular/SO024/contrato-fabricacion.md` (`0f7d27d`), que describe los formatos de entrada. Su arreglo del comprobador, `4f09742`, se explica en el apartado 3.2.6.

Tarea	Planificado	Lo que hay en el repositorio
PSP-01	Berme	`ProcesadorPedidosCustom`, subido por `omar1676` en `9dc66ac`
PSP-02	Omar	`ValidadorPedidoCustom` y `LectorCsv` (`f5426cb`), cambiados en `9dc66ac`
PSP-03	Álvaro	`Psp.AnalizadorTallas` (`e71947e`), usado sin cambios como hijo 2
PSP-04	Berme	`GeneradorFichaTaller`, subido por `omar1676` en `9dc66ac`
PSP-05	Omar	`GestorProcesos` (`9dc66ac`)
PSP-06	Álvaro	`ResultadoProceso` y `PruebasAnalizador` (`e71947e`); el coordinador no usa `ResultadoProceso`, sino `GestorProcesos`

El autor de un commit no prueba quién hizo el trabajo, y el README deja pendiente de acordar el reparto: PSP-01 y PSP-04 eran tareas de Berme en el plan y las subió Omar. **[EQUIPO: reparto real de PSP y quién revisa y defiende cada clase en la defensa]**

En cuanto a la metodología, trabajamos de forma incremental: primero una versión pequeña con dos hijos y después el coordinador con tres. No reescribimos el analizador: reutilizamos la clase de Álvaro tal como estaba, y el validador le prepara la entrada en el formato que ella ya leía. También partimos de los ejemplos del profesor que cita el README: `EstadisticasHijo2.java`, `LanzadorEstadisticas.java`, `EG1_PipelineTexto` y `LanzadorProcesoFinal2.java`.

El caso de prueba también cambió. El plan vinculaba PSP a SO024, pero lo que procesa PSP es CUSTOM-40, un pedido confirmado de PostgreSQL. SO024 solo se ha ensayado con datos inventados en una base desechable, y no se ha procesado ninguna exportación de Odoo.

Herramientas utilizadas

- **JDK 17 (probado con el 17.0.12), sin Maven ni librerías.** Usamos `ProcessBuilder`, `Process` (`waitFor` con límite, `exitValue`, `destroy` y `destroyForcibly`), los métodos de Java 17 que abren las tuberías con un juego de caracteres (`outputWriter`, `inputReader` y `errorReader`) y la clase `Thread`.
- **`javac` y `java` por consola**, en PowerShell sobre Windows 11. En el ordenador de pruebas no estaban en el PATH, así que se llaman con la ruta de `JAVA_HOME`.
- **PostgreSQL 17 y `psql`**, para generar el CSV de entrada con el script 15.
- **Python 3.13**, para el comprobador rápido de Berme.
- **Git y GitHub**, con ramas personales que se fusionan en `main`. Para probar el analizador de Álvaro antes de la fusión, se sacó de su rama con `git archive` (`03_PSP/evidencias/README.md`).
- **Inteligencia artificial.** En las sesiones de Omar se usaron Claude Code (Anthropic) y Codex (OpenAI) para escribir, revisar y probar código y documentación. Lo que usaron Álvaro y Berme está sin declarar. **[ÁLVARO: herramientas de IA usadas en PSP, si las hubo]** **[BERME: herramientas de IA usadas en PSP, si las hubo]**
- **Editor o IDE.** **[EQUIPO: indicar el editor o IDE que usó cada uno, si se quiere citar]**

Estas son las órdenes para compilar y procesar el pedido desde la carpeta `03_PSP` (`03_PSP/README.md`, apartados «Compilar» y «Procesar el pedido»):

```
& "$env:JAVA_HOME\bin\javac.exe" -encoding UTF-8 -d clases *.java pruebas\*.java src\Psp\*.java
& "$env:JAVA_HOME\bin\java.exe" -cp clases ProcesadorPedidosCustom
datos\custom40_fabricacion_psp.csv
```

3.2.2 Descripción de la solución

Hemos desarrollado un procesador auxiliar de pedidos Custom: una aplicación Java de consola que recibe la copia CSV de un pedido, la valida, agrupa sus pares por variante y por talla, y prepara una ficha para el taller. No escribe en PostgreSQL ni en Odoo, no toca el stock y no crea la orden de fabricación: solo lee una copia exportada y genera archivos en `salidas/` (`03_PSP/README.md`, línea 11). La propia ficha lo dice en su pie: «No sustituye a la orden de fabricacion de Odoo» (`GeneradorFichaTaller.java:174-176`).

Arquitectura: un proceso padre y tres hijos

Clase	Papel	Qué hace	Autoría según el repositorio
`ProcesadorPedidosCustom`	Proceso padre	Comprueba el archivo, lanza los hijos uno detrás de otro, les pasa los datos, revisa sus códigos y respuestas, y guarda la ficha y el registro	Commit `9dc66ac`, preparado en la sesión de Omar con ayuda de Claude
`GestorProcesos`	Apoyo del padre	Lanza un hijo, le escribe por stdin, recoge stdout y stderr por separado, espera con límite y lo cancela si tarda	Ídem
`ValidadorPedidoCustom` y	Hijo 1	Comprueba el CSV y	Commit `f5426cb`,

`LectorCsv`		devuelve los datos normalizados	cambiado en `9dc66ac`
`Psp.AnalizadorTallas`	Hijo 2	Agrupa los pares por modelo, color y talla, y por talla	Álvaro (commit `e71947e`), sin cambios
`GeneradorFichaTaller`	Hijo 3	Comprueba que las cantidades cuadran y escribe la ficha	Commit `9dc66ac`, como el padre

Cada hijo se ejecuta en otra máquina virtual de Java. El comando se construye con el mismo `java` y el mismo classpath que el padre, como en los ejemplos del profesor, para que el hijo encuentre sus clases (`GestorProcesos.java:52-55`):

```
        String java = Path.of(System.getProperty("java.home"), "bin", "java").toString();
        String classpath = System.getProperty("java.class.path");

        ProcessBuilder pb = new ProcessBuilder(java, "-cp", classpath, clase);
```

Los hijos no se hablan entre ellos: todo pasa por el padre, que decide en cada paso si el flujo continúa.

Flujo del procesamiento

El recorrido de un pedido es este (`ProcesadorPedidosCustom.java`):

10. **Comprobar el archivo.** Antes de lanzar nada, el padre comprueba que el argumento existe, es un archivo, acaba en `.csv` y no está vacío (líneas 210-241), y lo lee entero en UTF-8 (líneas 69-80).
11. **Paso 1, validar.** Envía el CSV al validador por stdin (líneas 85-87) y saca de su primera línea la referencia y el total de pares (líneas 90-101).
12. **Separar los datos.** Aparta el bloque `PEDIDO`, `LINEA` y `FIN` para el analizador, y guarda `CLIENTE` y `PERSONALIZACION` para la ficha (líneas 109-120).
13. **Paso 2, agrupar.** Envía el bloque al analizador (líneas 125-127) y comprueba que su `TOTAL` coincide con los pares del validador (líneas 134-148).
14. **Paso 3, la ficha.** Envía al generador el pedido, el archivo de origen, el cliente, la personalización y el resumen del analizador (líneas 157-173).
15. **Terminar.** Guarda la ficha en `salidas/ficha_taller_<referencia>.txt` (líneas 178-200), escribe la línea `Resultado: …`, añade el resumen al registro y sale con su código (líneas 325-354).

Después de cada paso, `comprobarPaso` (líneas 252-316) revisa el código de salida, que se hayan enviado todos los datos y que la respuesta empiece por `OK;` y hable del mismo pedido. Si algo falla, llama a `terminar()`, que acaba el programa, y el paso siguiente no se lanza. Este es el control de la respuesta (líneas 301-313):

```
        String respuesta = primeraLinea(paso.getSalida());
        String[] partes = respuesta.split(";");

        boolean valida = respuesta.startsWith("OK;") && partes.length >= 2;

        if (valida && referencia != null && !partes[1].equals(referencia)) {
            valida = false;
        }

        if (!valida) {
```

```
            anotar("  Respuesta recibida: " + respuesta);
            terminar(FALLO_PROCESO, "La respuesta del paso " + numero + " no es la esperada");
        }
```

Protocolo de cada hijo: stdin y stdout

Los datos van por la entrada y la salida estándar, y no como argumentos, porque un CSV entero no cabe cómodamente en la línea de órdenes. Cada hijo lee hasta que el padre cierra su entrada.

Hijo	Recibe por stdin	Devuelve por stdout si todo va bien
1 · `ValidadorPedidoCustom`	El CSV entero, tal cual	`OK;referencia;líneas;pares`, `CLIENTE;…`, una línea `PERSONALIZACION;…` por cada línea del texto, `PEDIDO;referencia`, una `LINEA;sku;modelo;color;talla;cantidad` por registro y `FIN`
2 · `Psp.AnalizadorTallas`	`PEDIDO;referencia`, las líneas `LINEA;…` y `FIN`	`OK;referencia`, una `VARIANTE_TALLA;modelo;color;talla;pares` por combinación, una `TALLA;talla;pares` por talla y `TOTAL;pares`
3 · `GeneradorFichaTaller`	`PEDIDO`, `ORIGEN`, `CLIENTE`, `PERSONALIZACION`, las líneas del analizador y `FIN`	`OK;referencia` seguido de la ficha

El validador lee toda la entrada de golpe, porque un registro puede ocupar varias líneas si un campo entre comillas lleva un salto de línea (`ValidadorPedidoCustom.java:28-33`). El analizador de Álvaro acumula las cantidades en un `LinkedHashMap`, que conserva el orden del archivo, y en un `TreeMap`, que ordena las tallas (`src/Psp/AnalizadorTallas.java:92-95`):

```
            String clave = modelo + ";" + color + ";" + talla;
            porVarianteYTalla.merge(clave, cantidad, Integer::sum);
            porTalla.merge(talla, cantidad, Integer::sum);
            total += cantidad;
```

Si un hijo rechaza los datos, no escribe nada por stdout, escribe `ERROR;motivo` por stderr y termina con código 2. Es el formato del ejemplo del profesor `EstadisticasHijo2.java`, y Álvaro usa el mismo código. En el validador (`ValidadorPedidoCustom.java:232-235`):

```
    private static void error(String motivo) {
        System.err.println("ERROR;" + motivo);
        System.exit(2);
    }
```

La salida de error, aparte: un hilo por tubería

Las tres tuberías de un hijo (entrada, salida y error) tienen un tamaño limitado. Si una se llena, quien escribe se queda bloqueado: el padre, si envía datos que el hijo no lee, o el hijo, si escribe algo que nadie recoge. Por eso `GestorProcesos` usa un hilo para cada tubería (`GestorProcesos.java:75-89`):

```
        Thread escritor = new Thread(() -> {
            try (BufferedWriter entradaDelHijo = proceso.outputWriter(StandardCharsets.UTF_8)) {
                entradaDelHijo.write(entrada);

            } catch (IOException e) {
                errorDeEscritura = e.getMessage();
            }
```

```
        });

        // Hilos 2 y 3: vacian la salida y la salida de error del hijo.
        Thread lectorSalida = new Thread(() -> leerTodo(
                proceso.inputReader(StandardCharsets.UTF_8), salida));

        Thread lectorError = new Thread(() -> leerTodo(
                proceso.errorReader(StandardCharsets.UTF_8), errores));
```

- **El escritor** envía los datos y cierra la entrada del hijo, que así sabe que no llegan más.
- **Los dos lectores** guardan stdout y stderr en dos listas distintas (`leerTodo`, líneas 139-154), así que la respuesta nunca se mezcla con los errores.
- **No hace falta sincronizar:** cada hilo escribe solo en su lista, y el hilo principal no las lee hasta después de `join()` (líneas 128-130).

Este diseño viene de la primera versión, `LanzadorValidacion`, donde se comprobó que, escribiendo sin hilo, un CSV de 6 MB enviado a un hijo que no lee dejaba el programa colgado más de 45 segundos (`LanzadorValidacion.java:137-139`).

Tiempos límite: waitFor, destroy y destroyForcibly

El padre da a cada hijo 10 segundos (`ProcesadorPedidosCustom.java:27`). `GestorProcesos` primero espera con ese límite, `proceso.waitFor(segundosMaximos, TimeUnit.SECONDS)` (línea 106), y después recoge lo que han leído los hilos. Si el hijo no ha terminado, le pide que acabe y, si un segundo después sigue vivo, lo mata (`GestorProcesos.java:108-121`):

```
        if (termino) {
            codigoSalida = proceso.exitValue();

        } else {
            // destroy() le pide que termine; destroyForcibly() lo mata.
            proceso.destroy();

            if (!proceso.waitFor(1, TimeUnit.SECONDS)) {
                proceso.destroyForcibly();
                proceso.waitFor();
            }

            codigoSalida = TIEMPO_AGOTADO;
        }
```

El resultado es `TIEMPO_AGOTADO` (−2), y el padre termina con código 3 (`ProcesadorPedidosCustom.java:257-261`). La sincronización que pide el RA1 la hacemos esperando con `waitFor` a cada hijo antes de lanzar el siguiente. Procesar varios pedidos a la vez es la ampliación opcional y no está hecha.

Códigos de salida

El convenio es el mismo en el padre y en los hijos: 0 es correcto, 1 es un problema con un archivo y 2 son datos del pedido no válidos. El padre añade el 3 para cuando falla un proceso, y el analizador de Álvaro usa el 3 para un error inesperado (`ProcesadorPedidosCustom.java:33-39`):

```
    public static final int CORRECTO = 0;
    public static final int ERROR_ARCHIVO = 1;
    public static final int DATOS_RECHAZADOS = 2;
    public static final int FALLO_PROCESO = 3;
```

```
    // Codigo con el que los tres hijos dicen "los datos no son validos".
    private static final int HIJO_RECHAZA_DATOS = 2;
```

Código	Padre (`ProcesadorPedidosCustom`)	Hijos
0	Correcto: ficha generada	Correcto
1	Problema con un archivo: el de entrada (no se lanza ningún hijo) o al guardar la ficha	Lo devuelve Java si no encuentra la clase o hay una excepción sin capturar
2	Un hijo ha rechazado los datos del pedido	Datos incorrectos, con `ERROR;motivo` por stderr
3	Un hijo no arranca, devuelve un código inesperado, agota el tiempo, da una respuesta no válida o las cuentas no cuadran	Error inesperado (solo el analizador de Álvaro)

Cuando el hijo no llega a dar código, `GestorProcesos` usa −1 si no ha podido arrancar y −2 si se ha cancelado por tiempo (`GestorProcesos.java:18` y `:21`). Así, solo con el número, se sabe si hay que corregir el archivo, los datos del pedido o un programa.

Formato de la entrada: el CSV de 8 columnas

La entrada es un `.csv` en UTF-8, separado por comas, con una fila por línea de pedido y esta cabecera exacta (`ValidadorPedidoCustom.java:15-16`): `referencia,sku,cliente,modelo,color,personalizacion,talla,cantidad`.

Columna	Qué es
`referencia`	El pedido (por ejemplo, `CUSTOM-40`): la clave que relaciona todo
`sku`	La referencia de inventario de la variante y la talla
`cliente`	Para quién se fabrica
`modelo` y `color`	Qué zapatilla y en qué color
`personalizacion`	Qué hay que personalizar; puede tener varias líneas
`talla` y `cantidad`	Cuántos pares de cada talla

El validador exige 8 campos en cada registro, que no falte ningún dato, que los textos no lleven `;` ni saltos de línea (romperían las líneas entre procesos), que todo el archivo sea de un pedido y un cliente, y que la talla y la cantidad sean enteros, con la cantidad mayor que cero (`ValidadorPedidoCustom.java:58-158`). `LectorCsv` respeta las comillas: un campo entrecomillado puede llevar comas, saltos de línea y comillas dobladas. El analizador de Álvaro añade su regla: tallas de 20 a 55 (`src/Psp/AnalizadorTallas.java:85-87`).

El ejemplo es `03_PSP/datos/custom40_fabricacion_psp.csv`, una copia sin cambios del CSV que generó el script 15 sobre la base de PRUEBA, no sobre la del equipo (`03_PSP/README.md`):

```
referencia,sku,cliente,modelo,color,personalizacion,talla,cantidad
CUSTOM-40,NOVA-4-40,Club Atletico NOVA,NOVA Velocity,Rojo,Club Atletico NOVA: logo bordado y
numeracion en el talon,40,10
CUSTOM-40,NOVA-4-41,Club Atletico NOVA,NOVA Velocity,Rojo,Club Atletico NOVA: logo bordado y
numeracion en el talon,41,10
CUSTOM-40,NOVA-4-42,Club Atletico NOVA,NOVA Velocity,Rojo,Club Atletico NOVA: logo bordado y
numeracion en el talon,42,10
```

```
CUSTOM-40,NOVA-4-43,Club Atletico NOVA,NOVA Velocity,Rojo,Club Atletico NOVA: logo bordado y
numeracion en el talon,43,10
```

La ficha y el registro

El proceso deja dos archivos en `salidas/`:

- **La ficha auxiliar**, `ficha_taller_<referencia>.txt`. Antes de escribirla, el generador comprueba que la suma por variante, la suma por talla y el total son iguales (`GeneradorFichaTaller.java:132-135`). El generador solo usa sus flujos estándar: es el padre quien guarda el archivo (`ProcesadorPedidosCustom.java:184-189`).
- **El registro**, `registro_psp.txt`, con un bloque por ejecución, también las que fallan: la fecha, el archivo, cada paso con su código, su tiempo y la respuesta del hijo, y el resultado final. Es el registro del resultado de cada subproceso que pide el enunciado.

El registro se escribe al final del archivo, sin borrar lo anterior (`ProcesadorPedidosCustom.java:341-347`):

```
        // Cada ejecucion anade su bloque al final (APPEND), separado por una linea en blanco.
        resumen.add("");

        try {
            Files.createDirectories(CARPETA_SALIDAS);
            Files.write(REGISTRO, resumen, StandardCharsets.UTF_8,
                    StandardOpenOption.CREATE, StandardOpenOption.APPEND);
```

Relación con PostgreSQL: el script 15

El CSV sale hoy de PostgreSQL con `02_Acceso_Datos/sql/15_exportar_fabricacion.sql`, y PSP solo lo lee: no se conecta a la base. El script exporta por defecto CUSTOM-40, y otro pedido con `-v referencia=…`. Antes de escribir, comprueba que el pedido existe, está confirmado, suma exactamente 40 pares y tiene una línea por talla (líneas 109-160). Esta es la comprobación del estado (líneas 131-135):

```
    IF v_estado <> 'confirmado' THEN
        RAISE EXCEPTION
            '% esta en estado "%" y solo se fabrica un pedido confirmado. Un pedido pendiente aun
no se fabrica y uno cancelado ya no',
            v_ref, v_estado;
    END IF;
```

Con `ON_ERROR_STOP`, si una comprobación falla, `psql` se detiene antes de abrir el archivo y el CSV anterior queda intacto. La exportación usa `\copy`, con cabecera, en UTF-8 y ordenada por talla (línea 197), y al final se vuelve a comprobar lo exportado (líneas 202-224), todo en la misma transacción.

Relación con Odoo

Odoo lleva la venta y la fabricación de verdad, y PSP no se conecta a Odoo. Hay tres límites claros:

- **Hoy no se procesa ninguna exportación de Odoo.** El único origen del CSV es el script 15.
- **S00005 no pasa por PSP.** Es la venta que documenta Berme en Odoo (Club Atletico NOVA, 40 Runner X), pero no tiene CSV ni ficha en PSP. Además, en Odoo es una sola línea sin talla, y el validador exige la talla en cada registro (`ValidadorPedidoCustom.java:113-115`), así que tal cual no se podría procesar sin inventar datos.

- **El SKU no sirve como clave común.** En Odoo, `NOVA-4-40` a `NOVA-4-44` son variantes de Runner X, y en PostgreSQL, de NOVA Velocity.

La clave para relacionar el resultado con una venta es la **referencia** del pedido, que se conserva de principio a fin: en el CSV, en cada respuesta `OK;`, en la ficha y en el registro. Sin programar nada, una ficha quedaría relacionada con una venta de Odoo si el CSV lleva como `referencia` la «Referencia del cliente» de esa venta (`03_PSP/memoria/DOCUMENTACION_PSP.md`, §9.3).

Solo hay un puente documentado, y solo en el texto del PDF de Berme: la venta S00001 tiene como «Referencia del cliente» `NOVA-0011`, que en PostgreSQL es un pedido del Club Atletico NOVA de 40 pares y 4.796 €. Pero ese pedido está pendiente, y el script 15 solo exporta pedidos confirmados. Hay que comprobarlo en el Odoo de Berme. **[EQUIPO: decidir si se demuestra la relación de PSP con una venta de Odoo y con cuál]**

3.2.3 Requisitos del proyecto

Los requisitos de PSP salen del enunciado intermodular: el apartado 4.3, el anexo 13 y los criterios del módulo 0490 de la página 9. Usamos los identificadores IM-PSP-xx de nuestro índice literal del enunciado (`rubrica_IM_RT.md`, §2). El estado de cada fila sale de la matriz A/B/C/D de la auditoría (`13_IM_ABCD.md`, §1.2): A es «cumplido», B es «a demostrar en clase», C es «pendiente» (se puede hacer sin programar) y D es «no hecho».

El enunciado pide para PSP un solo resultado de aprendizaje, el RA1: «Desarrolla aplicaciones compuestas por varios procesos reconociendo y aplicando principios de programación paralela». Sus criterios se resumen así: «CE c), e)-g). Se analizan las características de los procesos, se crean subprocesos y se utilizan mecanismos para compartir información, sincronizarlos y obtener sus valores de retorno» y «CE i). Se depuran y documentan las aplicaciones desarrolladas». En la columna del RA asociamos cada requisito con la parte de ese texto que le corresponde.

Objetivo, requisitos obligatorios y guion de la demostración

ID y RA	Requisito	Cómo se cumple	Estado
IM-PSP-01	Subsistema auxiliar con varios procesos, sin sustituir al ERP	Un padre y tres hijos; solo lee una copia y escribe en `salidas/`	Cumplido
IM-PSP-02	Procesar una copia o exportación del pedido en CSV o JSON	CSV del script 15 (`datos/custom40_fabricacion_psp.csv`); un `.json` se rechaza con código 1	Cumplido
IM-PSP-03	Aplicación Java de consola	Clases de `03_PSP/`, con JDK 17 y sin librerías	Cumplido
IM-PSP-04 · RA1 e)	Recibir la exportación y coordinar varios procesos	`ProcesadorPedidosCustom.java:54-80`, `:85`, `:125` y `:171`	Cumplido
IM-PSP-05	No sustituir a Ventas ni a Fabricación de Odoo	No escribe en ninguna base; pie de la ficha (`GeneradorFichaTaller.java:174-176`)	Cumplido
IM-PSP-06 · RA1 e)	Coordinador que lanza los hijos con `ProcessBuilder`	`GestorProcesos.java:55` y `:60`	Cumplido
IM-PSP-07	Procesos especializados: validar, resumir por talla, ficha o informe	Validador, analizador de Álvaro y generador de la ficha	Cumplido

IM-PSP-08 · RA1 f)	Comunicación por stdin/stdout, tuberías o IPC	`GestorProcesos.java:75-89`; pruebas 06a y 06b	Cumplido
IM-PSP-09 · RA1 g)	Gestionar, de forma justificada, códigos de salida, errores, tiempos de espera y finalización	`GestorProcesos.java:106-121`; `ProcesadorPedidosCustom.java:252-354`; pruebas 08 a 12	Cumplido
IM-PSP-10	Salida comprensible relacionada con el pedido original de Odoo	La ficha y el registro llevan la referencia, pero hoy solo hay CUSTOM-40, que no es una venta de Odoo	Pendiente
IM-PSP-11	Ejemplo SO024: validar, agrupar, ficha y registro de cada subproceso	Hecho con CUSTOM-40; SO024 solo como ensayo PROVISIONAL con datos inventados	Cumplido
IM-PSP-12	Evolución: varias órdenes, sincronización y comparación secuencial y concurrente	Se procesa un pedido por archivo	No hecho (opcional)
IM-PSP-13	Demo: el coordinador lanza los hijos, intercambia información y controla el resultado	Implementado; se enseña en la defensa	A demostrar en clase
IM-PSP-14	Demo: relacionar el resultado con el pedido original de Odoo	Igual que IM-PSP-10	Pendiente
IM-PSP-15 a 17	Autocomprobación: función auxiliar clara, lanza y controla procesos, intercambian información y gestionan errores	`psp_pruebas.txt` del cierre: 27 de 27	Cumplido
IM-PSP-18	Anexo: posible implementación externa y auxiliar a Odoo	Toda la carpeta `03_PSP/`	Cumplido

Anexo y criterios técnicos observables

ID y RA	Requisito	Cómo se cumple	Estado
IM-PSP-19	Padre: lee la exportación, lanza los hijos, encadena resultados y decide si sigue; resumen y códigos	`ProcesadorPedidosCustom.java:87`, `:127`, `:145-148` y `:173`; línea `Resultado: …`	Cumplido
IM-PSP-20	Validador: campos esperados y formato de tallas y cantidades; OK y datos normalizados	`ValidadorPedidoCustom.java:58-192`	Cumplido
IM-PSP-21	Analizador: resumen por talla y variante	`src/Psp/AnalizadorTallas.java:92-113`, de Álvaro	Cumplido
IM-PSP-22	Generador: ficha legible, sin crear ni sustituir la orden de	`GeneradorFichaTaller.java:132-177`	Cumplido

	fabricación		
IM-PSP-23 · RA1 e)	Uso correcto de `ProcessBuilder`	`GestorProcesos.java:52-60`	Cumplido
IM-PSP-24 · RA1 f)	Paso de parámetros o comunicación por stdin/stdout	`GestorProcesos.java:75-82`	Cumplido
IM-PSP-25 · RA1 f)	Lectura diferenciada y controlada de stdout y stderr	`GestorProcesos.java:85-89` y `:139-154`; prueba 07	Cumplido
IM-PSP-26 · RA1 g)	`waitFor` y tratamiento de los códigos de salida	`GestorProcesos.java:106-109`; prueba 08a	Cumplido
IM-PSP-27 · RA1 g)	Procesos bloqueados o que tardan demasiado	`GestorProcesos.java:106-121`; prueba 10	Cumplido
IM-PSP-28	Responsabilidades separadas y código legible	Un programa por tarea y el lanzamiento aparte, en `GestorProcesos`	Cumplido
IM-PSP-29	Registro que permita seguir el pedido y relacionarlo con su referencia de Odoo	`registro_psp.txt` guarda la referencia, pero no la de una venta de Odoo	Pendiente
IM-PSP-30	Ampliación: varios pedidos concurrentes, recurso compartido, procesos frente a hilos	No se ha hecho	No hecho (opcional)
RA1 c), e)-g)	Crear subprocesos, compartir información, sincronizarlos y obtener sus valores de retorno	`GestorProcesos.java` completo; `psp_pruebas.txt`	Cumplido
RA1 i)	Depurar y documentar	`03_PSP/README.md`, `03_PSP/memoria/DOCUMENTACION_PSP.md`, pruebas y comprobador	Cumplido
Restricción	«La gestión maestra de stock, ventas y fabricación seguirá perteneciendo a Odoo»	PSP no escribe en ninguna base (`03_PSP/README.md`, línea 11)	Cumplido

Lo que falta en PSP es la relación con una venta de Odoo (IM-PSP-10, 14 y 29). No hace falta programar: bastaría con procesar un CSV cuya `referencia` sea la «Referencia del cliente» de una venta de Odoo y documentar la equivalencia. La concurrencia con varios pedidos (IM-PSP-12 y 30) es opcional y no está hecha.

3.2.4 Resultados del proyecto

El resultado es un subsistema que procesa de principio a fin la exportación real de un pedido Custom, deja una ficha legible para el taller y un registro de cada ejecución, y se para de forma controlada cuando algo va mal.

PSP es una aplicación de consola, y en `03_PSP/evidencias/` no hay capturas de pantalla, sino el texto que escribieron los programas. Según `03_PSP/evidencias/README.md`, son salidas reales del 10/11 con el JDK 17.0.12 en Windows 11, sin editar: solo se cambiaron los finales de línea y se pasaron a UTF-8. Por eso las mostramos como bloques de texto, con el archivo del que sale cada una. En los

comandos, `clases` son nuestras clases compiladas y `clases_alvaro`, las de Álvaro, compiladas aparte desde su rama.

El pedido CUSTOM-40 de principio a fin

Esta es la ejecución del padre con el CSV de CUSTOM-40 (`03_PSP/evidencias/01_ejecucion_custom40.txt`, líneas 1-10). Se ven los tres pasos con su código y su tiempo, la respuesta `OK;` de cada hijo y el control de que el validador y el analizador cuentan los mismos 40 pares:

```
> java -cp "clases;clases_alvaro" ProcesadorPedidosCustom datos\custom40_fabricacion_psp.csv
===== NOVA Sneakers - PSP - 2026-11-10 14:52:30 =====
Archivo: datos\custom40_fabricacion_psp.csv
Paso 1 ValidadorPedidoCustom: codigo 0 en 0,21 s
  OK;CUSTOM-40;4;40
Paso 2 Psp.AnalizadorTallas: codigo 0 en 0,15 s
  OK;CUSTOM-40
  4 combinaciones de modelo, color y talla; 4 tallas; 40 pares
Paso 3 GeneradorFichaTaller: codigo 0 en 0,17 s
  OK;CUSTOM-40
```

Después, el padre muestra la ficha y termina con código 0 (mismo archivo, líneas 42-44):

```
Ficha para el taller: salidas\ficha_taller_CUSTOM-40.txt
Resultado: CORRECTO (codigo 0). Pedido CUSTOM-40 procesado
(codigo de salida del padre: 0)
```

En la regresión del 13/11 sobre el árbol final, CUSTOM-40 volvió a terminar con código 0 y con la misma ficha, sin contar las fechas (`06_Intermodular/evidencias/regresion-final/02_java_psp/00_resumen.txt`). En la regresión de cierre, sobre `4f09742`, la comprobación `psp_custom40` también terminó con código 0 (`cierre-entrega-20261113/regresion-resultados.json`).

La ficha auxiliar para el taller

Esta es la ficha que guardó el padre en `salidas/` (`03_PSP/evidencias/ficha_taller_CUSTOM-40.txt`, líneas 1-16). Lleva el pedido, el cliente, el archivo del que salen los datos, la personalización y los pares por modelo, color y talla:

```
================================================
NOVA SNEAKERS - FICHA AUXILIAR PARA TALLER
================================================
Pedido:           CUSTOM-40
Cliente:          Club Atletico NOVA
Datos exportados: custom40_fabricacion_psp.csv

Personalizacion:
  Club Atletico NOVA: logo bordado y numeracion en el talon

Pares por modelo, color y talla:
  Modelo               Color        Talla  Pares
  NOVA Velocity        Rojo            40     10
  NOVA Velocity        Rojo            41     10
  NOVA Velocity        Rojo            42     10
  NOVA Velocity        Rojo            43     10
```

La segunda parte de la ficha (mismo archivo, líneas 18-29) agrupa los pares por talla, da el total y termina con el aviso de que es un documento auxiliar:

```
Pares por talla:
  Talla 40: 10 pares
  Talla 41: 10 pares
  Talla 42: 10 pares
  Talla 43: 10 pares

TOTAL: 40 pares
-------------------------------------------------
Documento auxiliar generado por PSP a partir de
una exportacion del pedido. No sustituye a la
orden de fabricacion de Odoo.
=================================================
```

Cada hijo por separado

Cada hijo se puede lanzar solo, con una tubería, para ver qué recibe por stdin y qué devuelve por stdout. Estas son las salidas del hijo 1 y del hijo 2 de Álvaro con CUSTOM-40 (`03_PSP/evidencias/03_hijos_por_separado.txt`, líneas 1-25):

```
# Hijo 1 suelto: el CSV entra por stdin y los datos normalizados salen por stdout
> type datos\custom40_fabricacion_psp.csv | java -cp "clases;clases_alvaro" ValidadorPedidoCustom
OK;CUSTOM-40;4;40
CLIENTE;Club Atletico NOVA
PERSONALIZACION;Club Atletico NOVA: logo bordado y numeracion en el talon
PEDIDO;CUSTOM-40
LINEA;NOVA-4-40;NOVA Velocity;Rojo;40;10
LINEA;NOVA-4-41;NOVA Velocity;Rojo;41;10
LINEA;NOVA-4-42;NOVA Velocity;Rojo;42;10
LINEA;NOVA-4-43;NOVA Velocity;Rojo;43;10
FIN
(codigo de salida: 0)
# Hijo 2 suelto (de Alvaro): recibe el bloque PEDIDO / LINEA / FIN que ha dado el hijo 1
> (bloque PEDIDO..FIN del hijo 1) | java -cp "clases;clases_alvaro" Psp.AnalizadorTallas
OK;CUSTOM-40
VARIANTE_TALLA;NOVA Velocity;Rojo;40;10
VARIANTE_TALLA;NOVA Velocity;Rojo;41;10
VARIANTE_TALLA;NOVA Velocity;Rojo;42;10
VARIANTE_TALLA;NOVA Velocity;Rojo;43;10
TALLA;40;10
TALLA;41;10
TALLA;42;10
TALLA;43;10
TOTAL;40
(codigo de salida: 0)
```

Y este es un rechazo del hijo 1 con el pedido de prueba que trae una cantidad 0: nada por stdout, el motivo por stderr y código 2 (mismo archivo, líneas 50-54):

```
# Hijo 1 con un dato mal: nada por stdout, el motivo por stderr y codigo 2
> type datos\prueba_cantidad_cero.csv | java -cp "clases;clases_alvaro" ValidadorPedidoCustom
stdout: []
stderr: ERROR;La cantidad del registro 2 tiene que ser mayor que cero, y es 0
(codigo de salida: 2)
```

Errores controlados: códigos 1, 2 y 3

Con un archivo que no existe, el padre termina con código 1 sin lanzar ningún hijo. Con un pedido que trae una cantidad 0, el hijo 1 lo rechaza y el padre termina con código 2, sin lanzar los pasos 2 y 3 (`03_PSP/evidencias/02_errores_controlados.txt`, líneas 1-13):

```
> java -cp "clases;clases_alvaro" ProcesadorPedidosCustom datos\no_existe.csv
===== NOVA Sneakers - PSP - 2026-11-10 14:52:31 =====
```

```
Archivo: datos\no_existe.csv
Resultado: ERROR EN EL ARCHIVO (codigo 1). No existe el archivo: datos\no_existe.csv
(codigo de salida del padre: 1)

> java -cp "clases;clases_alvaro" ProcesadorPedidosCustom datos\prueba_cantidad_cero.csv
===== NOVA Sneakers - PSP - 2026-11-10 14:52:31 =====
Archivo: datos\prueba_cantidad_cero.csv
Paso 1 ValidadorPedidoCustom: codigo 2 en 0,14 s
  ERROR;La cantidad del registro 2 tiene que ser mayor que cero, y es 0
Resultado: PEDIDO RECHAZADO (codigo 2). El paso 1 ha rechazado los datos del pedido
(codigo de salida del padre: 2)
```

El tercer caso es un fallo de un proceso: el mismo pedido correcto, pero sin el analizador de Álvaro en el classpath. Java no encuentra la clase, el hijo 2 termina con código 1 y el padre termina con código 3, con el aviso de que hay que compilarla (mismo archivo, líneas 15-24):

```
# Mismo pedido correcto, pero SIN el analizador de Alvaro en el classpath:
> java -cp clases ProcesadorPedidosCustom datos\custom40_fabricacion_psp.csv
===== NOVA Sneakers - PSP - 2026-11-10 14:52:32 =====
Archivo: datos\custom40_fabricacion_psp.csv
Paso 1 ValidadorPedidoCustom: codigo 0 en 0,17 s
  OK;CUSTOM-40;4;40
Paso 2 Psp.AnalizadorTallas: codigo 1 en 0,15 s
  Error: Could not find or load main class Psp.AnalizadorTallas
Resultado: FALLO DE UN PROCESO (codigo 3). El paso 2 ha fallado con codigo 1. No se encuentra la
clase Psp.AnalizadorTallas: hay que compilarla (ver README)
(codigo de salida del padre: 3)
```

El registro de las ejecuciones

El registro acumula un bloque por ejecución, también las que fallan. Estos son los dos primeros, la ejecución correcta y la del archivo inexistente (`03_PSP/evidencias/registro_psp.txt`, líneas 1-15); las líneas 17-29 son los casos con código 2 y 3:

```
===== NOVA Sneakers - PSP - 2026-11-10 14:52:30 =====
Archivo: datos\custom40_fabricacion_psp.csv
Paso 1 ValidadorPedidoCustom: codigo 0 en 0,21 s
  OK;CUSTOM-40;4;40
Paso 2 Psp.AnalizadorTallas: codigo 0 en 0,15 s
  OK;CUSTOM-40
  4 combinaciones de modelo, color y talla; 4 tallas; 40 pares
Paso 3 GeneradorFichaTaller: codigo 0 en 0,17 s
  OK;CUSTOM-40
Ficha para el taller: salidas\ficha_taller_CUSTOM-40.txt
Resultado: CORRECTO (codigo 0). Pedido CUSTOM-40 procesado

===== NOVA Sneakers - PSP - 2026-11-10 14:52:31 =====
Archivo: datos\no_existe.csv
Resultado: ERROR EN EL ARCHIVO (codigo 1). No existe el archivo: datos\no_existe.csv
```

La exportación desde PostgreSQL

En la regresión de cierre del 13/11, el script 15 volvió a exportar CUSTOM-40 sobre una base nueva y pasó sus comprobaciones (`06_Intermodular/evidencias/cierre-entrega-20261113/sql_15_exportar_fabricacion.txt`, líneas 21-27; la ruta del espacio de trabajo aparece sustituida por `<workspace>` en el original):

```
--- Exportacion
COPY 4
```

```
psql:<workspace>/work/verificacion-entrega-
20261113/02_Acceso_Datos/sql/15_exportar_fabricacion.sql:224: NOTICE:  PASS: la orden exportada
son 4 filas, 4 tallas y 40 pares.
DO
COMMIT

Exportacion terminada.
```

Además, se hizo un ensayo con SO024 en una base desechable y con datos inventados, rotulado PROVISIONAL: el padre terminó con código 0, con 9 tallas y 40 pares (`03_PSP/evidencias/so024_provisional/`). No es un pedido oficial ni procede de Odoo.

3.2.5 Pruebas

PSP tiene tres baterías de pruebas, y las tres se repitieron en la regresión de cierre del 13/11 sobre el código `4f09742`, en un clon nuevo y con el JDK 17.0.12 (`06_Intermodular/evidencias/cierre-entrega-20261113/README.md`):

Batería	Autoría según el repositorio	Resultado en el cierre	Evidencia
`PruebasPsp`	Commit `9dc66ac`, preparado en la sesión de Omar con ayuda de Claude	27 de 27	`psp_pruebas.txt`
`Psp.PruebasAnalizador`	Álvaro (commit `e71947e`)	8 de 8	`psp_analizador.txt`
`scripts/comprobar_psp.py`	Berme (commits `f1983d2` y `4f09742`)	Correcto	`psp_comprobador.txt`

PruebasPsp: 27 de 27

`PruebasPsp` lanza los programas de verdad, como procesos, en una carpeta temporal nueva, sin escribir en el repositorio. Al padre completo también lo lanza con `ProcessBuilder` y guarda su consola en un archivo (`pruebas/PruebasPsp.java:342-366`). Termina con código 0 si pasan todas y con 1 si falla alguna.

Para provocar lo que un hijo de verdad no debería hacer, usa `pruebas/HijoDePrueba.java`. La primera línea que recibe le dice qué hacer: `ECO` devuelve lo que recibe, `DOS_SALIDAS` escribe por stdout y por stderr, `CODIGO;7` termina con ese código, `EXCEPCION` lanza una excepción sin capturar y `DORMIR` se queda parado 60 segundos.

Las pruebas cubren 12 puntos (`03_PSP/evidencias/README.md`):

Punto	Pruebas	Qué comprueba
1. Pedido Custom válido	01	CUSTOM-40 de principio a fin: código 0, `Resultado: CORRECTO` y la ficha creada
2. Varias tallas y cantidades	02	Un pedido de prueba con nueve tallas (38 a 46), dos colores y una talla repetida: la 42 suma 12 y el total, 40
3. Validación correcta	03	El validador solo: `OK;CUSTOM-40;4;40`, el cliente, el bloque PEDIDO y las cuatro líneas normalizadas

4. Agrupación de tallas	04	Validador y analizador de Álvaro: Rojo 42 = 8, talla 42 = 12 y total 40
5. Ficha auxiliar	05	La ficha de CUSTOM-40 lleva pedido, cliente, personalización y total, y el registro dice CORRECTO
6. Comunicación entre procesos	06a y 06b	Lo que el padre envía por stdin vuelve igual por stdout, y una personalización de dos líneas, con coma y comillas, cruza los tres procesos
7. stdout y stderr	07	Cada salida llega a su lista
8. Códigos de salida	08a y 08b	Se recoge el código exacto del hijo (7), y el validador rechaza con 2, ERROR; por stderr y nada por stdout
9. Fallo de un proceso	09a y 09b	Un hijo que lanza una excepción y una clase que no existe: código 1 en los dos
10. Tiempo agotado	10	Un hijo que duerme 60 s se cancela a los 2 s (en la prueba el límite es 2)
11. Archivo inexistente	11	El padre avisa con código 1 y no lanza ningún hijo
12. Datos mal formados	12a a 12m	Trece casos rechazados sin crear ninguna ficha: código 2, o 1 si el archivo está vacío o es un `.json`

Esta es la salida de la regresión de cierre (`06_Intermodular/evidencias/cierre-entrega-20261113/psp_pruebas.txt`, líneas 3-27):

```
=== 1. Pedido Custom valido (exportacion real de CUSTOM-40) ===
[SUPERADA] 01 CUSTOM-40 de principio a fin -> codigo 0, Resultado: CORRECTO (codigo 0). Pedido
CUSTOM-40 procesado
=== 2. Varias tallas y cantidades (pedido de prueba, no real) ===
[SUPERADA] 02 Nueve tallas (38 a 46), dos colores y una talla repetida -> codigo 0; 9 tallas, la
42 suma 6 + 2 + 4 = 12, total 40
=== 3. Validacion correcta (el validador, solo) ===
[SUPERADA] 03 El validador acepta CUSTOM-40 y normaliza sus 4 lineas -> primera linea: OK;CUSTOM-
40;4;40
=== 4. Agrupacion de tallas (validador -> analizador de Alvaro) ===
[SUPERADA] 04 Agrupa por variante y por talla -> Rojo 42 = 6 + 2 = 8; talla 42 = 12; total 40
=== 5. Generacion de la ficha auxiliar ===
[SUPERADA] 05 Ficha de CUSTOM-40 con sus datos y registro del procesamiento -> ficha de 29 lineas
y registro_psp.txt
=== 6. Comunicacion entre procesos ===
[SUPERADA] 06a Lo que el padre envia por stdin vuelve igual por stdout -> recibido: [OK;eco, uno,
dos;tres, cuatro, con coma]
[SUPERADA] 06b La personalizacion de 2 lineas cruza los tres procesos y llega a la ficha -> las
dos lineas, con la coma y las comillas
=== 7. Captura de stdout y stderr por separado ===
[SUPERADA] 07 stdout y stderr llegan cada uno a su lista -> stdout [OK;salida] / stderr
[AVISO;esto va por stderr]
=== 8. Codigos de salida ===
[SUPERADA] 08a Se recoge el codigo exacto del hijo -> codigo 7
[SUPERADA] 08b El validador rechaza con 2, ERROR; por stderr y nada por stdout -> codigo 2,
[ERROR;La cantidad del registro 1 tiene que ser un numero entero, y es: diez]
=== 9. Fallo controlado de un proceso ===
[SUPERADA] 09a Un hijo que revienta con una excepcion -> codigo 1, Exception in thread "main"
java.lang.RuntimeException: fallo provocado por la prueba
[SUPERADA] 09b Un hijo que no se puede cargar -> codigo 1, Error: Could not find or load main
class ClaseQueNoExiste
=== 10. Tiempo de espera agotado ===
```

```
[SUPERADA] 10 Un hijo que no termina se cancela a los 2 s -> codigo -2 (TIEMPO_AGOTADO) en 2,0 s,
en vez de 60
=== 11. Archivo inexistente ===
[SUPERADA] 11 El padre avisa y no lanza ningun hijo -> Resultado: ERROR EN EL ARCHIVO (codigo 1).
No existe el archivo: no_existe.csv
```

Y estos son los datos mal formados y el total (mismo archivo, líneas 28-43). El carácter «□» de la prueba 12k está así en el archivo de evidencia: es la «í» de «Línea», que no se pudo decodificar al guardar la salida de consola, y lo dejamos tal cual:

```
=== 12. Datos mal formados (el padre debe parar) ===
[SUPERADA] 12a Comilla sin cerrar -> codigo 2, sin ficha: ERROR;El CSV esta mal formado: falta la
comilla de cierre al final del archivo
[SUPERADA] 12b Cabecera distinta -> codigo 2, sin ficha: ERROR;La cabecera no es la esperada.
Esperaba: referencia,sku,cliente,modelo,color,personalizacion,talla,cantidad / He recibido:
referencia,talla,cantidad
[SUPERADA] 12c Cantidad que no es un numero -> codigo 2, sin ficha: ERROR;La cantidad del
registro 1 tiene que ser un numero entero, y es: diez
[SUPERADA] 12d Cantidad cero -> codigo 2, sin ficha: ERROR;La cantidad del registro 1 tiene que
ser mayor que cero, y es 0
[SUPERADA] 12e Cantidad negativa -> codigo 2, sin ficha: ERROR;La cantidad del registro 1 tiene
que ser mayor que cero, y es -5
[SUPERADA] 12f Falta la talla -> codigo 2, sin ficha: ERROR;Falta la talla en el registro 1
[SUPERADA] 12g Falta el modelo -> codigo 2, sin ficha: ERROR;Falta el modelo en el registro 1
[SUPERADA] 12h Solo la cabecera -> codigo 2, sin ficha: ERROR;El pedido solo trae la cabecera,
sin ningun registro de pedido
[SUPERADA] 12i Registro con campos de menos -> codigo 2, sin ficha: ERROR;El registro 1 tiene 7
campos y tendria que tener 8
[SUPERADA] 12j Dos pedidos en el mismo archivo -> codigo 2, sin ficha: ERROR;El registro 2 es del
pedido OTRO-99 pero las anteriores son del pedido MALO-10: un pedido por archivo
[SUPERADA] 12k Talla 99: la acepta el validador y la rechaza el analizador (paso 2) -> codigo 2,
sin ficha: ERROR;L□nea 2: talla fuera de rango (20-55)
[SUPERADA] 12l Archivo vacio -> codigo 1, sin ficha: Resultado: ERROR EN EL ARCHIVO (codigo 1).
El archivo esta vacio
[SUPERADA] 12m Formato que no es CSV -> Resultado: ERROR EN EL ARCHIVO (codigo 1). Formato no
admitido: el pedido tiene que venir en un archivo .csv

Pruebas superadas: 27 de 27
```

La prueba 12k implica a dos hijos: la talla 99 es un entero positivo y el validador la acepta, pero el analizador de Álvaro solo admite tallas de 20 a 55 y la rechaza en el paso 2.

El analizador de Álvaro: 8 de 8

`Psp.PruebasAnalizador` lanza el analizador con el `LanzadorAnalizador` de Álvaro. Tiene una prueba positiva, seis negativas y una de fallo del hijo, con una clase que no existe. Esta es su salida en el cierre (`06_Intermodular/evidencias/cierre-entrega-20261113/psp_analizador.txt`, líneas 1-21):

```
PRUEBA POSITIVA: pedido correcto, debe aceptarse
[SUPERADA] Pedido correcto -> aceptado. Resultado del análisis:
OK;NOVA-0001
VARIANTE_TALLA;Adidos Zamba;Negro;42;3
VARIANTE_TALLA;Nokia Air Maximo;Blanco;42;1
VARIANTE_TALLA;Adidos Zamba;Negro;43;1
TALLA;42;4
TALLA;43;1
TOTAL;5

=== PRUEBAS NEGATIVAS: datos incorrectos a propósito, deben rechazarse ===
[SUPERADA] Datos incompletos -> rechazado: Línea 2: faltan datos (formato
LINEA;sku;modelo;color;talla;cantidad)
[SUPERADA] Cantidad cero -> rechazado: Línea 2: la cantidad debe ser mayor que cero
[SUPERADA] Cantidad no numérica -> rechazado: Línea 2: cantidad no es un número
[SUPERADA] Entrada incorrecta -> rechazado: La primera línea debe ser PEDIDO;referencia
```

```
[SUPERADA] Falta FIN -> rechazado: Pedido incompleto: falta la línea FIN
[SUPERADA] Talla fuera de rango -> rechazado: Línea 2: talla fuera de rango (20-55)
 PRUEBA DE FALLO DEL PROCESO HIJO: el hijo no arranca, debe detectarse
[SUPERADA] Error en el proceso hijo -> fallo detectado (código 1)

Pruebas superadas: 8 de 8
```

El comprobador rápido de Berme

`scripts/comprobar_psp.py` es un script de Python que compila y prueba PSP en una carpeta temporal, con `java` y `javac` en el PATH. Comprueba cuatro cosas:

16. que el validador acepta el CSV sintético multilínea `03_PSP/pruebas/custom40.csv` y contesta `OK;CUSTOM-40;4;40`, con cuatro líneas `LINEA`, `PEDIDO;CUSTOM-40` y `FIN` (líneas 39-45);
17. que la versión anterior, `LanzadorValidacion`, da 10 pares en cada talla de la 40 a la 43 y 40 en total (líneas 46-49);
18. que tres muestras malas (una cantidad 0, otra referencia y una comilla sin cerrar) terminan con código 2 y `ERROR;` por stderr (líneas 50-56);
19. que las pruebas del paquete `Psp` de Álvaro se superan todas (líneas 57-62).

No lanza el padre `ProcesadorPedidosCustom`: eso lo prueba `PruebasPsp`. Su salida en el cierre es una sola línea (`06_Intermodular/evidencias/cierre-entrega-20261113/psp_comprobador.txt`; la «□» está así en el original, en lugar de la «í» de «multilínea»):

```
OK: CSV multil□nea, resumen, tres rechazos y pruebas del paquete Psp.
```

Lo que no está probado

- **El tiempo agotado dentro del padre completo.** Se ha probado en `GestorProcesos`, que es lo que usa el padre (prueba 10), pero no lanzando `ProcesadorPedidosCustom` con un hijo real que se cuelgue, porque ninguno de los tres hijos se cuelga (`03_PSP/evidencias/README.md`).
- **Un pedido exportado de Odoo.** No hay ninguno.
- **Varios pedidos a la vez.** Es la ampliación opcional, y no está hecha.

3.2.6 Problemas encontrados y soluciones

La codificación UTF-8 en Windows

Las tildes entre procesos. El padre lee las tuberías de cada hijo en UTF-8. Si un hijo escribiera con otra codificación, una letra con tilde del cliente o de la personalización le llegaría cambiada. Por eso cada hijo fija UTF-8 en su salida y en su salida de error nada más empezar (`ValidadorPedidoCustom.java:20-26`; lo mismo en `GeneradorFichaTaller.java:22-23` y en el analizador de Álvaro, `src/Psp/AnalizadorTallas.java:19-20`):

```
        /*
         * Escribimos en UTF-8, que es como lee el proceso padre. Si no, una
         * letra con tilde de un cliente o de una personalizacion le llegaria
         * cambiada.
         */
        System.setOut(new PrintStream(System.out, true, StandardCharsets.UTF_8));
        System.setErr(new PrintStream(System.err, true, StandardCharsets.UTF_8));
```

Por su parte, `GestorProcesos` abre las tres tuberías en UTF-8 (`GestorProcesos.java:76`, `:86` y `:89`), y el padre lee el CSV en UTF-8 y, si no lo es, termina con código 1 y el mensaje «El archivo no esta en UTF-8» (`ProcesadorPedidosCustom.java:73-74`).

Las pruebas con `-Dfile.encoding=UTF-8`. `PruebasPsp` lee la consola del padre con la codificación por defecto (`pruebas/PruebasPsp.java:363-364`). Si se lanzan las pruebas con esa opción, solo el proceso de pruebas cambia de codificación, y los textos con tilde ya no coinciden. Lo vimos el 12/11 y lo avisan `03_PSP/README.md` y la memoria de PSP (`DOCUMENTACION_PSP.md`, §10). En el cierre volvió a pasar: la primera ejecución impuso UTF-8 solo al proceso de pruebas, mientras sus hijos usaban la codificación de Windows (`cierre-entrega-20261113/README.md`). Al llegar a la prueba 12k, cuyo mensaje lleva la «í» de «Línea», la lectura falló (`06_Intermodular/evidencias/cierre-entrega-20261113/psp_primer-intento-codificacion.txt`, líneas 38-49):

```
[SUPERADA] 12j Dos pedidos en el mismo archivo -> codigo 2, sin ficha: ERROR;El registro 2 es del
pedido OTRO-99 pero las anteriores son del pedido MALO-10: un pedido por archivo
Exception in thread "main" java.nio.charset.MalformedInputException: Input length = 3
        at java.base/java.lang.String.throwMalformed(String.java:1244)
        at java.base/java.lang.String.decodeUTF8_UTF16(String.java:1131)
        at java.base/java.lang.String.newStringUTF8NoRepl(String.java:734)
        at java.base/java.lang.String.newStringNoRepl1(String.java:762)
        at java.base/java.lang.String.newStringNoRepl(String.java:744)
        at java.base/java.lang.System$2.newStringNoRepl(System.java:2394)
        at java.base/java.nio.file.Files.readString(Files.java:3369)
        at PruebasPsp.coordinador(PruebasPsp.java:364)
        at PruebasPsp.rechazado(PruebasPsp.java:326)
        at PruebasPsp.main(PruebasPsp.java:275)
```

Fue un fallo del arnés de pruebas, no del programa. Se repitió sin esa opción, como dice el README, y dio 27 de 27 (`psp_pruebas.txt`). No se cambió el código de producción, y la salida del primer intento se conserva.

Un problema parecido, también de Windows, apareció al exportar el CSV: `\o` cambiaba los saltos de línea de dentro de un campo, y por eso el script 15 usa `\copy` (`15_exportar_fabricacion.sql`, líneas 44-58).

El comprobador de Berme fallaba por la respuesta antigua

El comprobador de Berme esperaba que el validador contestara solo `OK;CUSTOM-40;4;40`, y comparaba toda la salida con ese texto. Desde el commit `9dc66ac`, el validador añade debajo los datos normalizados para el analizador de Álvaro, así que el comprobador fallaba aunque el validador funcionara bien (`DOCUMENTACION_PSP.md`, §10). Así quedó en la regresión del 13/11 (`06_Intermodular/evidencias/regresion-final/02_java_psp/00_resumen.txt`, líneas 13-14):

```
== Comprobacion rapida de Berme (scripts/comprobar_psp.py), con fallo conocido; JDK 17 en el PATH
comprobar_psp.py -> salida 1 | ERROR: El validador no reconoce los cuatro registros CSV
multilínea.
```

Berme lo arregló en el commit `4f09742` («fix(psp): adapt quick check to the validator's normalized output»; antes `0baf61f`, según `06_Intermodular/EQUIVALENCIAS_COMMITS.md`). Ahora compara solo la primera línea y comprueba además el bloque normalizado. Este es el cambio, tal como lo muestra `git show 4f09742`:

```
-        if response.stdout.strip() != 'OK;CUSTOM-40;4;40':
+        lines = response.stdout.splitlines()
+        if not lines or lines[0].strip() != 'OK;CUSTOM-40;4;40':
             raise RuntimeError('El validador no reconoce los cuatro registros CSV multilínea.')
```

```
+        normalized = [line for line in lines[1:] if line.startswith('LINEA;')]
+        if len(normalized) != 4 or 'FIN' not in lines or 'PEDIDO;CUSTOM-40' not in lines:
+            raise RuntimeError('El validador no devuelve los cuatro registros normalizados.')
```

El arreglo se incorporó a `main` sin cambiar su autoría, y en la regresión de cierre el comprobador dio correcto. El protocolo del validador no se cambió.

El tiempo de espera del LanzadorAnalizador no llega a actuar

El `LanzadorAnalizador` de Álvaro lee toda la salida del hijo antes de esperar con límite (`src/Psp/LanzadorAnalizador.java:25-36`):

```
        try (OutputStream os = proceso.getOutputStream()) {
            os.write(entrada.getBytes(StandardCharsets.UTF_8));
        }

        String salida = new String(proceso.getInputStream().readAllBytes(),
StandardCharsets.UTF_8);
        String errores = new String(proceso.getErrorStream().readAllBytes(),
StandardCharsets.UTF_8);

        boolean terminado = proceso.waitFor(SEGUNDOS_MAXIMOS, TimeUnit.SECONDS);
        if (!terminado) {
            proceso.destroyForcibly();
            return new ResultadoProceso(ResultadoProceso.SIN_RESPUESTA, salida, errores);
        }
```

`readAllBytes()` no vuelve hasta que el hijo cierra su salida, es decir, hasta que termina. Cuando se llega a `waitFor(10 s)`, el hijo ya ha acabado, así que el límite no actúa. Según `03_PSP/README.md`, se midió el 10/11 con un hijo que tarda 20 segundos: el lanzador esperó 20,1 s y devolvió código 0 en vez de `SIN_RESPUESTA`. Además, como lee primero toda la salida y después los errores, podría bloquearse si el hijo escribe mucho por stderr.

La solución fue no usarlo en el coordinador. El padre lanza el analizador con `GestorProcesos`, que espera primero con límite y lee las tuberías con hilos (apartado 3.2.2). No hemos cambiado el archivo de Álvaro: su `PruebasAnalizador` sigue usándolo y da 8 de 8, porque ninguna de sus pruebas tiene un hijo lento. Queda como propuesta para que la revise él. **[ÁLVARO: indicar si revisa o corrige el tiempo de espera de LanzadorAnalizador]**

Dos versiones del analizador

Al integrar la rama de Álvaro quedaron dos analizadores: el `AnalizadorTallas` de la raíz de `03_PSP/`, de la primera versión, que solo agrupa por talla y lee el CSV directamente, y `Psp.AnalizadorTallas`, que agrupa por variante y por talla. Para no tener dos versiones de la misma tarea, el coordinador usa solo el de Álvaro. La versión anterior (`LanzadorValidacion` y el analizador de la raíz) sigue funcionando, porque el validador mantiene su primera línea, y el comprobador de Berme todavía la prueba. No se ha borrado nada. **[EQUIPO: decidir si la versión anterior se queda como ejemplo o se quita]**

3.3 Sistemas de Gestión Empresarial (Odoo)

Fuente de este capítulo. Se basa en la documentación original de Iván Bermejo (Berme), `odoo/documentacion/Odoo-NOVA-Sneakers.pdf` (19 páginas y 28 capturas), que va íntegra en los anexos. Reutilizamos su contenido correcto y sus capturas, sin modificarlas, y cada figura indica la página de la que sale. Todas las capturas son del Odoo instalado en el Mac de Berme. Separamos siempre lo que **demuestra una captura** de lo que **solo dice el texto** del PDF.

3.3.1 Planificación y metodología del proyecto

El enunciado (§4.2) pide instalar, configurar y adaptar Odoo a la empresa, y demostrar cómo se registra y evoluciona una venta realista: cliente, venta, fabricación, inventario, entrega y factura.

Planificación del trabajo

Nuestro plan de octubre de 2026 (`Sneaker_Factory_Reparto_Tareas_Git_Worktrees.pdf`, p. 6) dividía Odoo en seis tareas. La tabla recoge **lo planificado** y dónde lo documenta el PDF de Berme (tabla de su p. 18).

ID	Empieza (plan)	Tarea planificada	Evidencia mínima (plan)	Dónde está en el PDF
OD-01	Berme	Instalación y configuración de Ubuntu y Odoo; pasos reproducibles	Entorno operativo	pp. 3-6
OD-02	Álvaro	Empresa Sneaker Factory; catálogo, variantes y usuarios iniciales	Productos consistentes	pp. 16-17. La compañía se llama «Nova Sneaker»
OD-03	Omar	Clientes, presupuestos y pedido de venta Custom SO024	Venta trazable	p. 10. La venta es S00005; SO024 era el ejemplo del enunciado
OD-04	Berme	Fabricación: materiales, productos fabricados y orden de fabricación	Flujo documentado	pp. 7-11
OD-05	Álvaro	Existencias, movimientos de almacén y entrega	Stock explicado	pp. 11 y 17
OD-06	Omar	Facturación y evidencias del recorrido completo	Pedido demostrable	p. 12

El plan ponía Odoo en dos hitos: «Odoo operativo» (H3) y la integración, en la que el pedido común «recorre los módulos con la misma referencia» (H4). El H3 está documentado; el H4 no se ha cumplido para Odoo (apartado 3.3.6).

Las fechas de los commits no sirven para el calendario, porque la historia de GitHub se reescribió el 13/11. Usamos las de las capturas. El **5 de noviembre** arranca el servicio (09:09), se crea la Runner X,

entran sus 50 unidades (10:55) y se prepara la orden de prueba. El **9 de noviembre** se termina esa orden, WH/MO/00001. El **13 de noviembre** se recorre S00005, entre las 12:50 y las 12:53; es también la fecha de la portada del PDF.

En la portada figuran como autores «Ivan Bermejo, Omar y Álvaro», y las seis tareas como «Hechas». Todas las capturas son del entorno de Berme (usuario `ivanbermejo` y administrador `berme@nova.com`). No consta quién hizo cada tarea: **[EQUIPO: reparto real de OD-01 a OD-06]**.

Metodología de trabajo

Seguimos la regla del plan, «Los tres tocan todo»: alguien empieza la tarea, los otros dos la revisan y no está terminada hasta que los tres saben explicarla.

En Odoo no se programa: se configura desde la interfaz web. El orden fue el del flujo de referencia del enunciado: primero el entorno y la base de datos; después los datos maestros (compañía, productos, componentes y lista de materiales); luego una orden de fabricación de prueba de una unidad; después el caso completo de 40 pares, y por último las variantes por talla y los usuarios. Cada paso se documentó con una captura numerada. Usamos solo datos ficticios y ninguna contraseña aparece en el documento (pp. 5 y 14). La base de datos y la máquina se quedan fuera de Git, como pide el plan (p. 7).

Herramientas utilizadas

- **OrbStack**, en el Mac de Berme: crea la máquina Linux `ubuntu`, con el dominio local `ubuntu.orb.local`.
- **Ubuntu 24.04.5 LTS** (noble), arm64.
- **Odoo 19.0 Community**, con el paquete `.deb` oficial del repositorio `nightly.odoo.com/19.0`, como servicio del sistema. Versión de las capturas: 19.0.20261105.
- **PostgreSQL** de la misma máquina, instalado con `apt`; el paquete de Odoo crea el usuario `odoo`. El PDF no da la versión: **[BERME: versión de PostgreSQL]**.
- **Aplicaciones de Odoo:** Contactos, Ventas, Inventario, Fabricación y Facturación.
- **Inteligencia artificial:** sin declarar para esta parte. **[BERME: herramientas de IA usadas, si las hubo]**

3.3.2 Descripción de la solución

Odoo es el núcleo empresarial de NOVA Sneakers: clientes, ventas, fabricación, inventario y facturación (PDF, p. 2). El enunciado permite unir los bloques con ficheros y procedimientos documentados, sin usar la API de Odoo (enunciado, p. 3).

Entorno

Elemento	Valor
Máquina	`ubuntu` de OrbStack, Ubuntu 24.04.5 LTS, arm64 (captura 01)
Acceso	`http://ubuntu.orb.local:8069`
Odoo	19.0.20261105, servicio `odoo.service` activo (capturas 03 y 04)
Configuración y registro	`/etc/odoo/odoo.conf` y `/var/log/odoo/odoo-server.log` (p. 5, texto)

Base de datos	`nova_sneaker`: español, España, sin datos de demostración (captura 11)

La base de Odoo (`nova_sneaker`) y la del proyecto (`nova_sneakers`) son distintas. Según el PDF (p. 6), Odoo no lee las tablas del proyecto ni al revés.

Compañía, aplicaciones y usuarios

La compañía es «Nova Sneaker», con el logotipo de la marca, en español, en euros y con IVA del 21 %. Estas son las aplicaciones del flujo (p. 6):

Aplicación	Para qué la usamos
Contactos	Clientes
Ventas	Presupuestos y pedidos de venta
Inventario	Productos, existencias, movimientos y entregas
Fabricación	Listas de materiales y órdenes de fabricación
Facturación	Facturas y pagos

En el menú no aparecen CRM ni Compras. Además del administrador hay dos usuarios internos con el rol «Usuario»: «Vendedor NOVA» y «Almacén NOVA» (captura 28). Sus permisos por aplicación (Ventas para el primero; Inventario y Fabricación para el segundo) **solo los dice el texto** (p. 17).

Productos, variantes y lista de materiales

- **Runner X**, el producto terminado: tipo Bienes, con rastreo de inventario, coste de 35 € y 50 unidades iniciales (captura 06).
- **Componentes:** Suela, Tela y Cordones, también con inventario. Solo hay captura de la Suela: 100 unidades y 15 € de coste (captura 13).
- **Lista de materiales** «Runner X (nuevo) 4»: un par consume una suela, una tela y unos cordones (p. 7). Su ficha no tiene captura; se ve en las órdenes de fabricación.
- **Variantes:** el atributo Talla, con los valores 38 a 46, genera 9 variantes con las referencias internas `NOVA-4-38` a `NOVA-4-46` (capturas 26 y 27).

Precio de la Runner X. El texto dice que el precio pasó de 89 € a 99,90 € (pp. 7, 13 y 17). Ninguna captura lo muestra: todas dan 89,00 €, incluida la lista de las 9 variantes. Usamos 89 €, que es lo acreditado.

Flujo de negocio

El flujo es el de referencia del enunciado, sin CRM: **Ventas → Fabricación → Inventario/Entrega → Facturación**. El caso documentado es la venta **S00005**, de 40 Runner X para el **Club Atletico NOVA**:

Paso	Documento	Qué ocurre	Capturas (página)
Venta (OD-03)	S00005	40 Runner X a 89 €, confirmada el 13/11 a las 12:50	17 y 18 (p. 10)
Fabricación (OD-04)	WH/MO/00002	40 de 40; consume 40 suelas, 40 telas y 40 cordones	19 a 21 (p. 11)
Entrega (OD-05)	WH/OUT/00005	Salen 40 Runner X hacia el	22 y 23 (p. 11)

		cliente; origen S00005	
Factura (OD-06)	INV/2026/00001	3.560,00 € más IVA, pagada y enlazada a S00005	24 (p. 12)
Stock	Movimientos de la Runner X	+50, +1, +40 y −40: quedan 51	25 (p. 17)

Al confirmar la venta, Odoo crea solo la entrega, y la factura sale del propio pedido. **La orden de fabricación se creó a mano**, porque no se configuró la fabricación bajo pedido (ruta MTO). El texto dice que su origen es S00005 (p. 11), pero **ninguna captura lo muestra**.

Ampliación documentada: conexión por API

> **Trabajo de Berme en su entorno, no verificado desde la entrega.** S00005 no pasa por esta conexión, ni por PostgreSQL, PSP o Python.

Las pp. 13 a 15 del PDF describen un conector del servidor Node con la API JSON-2 de Odoo 19. Busca el cliente por correo y los productos por su referencia interna, crea la venta, la confirma y compara el importe con el de PostgreSQL. Según la p. 13, así se crearon las ventas S00001 a S00004 (la primera, con un script de Python). **No hay captura de ninguna** ni de ninguna respuesta de la API; solo de la creación de la clave (apartado 3.3.6).

En el repositorio entregado, el conector está en `servidor-admin/odoo.js` y `pedidos.js`, pero sus rutas **no están montadas**: `/api/pedidos` responde 404. Además, los códigos chocan: en Odoo, `NOVA-4-40` a `NOVA-4-44` son variantes de la Runner X, y en PostgreSQL, NOVA Velocity. El texto (p. 13) da a S00001 la referencia del cliente `NOVA-0011` y 4.796,00 €, como el pedido `NOVA-0011` de PostgreSQL. Ese puente está **documentado en el texto y pendiente de comprobar** (capítulo 4).

3.3.3 Requisitos del proyecto

Los requisitos salen del enunciado: §4.2 (pp. 2-5 y 10), los resultados de aprendizaje del módulo 0491 (p. 9) y el entregable del ERP (p. 7). Los identificadores son los de nuestro índice y el estado se basa en la matriz `12_odoo_ABCD.md` de la auditoría.

ID	Requisito (resumen)	Cómo se cumple (PDF)	Estado
IM-SGE-01	Configurar el flujo de clientes y oportunidades, ventas, fabricación, inventario y facturación	pp. 6-12	Cumplido, salvo las oportunidades (no hay CRM)
IM-SGE-02	Registrar el cliente, la oportunidad si hay CRM, y pasar de presupuesto a pedido	p. 10	Presupuesto y pedido: cumplido. Ficha del cliente: a demostrar en clase
IM-SGE-03	Producto y variantes, pedido, fabricación, inventario, entrega y facturación	pp. 7-12 y 16-17	Cumplido. Las variantes no aparecen en la venta
IM-SGE-04	Flujo CRM → Ventas → Fabricación → Inventario/Entrega → Facturación	pp. 10-12	Cumplido, sin CRM
IM-SGE-05	Ruta MTO, como	p. 18, como mejora	Opcional, no realizado

	ampliación		
IM-SGE-06	Instalar, configurar y adaptar Odoo a la empresa	pp. 3-7 y 16	Cumplido. Adaptación limitada (un modelo; almacén «My Company»)
IM-SGE-07	La venta encadena los demás procesos	pp. 10-12	Entrega y factura: cumplido. Fabricación: a demostrar en clase
IM-SGE-08	Máquina virtual con Ubuntu y Odoo instalado, documentado y operativo	pp. 3-5	Documentado: cumplido. Operativo: a demostrar en clase
IM-SGE-09	Compañía y perfiles: comercial, producción, almacén y administración, cuando proceda	pp. 6 y 17	Compañía y dos usuarios: cumplido. Permisos: a demostrar. Producción y administración: pendiente
IM-SGE-10	Registro de clientes y contactos	pp. 10-12 (solo el nombre)	A demostrar en clase
IM-SGE-11	Oportunidades, si CRM entra en el alcance	p. 6: no hay CRM	Pendiente, si entra en el alcance
IM-SGE-12	Presupuestos y pedidos con líneas, cantidades, variantes y condiciones	p. 10	Cumplido en parte. Pendiente: venta con líneas por variante
IM-SGE-13	Fabricación: productos fabricados, listas de materiales y órdenes	pp. 7-9 y 11	Cumplido
IM-SGE-14	Stock, movimientos y entrega	pp. 9, 11 y 16-17	Cumplido
IM-SGE-15	Facturación de la venta	p. 12	Cumplido
IM-SGE-16	Capturas, exportaciones o informes que sigan el mismo pedido	pp. 10-12 y 17	Capturas: cumplido. Exportaciones e informes: pendiente
IM-SGE-17	Odoo como núcleo y demostración de una venta realista	pp. 10-12	Documentado: cumplido. Demostración: a demostrar en clase
IM-SGE-18	Ejemplo SO024: Odoo conserva la venta y la orden de fabricación	pp. 10-11	Cumplido con S00005 y WH/MO/00002 (SO024 no es obligatorio)
IM-SGE-19 y 20	Guion: cliente y venta; fabricación, entrega y factura	pp. 10-12	A demostrar en clase
IM-SGE-21	Autocomprobación: Odoo adaptado y con un flujo realista	Todo el PDF	Flujo: cumplido. Adaptación: en parte

ID	**Requisito (resumen)**	**Cómo se cumple (PDF)**	**Estado**
RA1 (CE d-h)	Verificar el entorno: sistema operativo, gestor de datos, configuración, operaciones e incidencias	pp. 3-5 y 14-16	Cumplido en parte: PostgreSQL sin versión. Incidencias reunidas en 3.3.6
RA2 (CE a-f)	Implantar: módulos, instalación, configuración, verificación y	pp. 3-6	Cumplido

	documentación		
RA3 (CE d, g, h)	Exportar datos e informes y sacar información	No hay	Pendiente
RA4 (CE a, d, h, i)	Adaptar entradas de datos y procesos, probar y documentar	pp. 7-8 y 16-17	Cumplido en parte: variantes, referencias internas, lista de materiales y usuarios
IM-COM-16	ERP configurado: instancia o máquina virtual y copia de seguridad	p. 19 (solo instrucciones)	Instancia: a demostrar en clase. Copia de seguridad: pendiente

Lo que falta

Todo se puede hacer sin programar, con la configuración que ya existe:

- **Copia de seguridad (IM-COM-16).** La p. 19 solo explica cómo hacerla. Falta la copia de `nova_sneaker` con fecha, tamaño y SHA-256, y una restauración probada. **[BERME: copia de seguridad y restauración]**
- **Exportaciones e informes (RA3 e IM-SGE-16).** Exportar a CSV las líneas de la venta y sacar un informe de ventas o de existencias. **[BERME: exportación e informe]**
- **Venta con variantes (IM-SGE-12).** Un pedido nuevo con una línea por talla, sin tocar S00005. **[BERME: pedido con líneas por talla, si se hace]**

3.3.4 Resultados del proyecto

Las figuras siguen el orden del PDF. Son capturas de Iván Bermejo sin modificar; cada pie indica su número de captura y su página.

Entorno e instalación

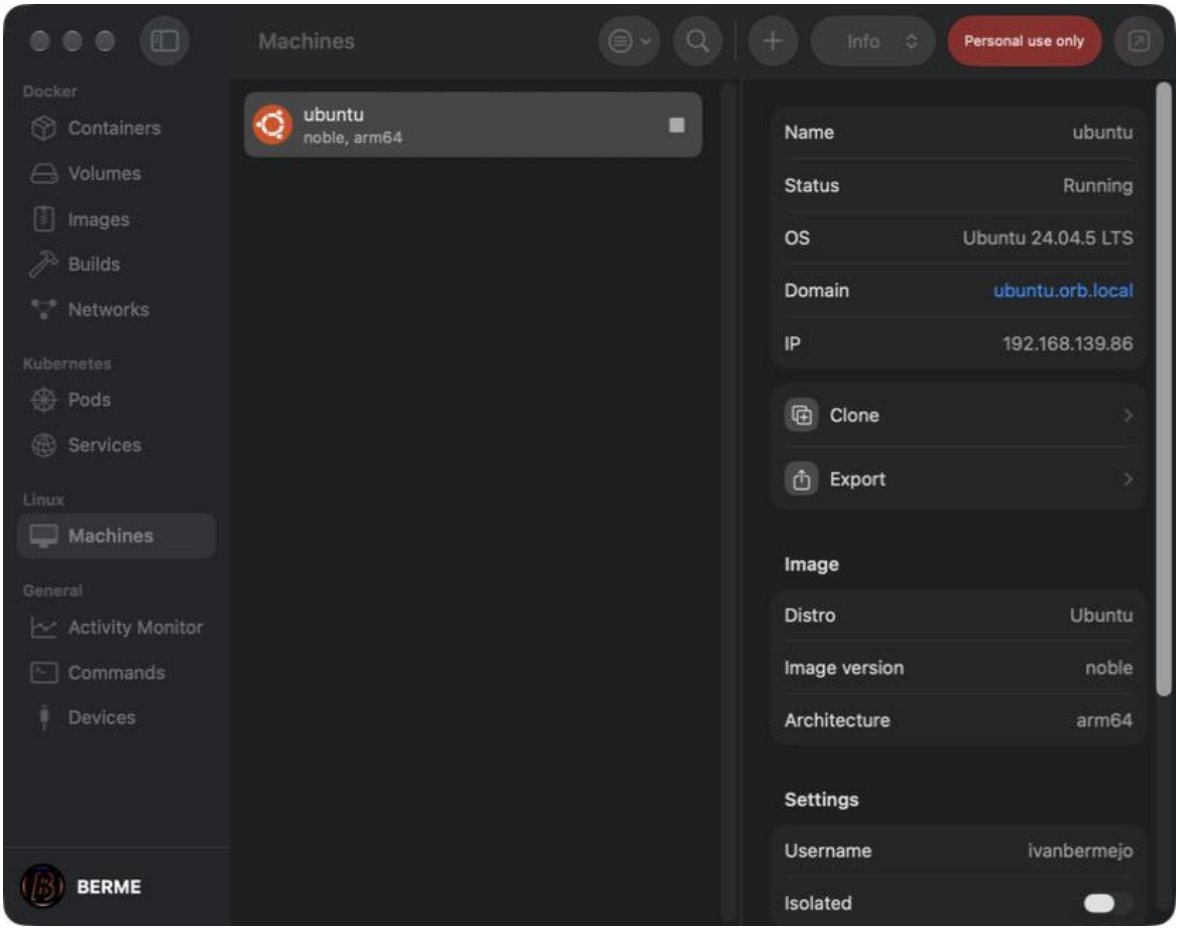


Figura 12. *Máquina «ubuntu» de OrbStack en ejecución (captura 01). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 3*

La máquina está en marcha (*Running*), con Ubuntu 24.04.5 LTS (arm64), el dominio `ubuntu.orb.local` y el usuario `ivanbermejo`. El navegador del Mac llega a Odoo sin configurar la red.

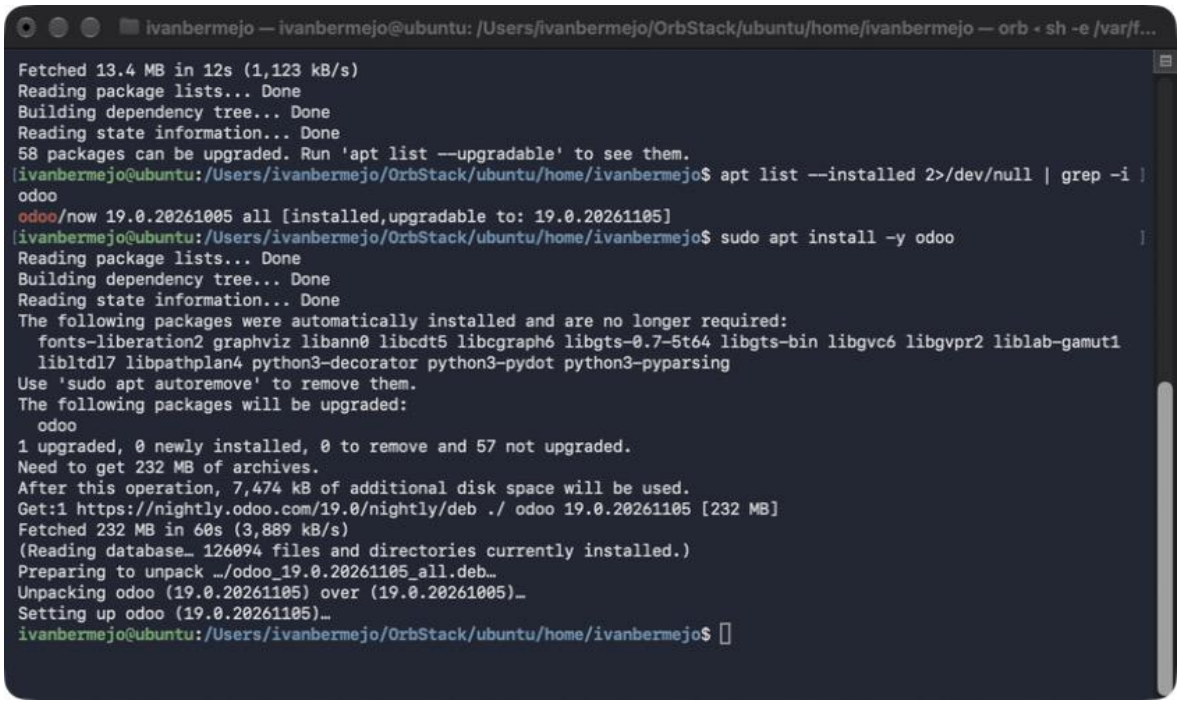


Figura 13. *Actualización del paquete odoo de la 19.0.20261005 a la 19.0.20261105 (captura 03). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 4*

Odoo se instala con `apt` desde `nightly.odoo.com/19.0`. En la captura ya estaba instalado, y el mismo `sudo apt install -y odoo` lo actualiza a la 19.0.20261105.

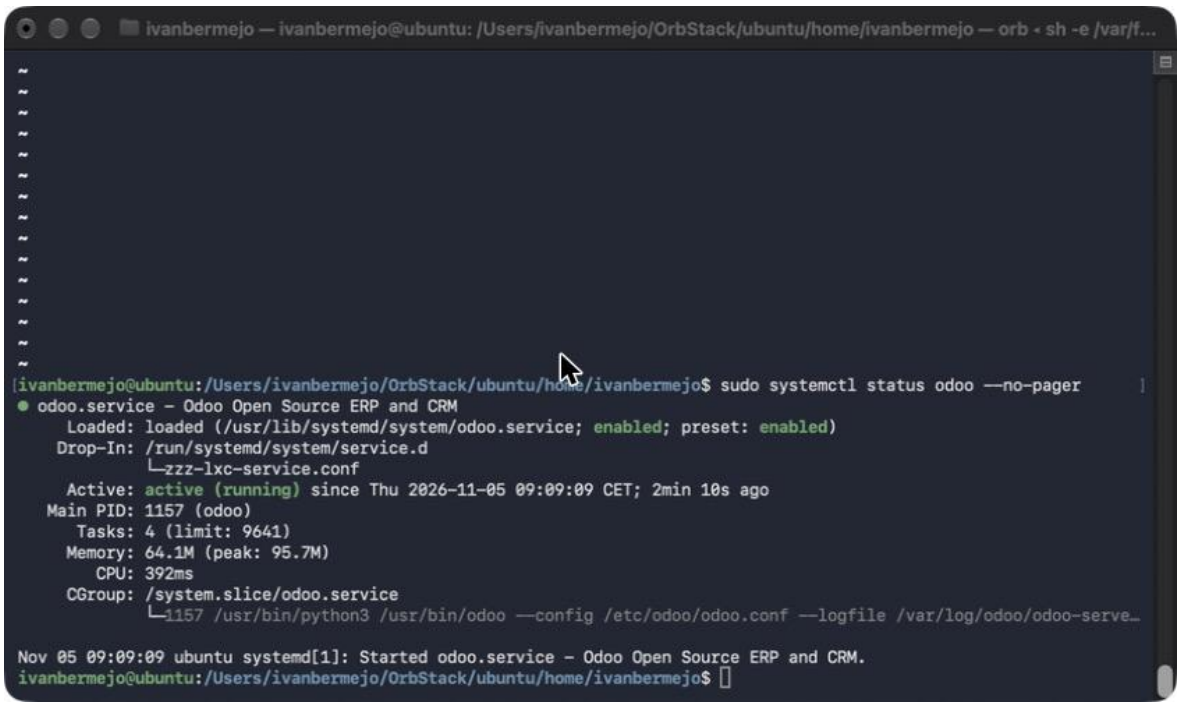


Figura 14. *Servicio odoo activo (captura 04). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 5*

`odoo.service` está habilitado y *active (running)* desde el 5/11/2026 a las 09:09:09. Arranca con `--config /etc/odoo/odoo.conf` y escribe su registro en `/var/log/odoo/`.

Base de datos, compañía y aplicaciones

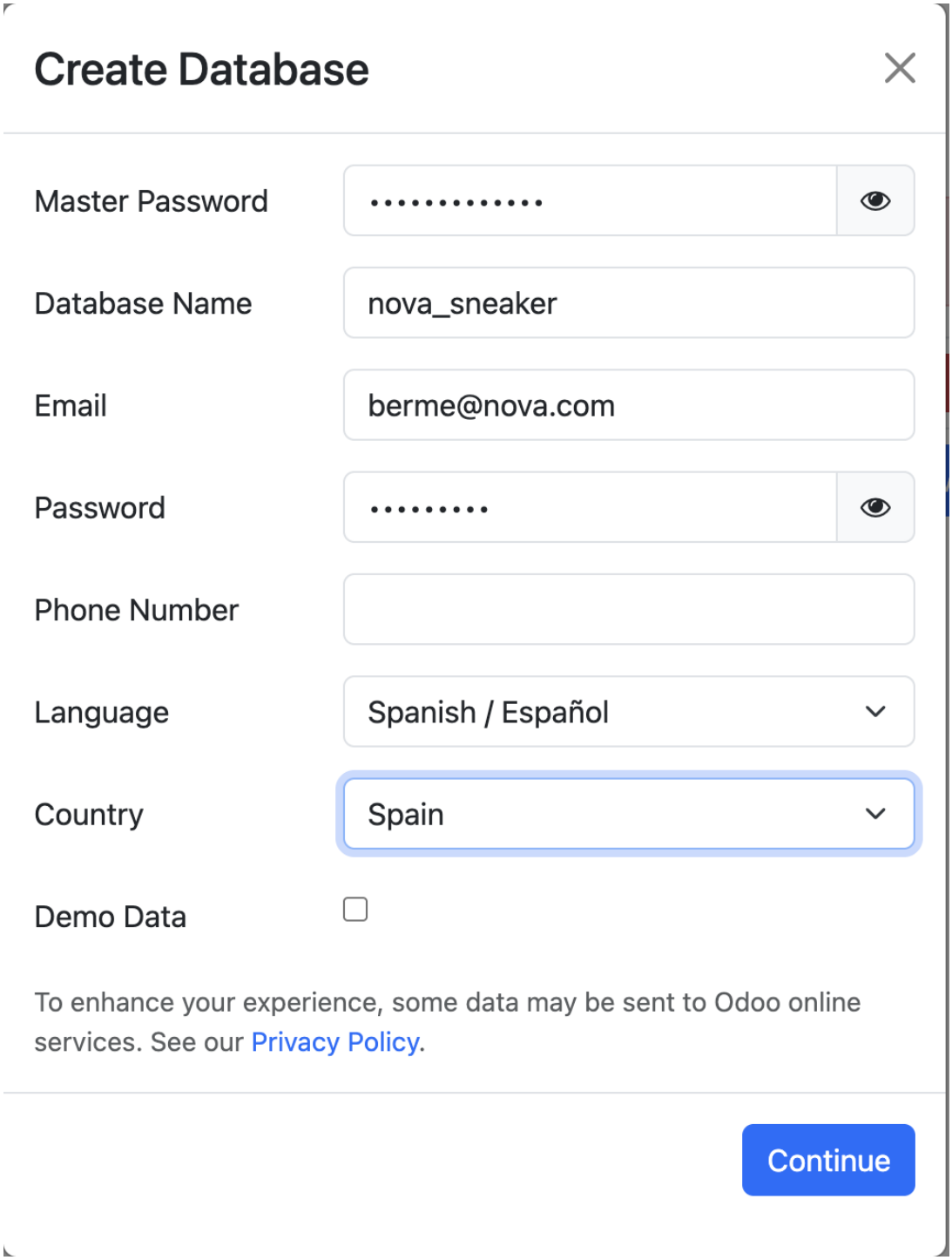


Figura 15. *Creación de la base nova_sneaker: español, España y sin datos de demostración (captura 11). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 6*

La base se crea con el administrador `berme@nova.com` y las contraseñas ocultas. Sin datos de demostración, todo lo que aparece después lo creó el equipo.

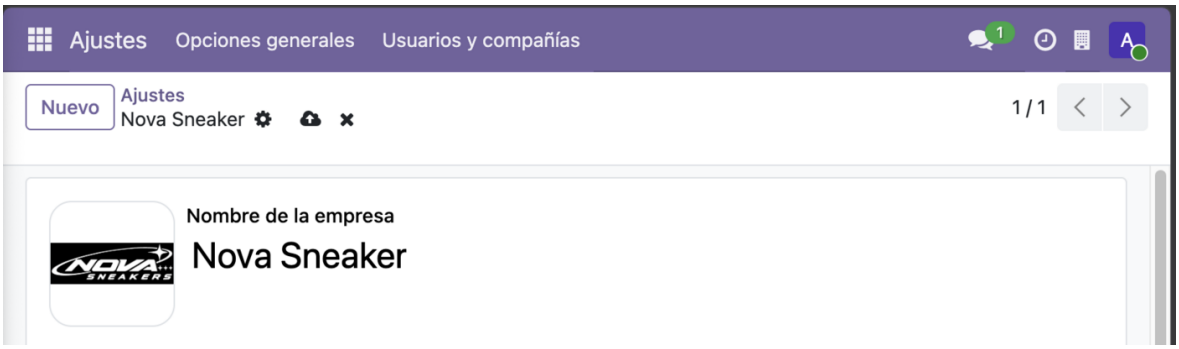


Figura 16. *Compañía Nova Sneaker con su logotipo (captura 12). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 6*

En Ajustes, la compañía se llama «Nova Sneaker» y lleva el logotipo de NOVA Sneakers: es el primer paso para adaptar Odoo a la empresa.

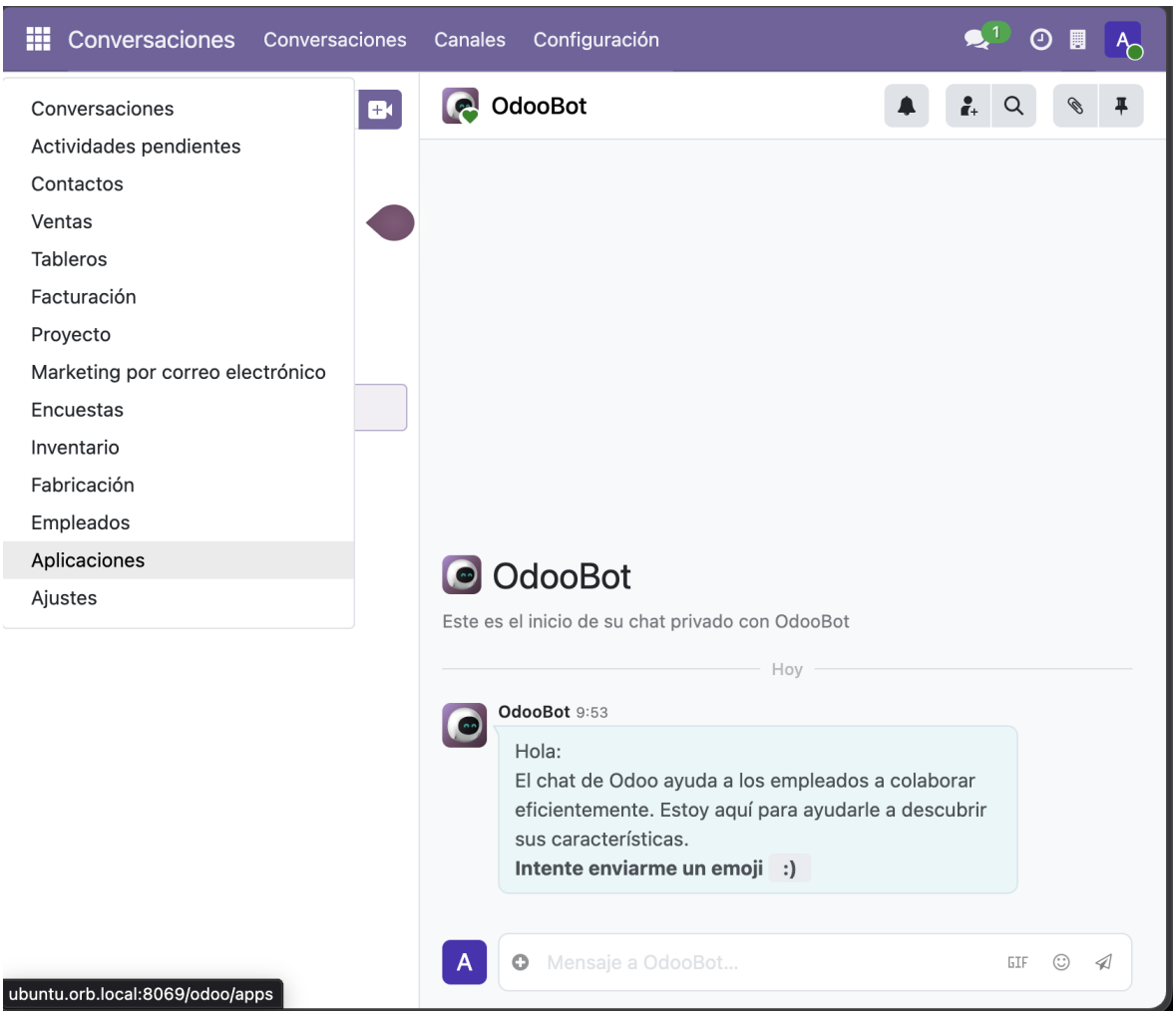


Figura 17. *Menú con las aplicaciones instaladas (captura 05). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 6*

Están Contactos, Ventas, Facturación, Inventario y Fabricación, además de otras que el flujo no usa. No aparece CRM.

Producto terminado y componentes

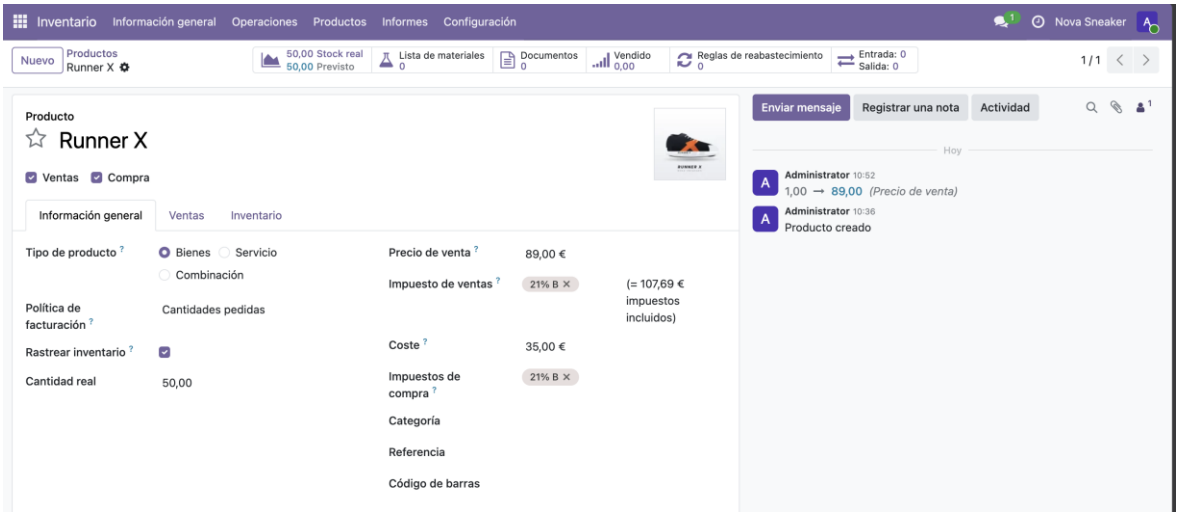


Figura 18. *Ficha de la Runner X con 50 unidades (captura 06). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 7*

La Runner X es de tipo Bienes, con rastreo de inventario, 50 unidades, precio de 89,00 € (107,69 € con IVA) y coste de 35,00 €. El historial registra el cambio de precio de 1,00 a 89,00 € a las 10:52.

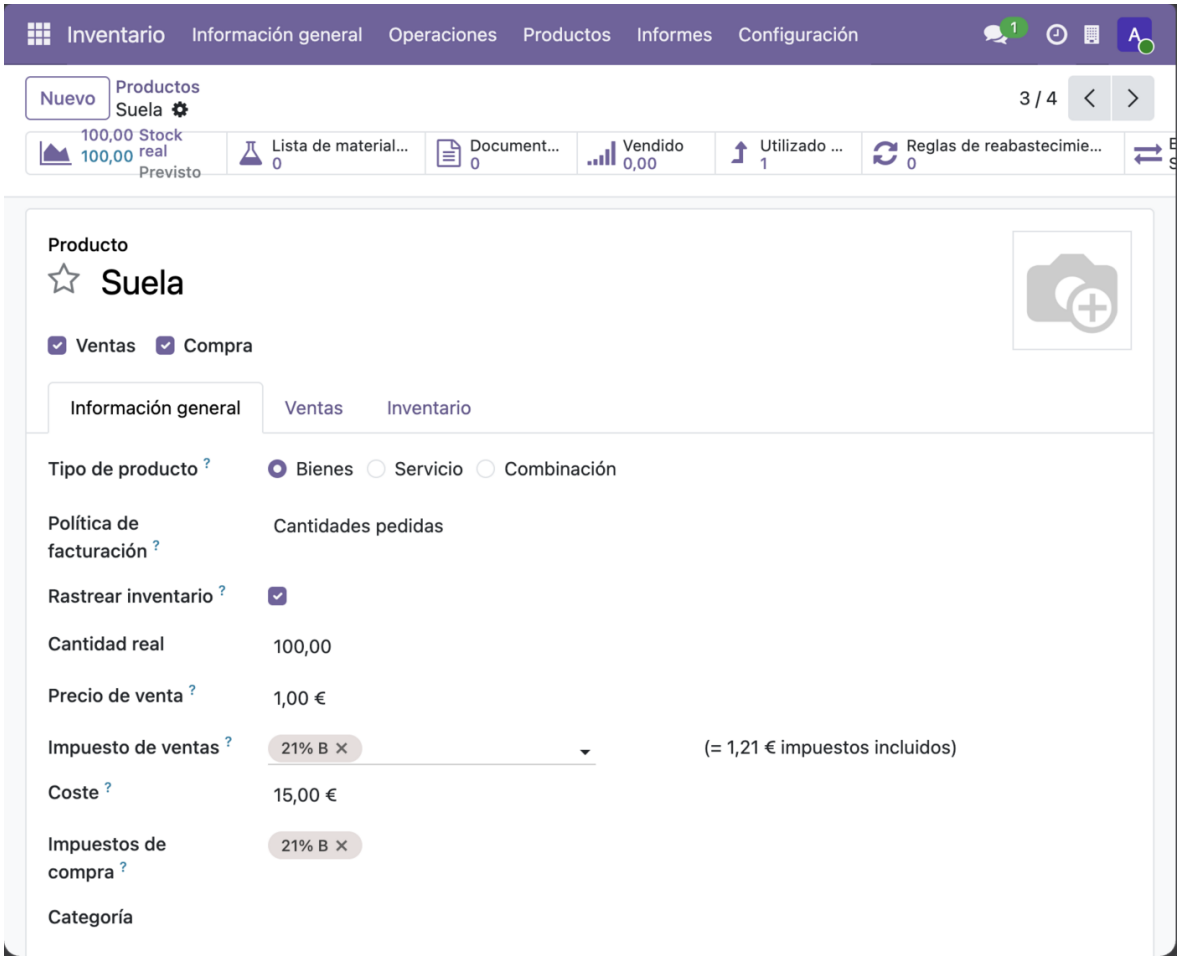


Figura 19. *Componente Suela: producto con inventario, 100 unidades y coste de 15 € (captura 13). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 7*

Los componentes se crean igual que el producto terminado. El botón «Utilizado…» de la Suela marca 1: forma parte de una lista de materiales.

Fabricación de prueba: WH/MO/00001

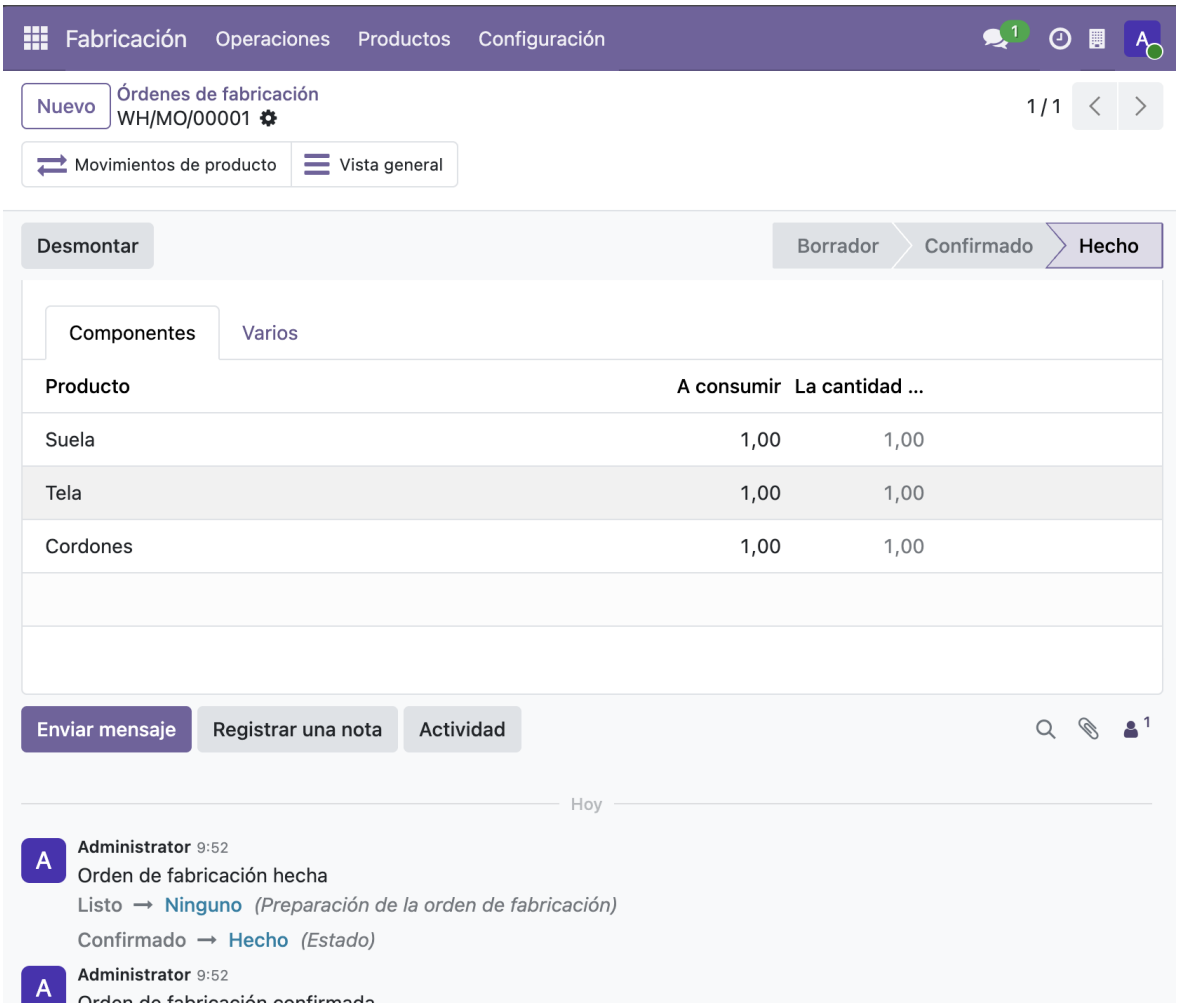


Figura 20. *WH/MO/00001 en estado Hecho: consumido 1 de cada componente (captura 09). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 9*

Una orden de una unidad para probar la lista de materiales. Se terminó el 9/11 a las 9:52 y consumió 1 suela, 1 tela y 1 par de cordones. El historial guarda el paso de Confirmado a Hecho.

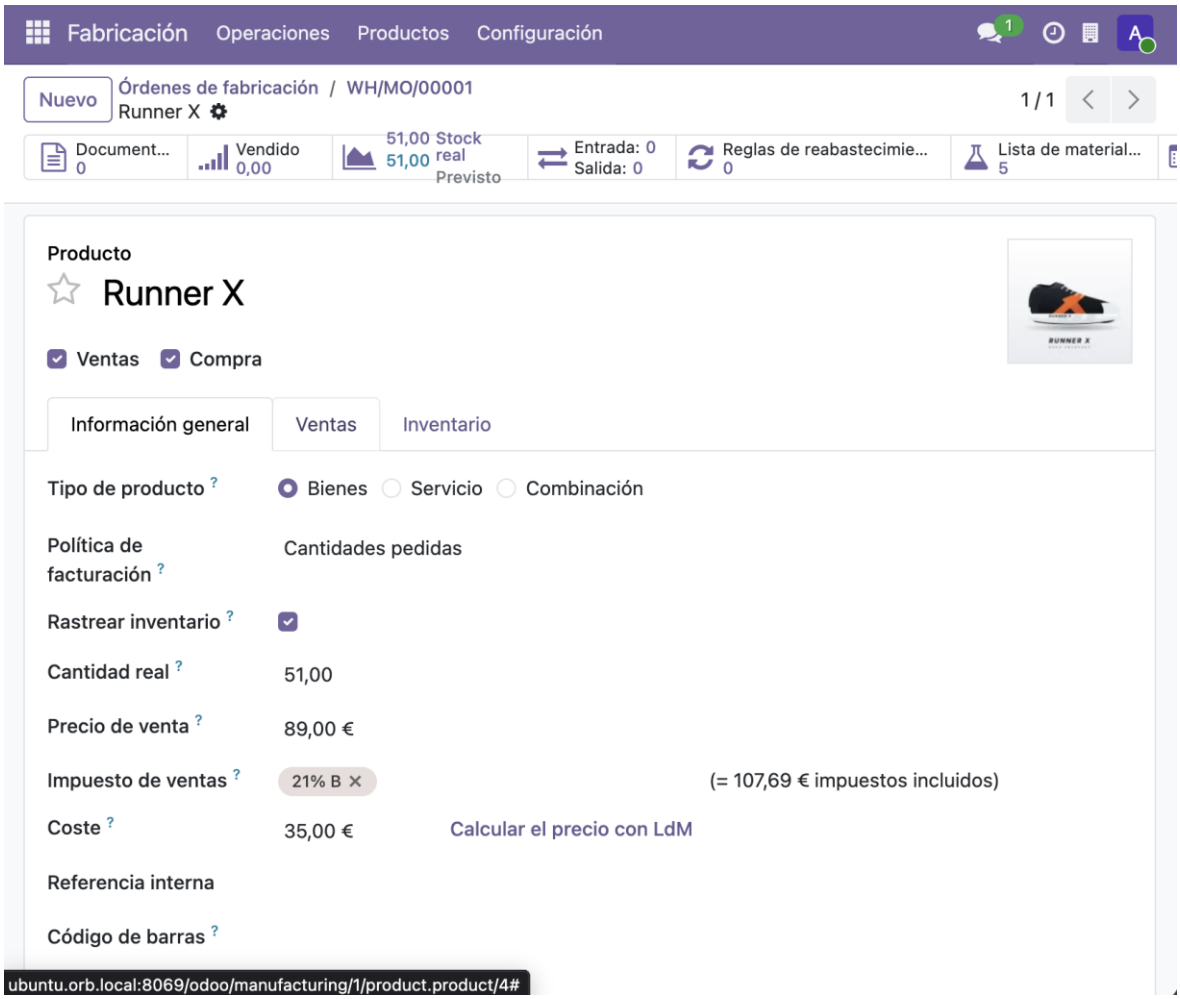


Figura 21. *La Runner X pasa de 50 a 51 unidades (captura 10). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 9*

El producto fabricado entra solo en el inventario: 51 unidades. El botón «Lista de material…» marca 5, porque entonces había cinco listas (apartado 3.3.6).

Caso documentado: S00005 (Club Atletico NOVA)

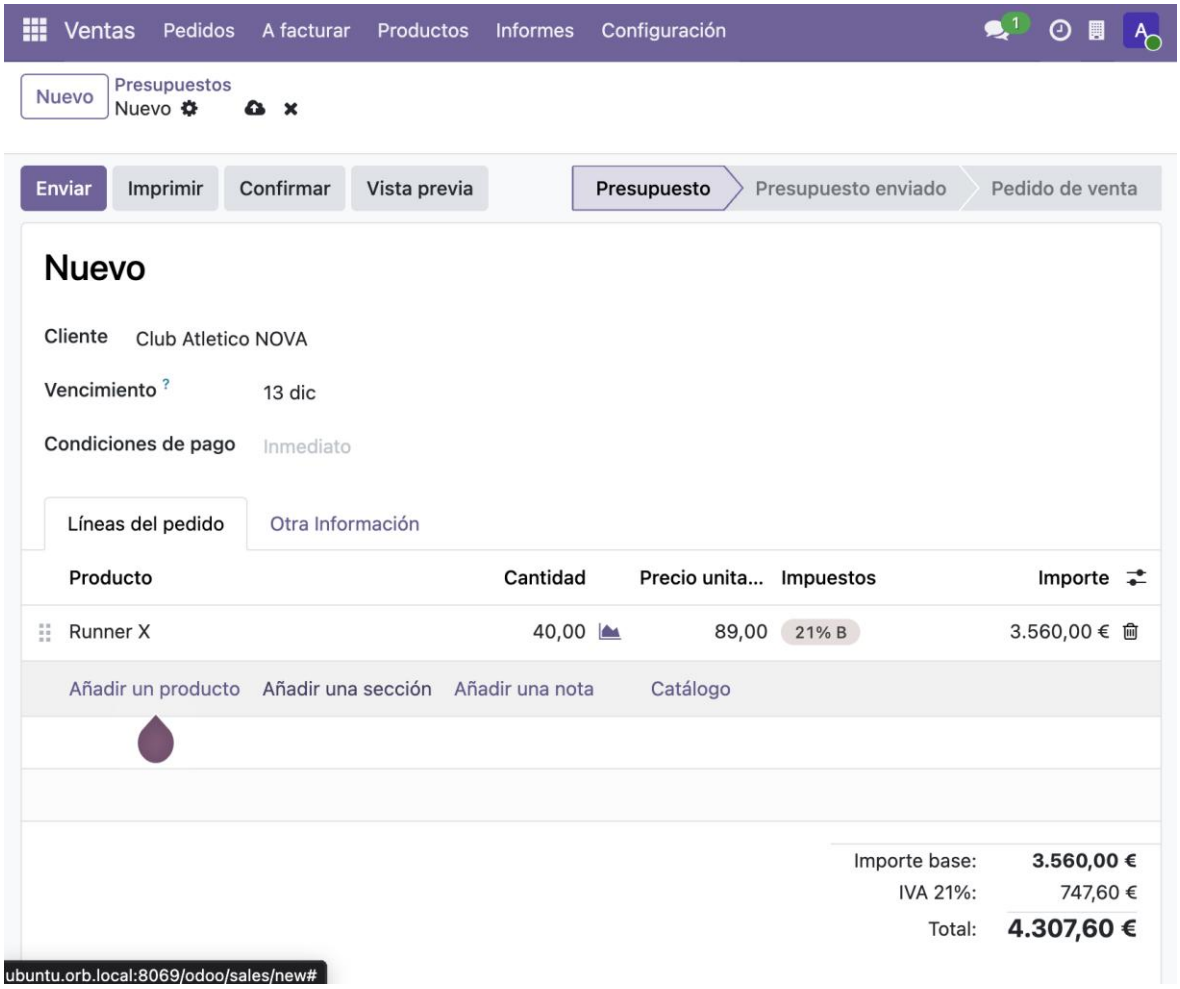


Figura 22. *Presupuesto con 40 Runner X (captura 17). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 10*

Una línea de 40 Runner X a 89,00 € con IVA del 21 %: base de 3.560,00 €, IVA de 747,60 € y total de 4.307,60 €. La línea no lleva talla, y las condiciones de pago muestran «Inmediato» en gris.

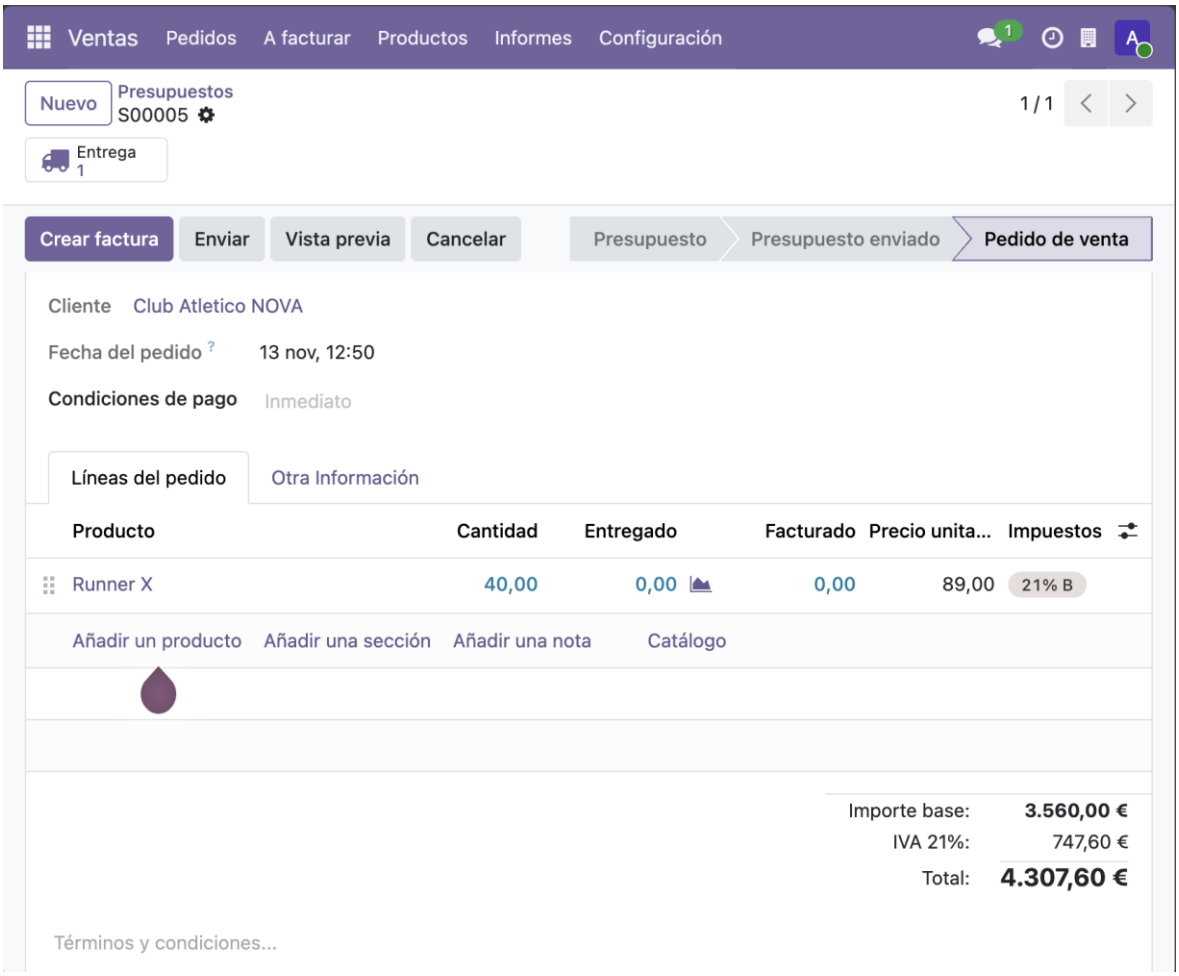


Figura 23. *Pedido S00005 confirmado, con el botón Entrega (captura 18). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 10*

Al confirmar, el presupuesto pasa a «Pedido de venta» S00005, con fecha del 13/11 a las 12:50. Odoo ya ha creado la entrega (botón «Entrega 1»); aún no hay nada entregado ni facturado.

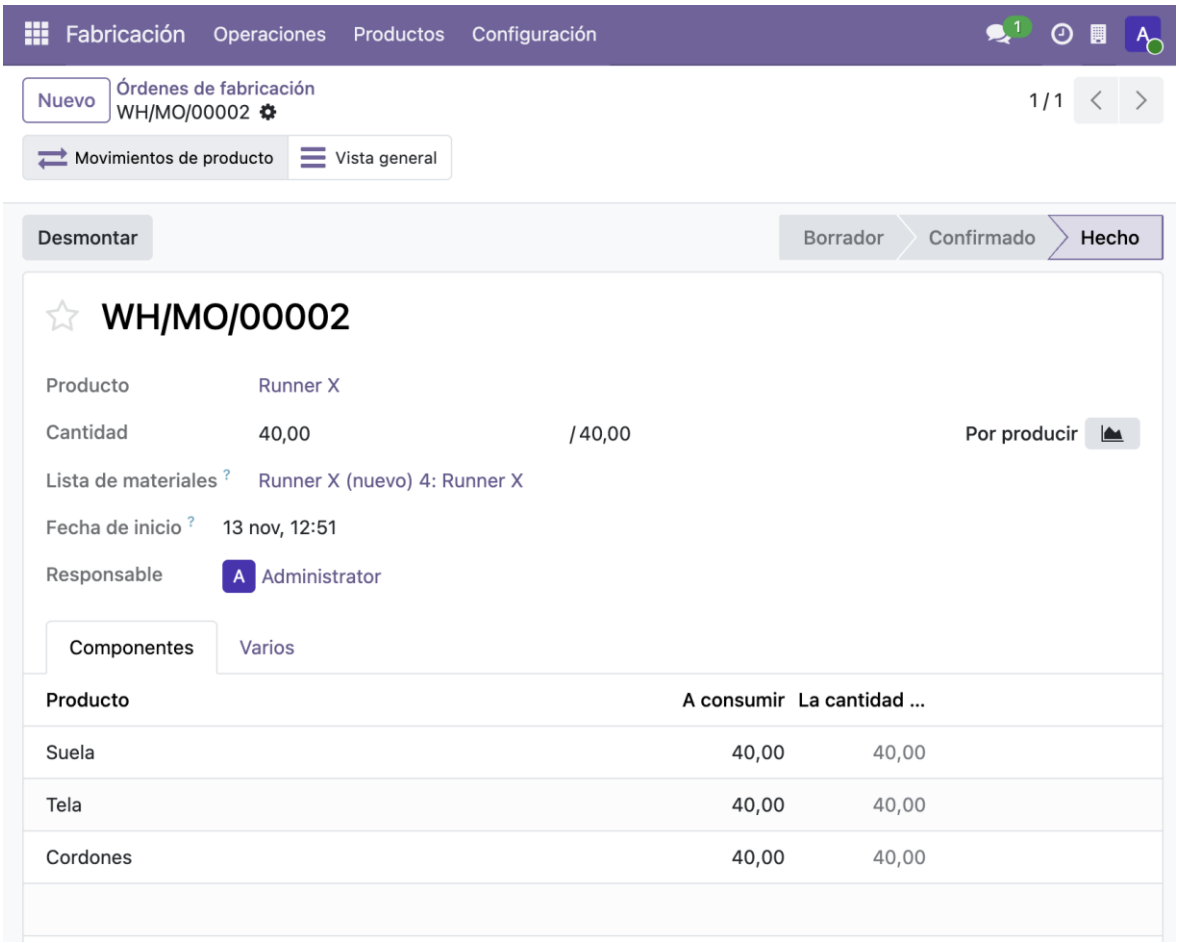


Figura 24. *WH/MO/00002 en Hecho: 40 de 40 (captura 21). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 11*

La orden se creó a mano con la lista «Runner X (nuevo) 4» (capturas 19 y 20, en la misma página). Empezó a las 12:51 y está terminada: 40 de 40, con 40 unidades de cada componente consumidas. La pestaña «Varios», donde estaría el origen, no se ve.

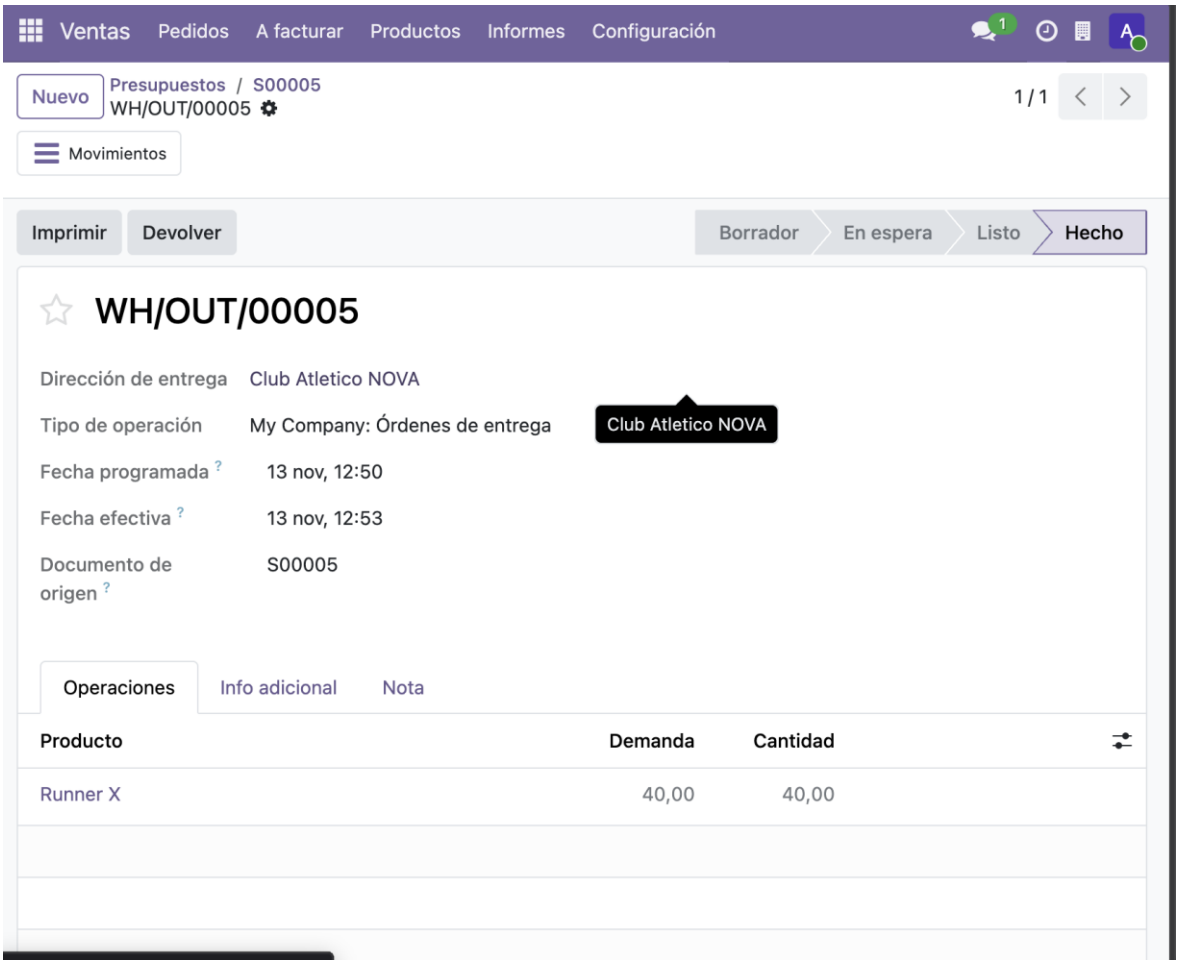


Figura 25. *WH/OUT/00005 en Hecho (captura 23). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 11*

La entrega se abre desde S00005, que es su documento de origen. Va al Club Atletico NOVA y se validó a las 12:53, con 40 de 40 unidades; la captura 22 la muestra antes, en «Listo». El tipo de operación se llama todavía «My Company: Órdenes de entrega».

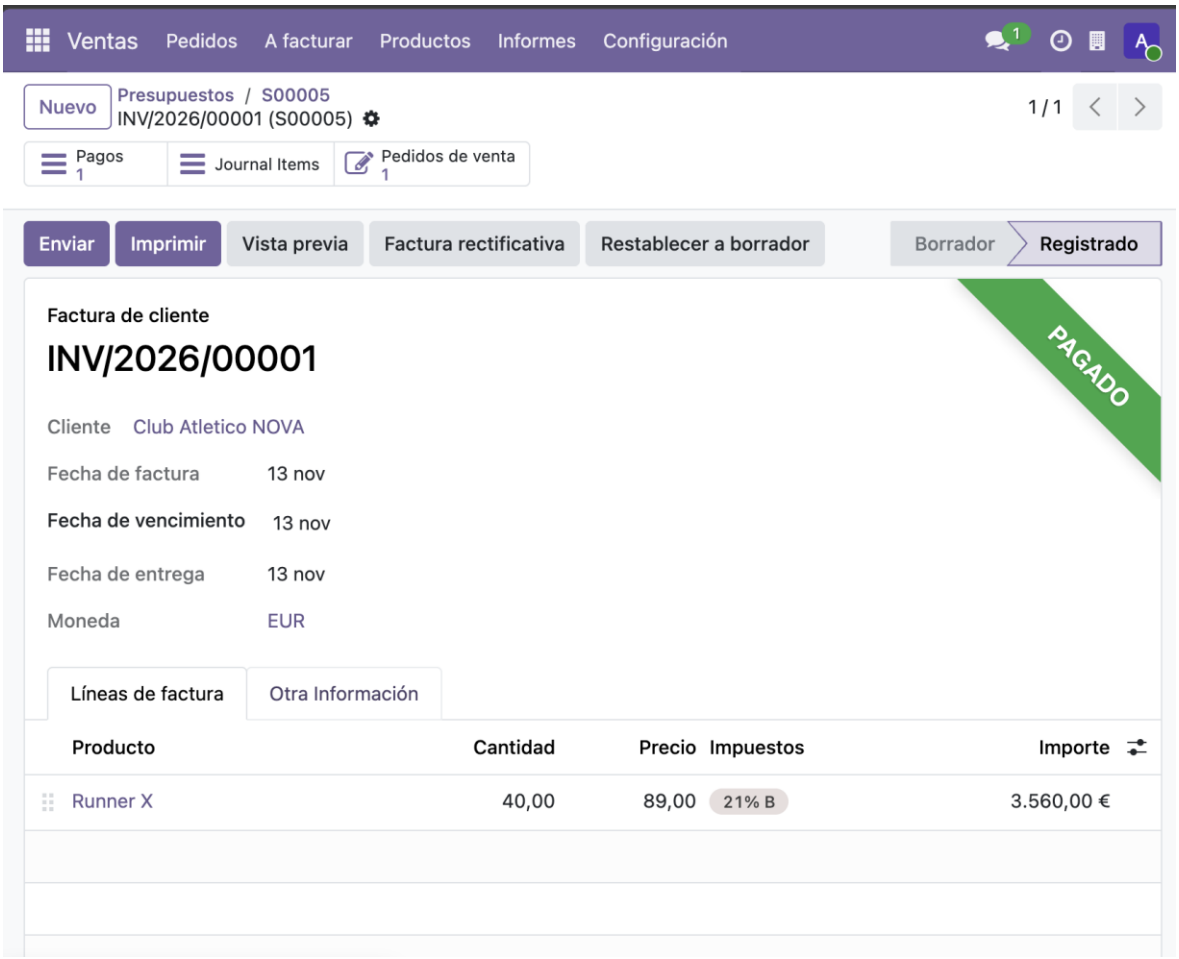


Figura 26. *Factura INV/2026/00001 del Club Atletico NOVA, pagada (captura 24). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 12*

La factura INV/2026/00001 (S00005) está «Registrado» y lleva la cinta «PAGADO». Su línea es 40 × 89,00 € = 3.560,00 €, y los botones «Pagos 1» y «Pedidos de venta 1» la enlazan con su pago y con S00005. El total con IVA no aparece.

Variantes por talla

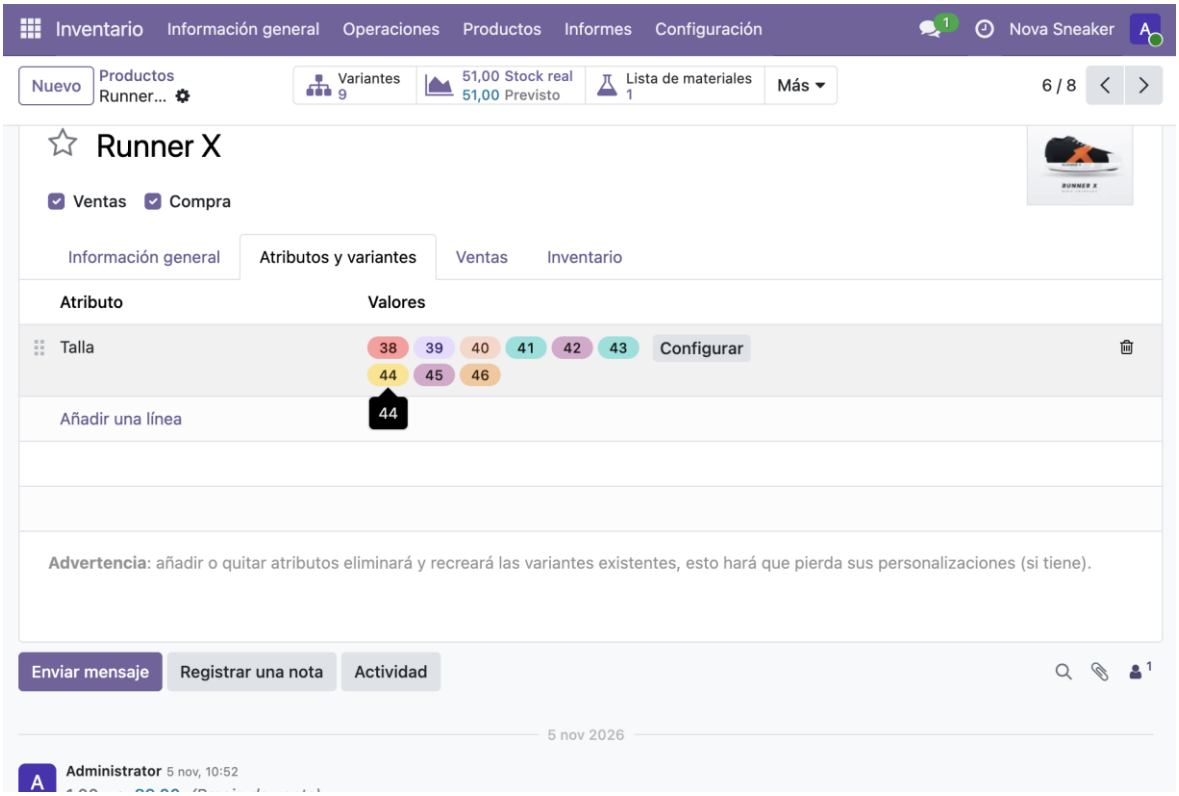


Figura 27. *Runner X: una línea Talla con 9 valores y el botón Variantes (9) (captura 26). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 16*

El atributo Talla tiene los valores 38 a 46 en una sola línea. Los botones muestran 9 variantes, 51 unidades y una sola lista de materiales.

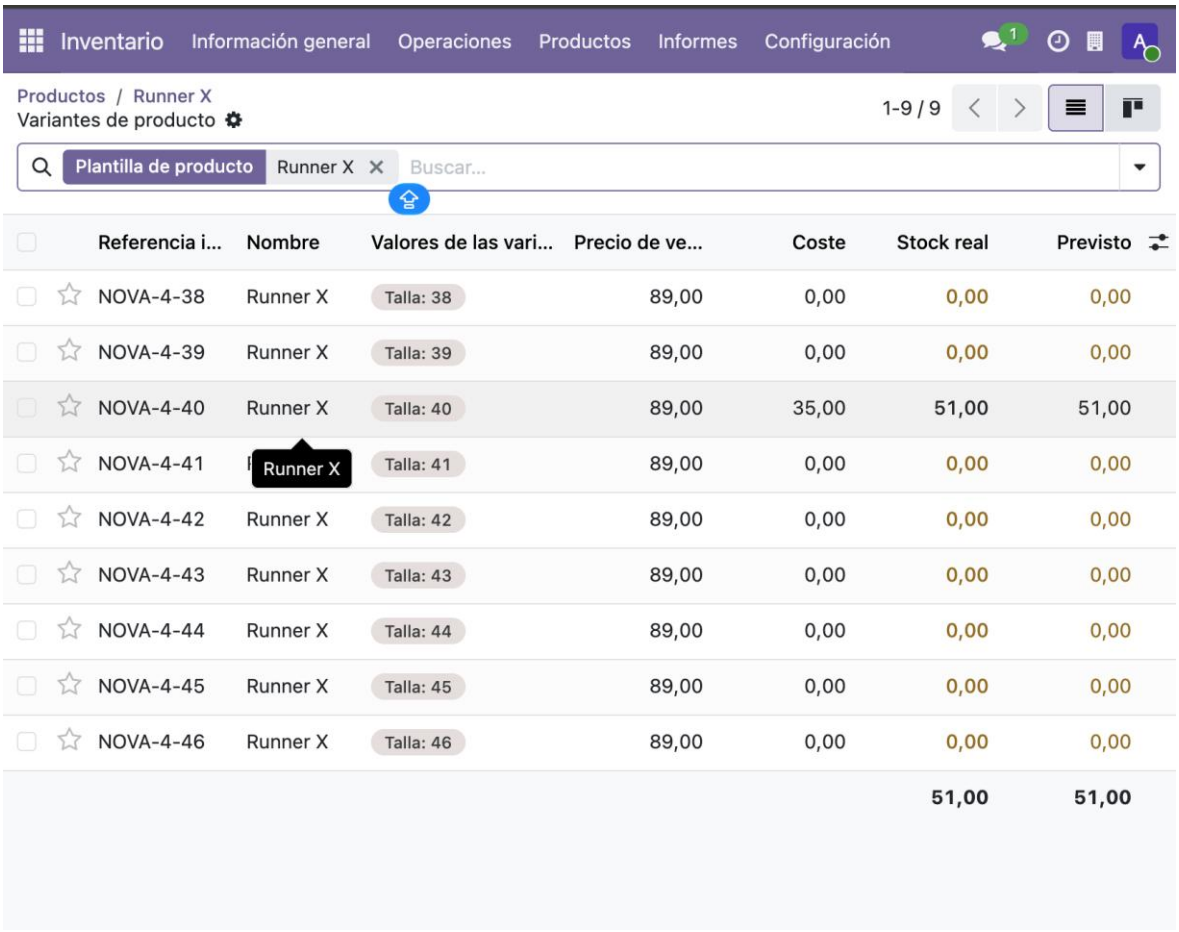


Figura 28. *Variantes de producto de la Runner X con su referencia interna (captura 27). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 16*

Cada talla tiene su referencia, de `NOVA-4-38` a `NOVA-4-46`, y todas cuestan 89,00 €. Las 51 unidades y el coste de 35 € quedaron en la talla 40.

Trazabilidad del stock

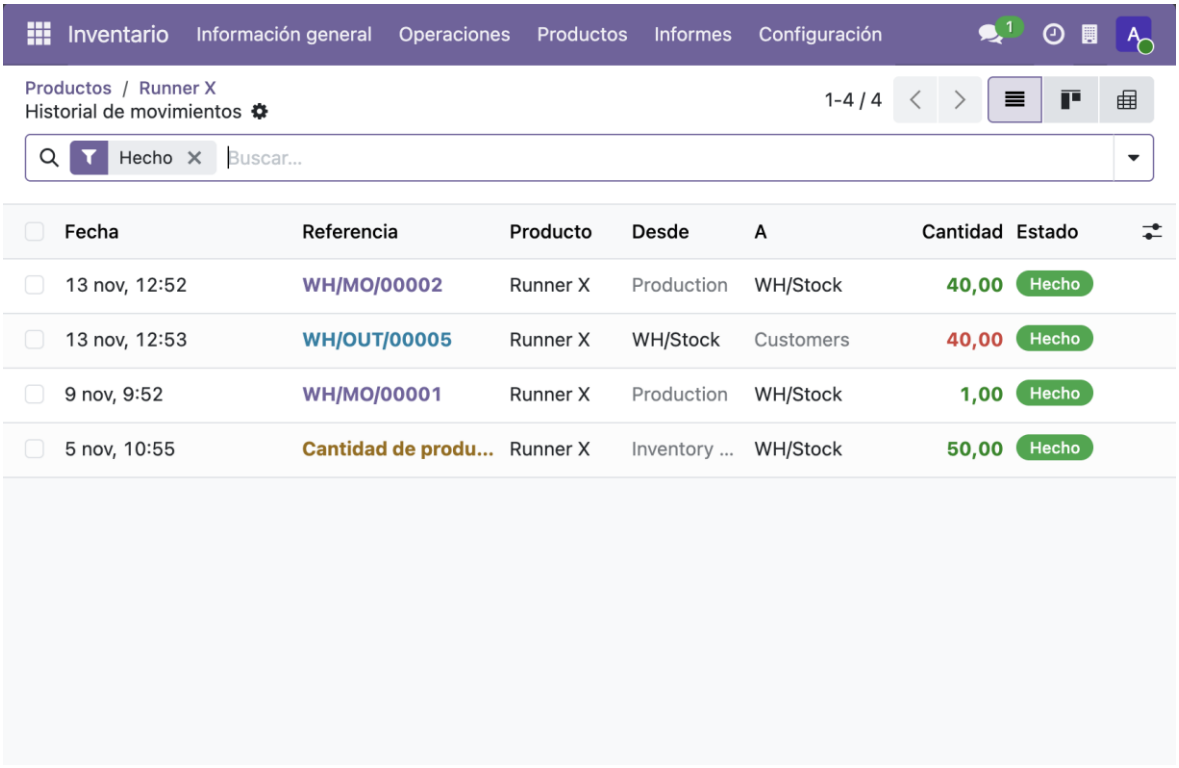


Figura 29. *Historial de movimientos: +50, +1 (WH/MO/00001), +40 (WH/MO/00002) y −40 (WH/OUT/00005) = 51 (captura 25). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 17*

Con el filtro «Hecho» aparecen los cuatro movimientos de la Runner X: el inventario inicial (5/11), la orden de prueba (9/11) y la fabricación y la entrega de S00005 (13/11, a las 12:52 y a las 12:53). El stock sigue al pedido.

Usuarios

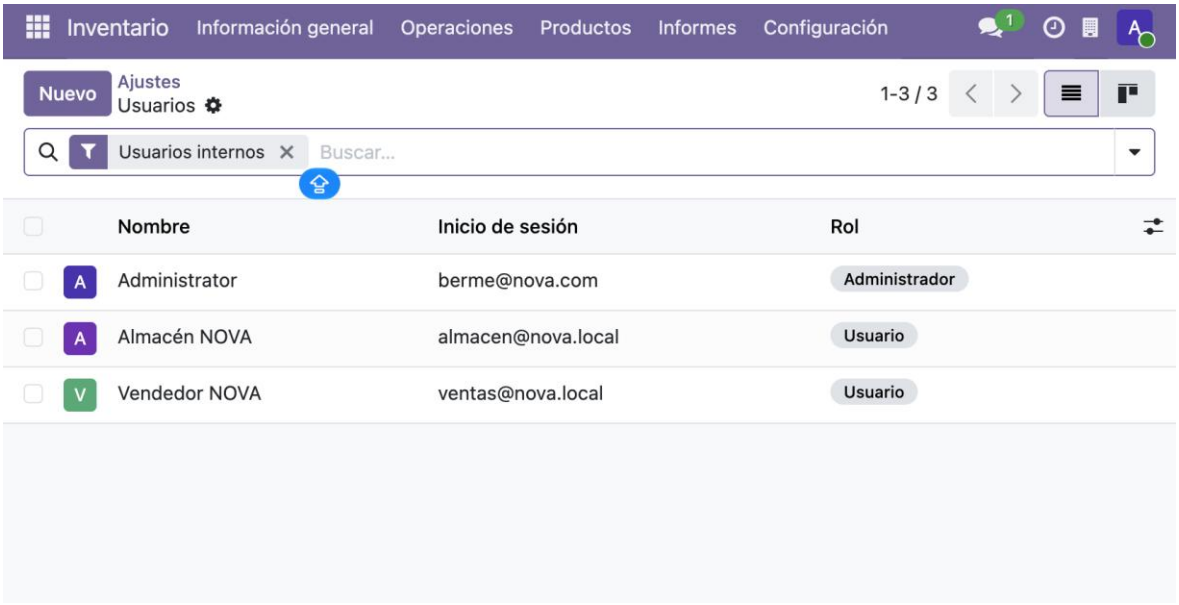


Figura 30. *Usuarios internos de Odoo (captura 28). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 17*

Hay tres usuarios internos: Administrator (`berme@nova.com`, Administrador), Almacén NOVA (`almacen@nova.local`) y Vendedor NOVA (`ventas@nova.local`), con el rol Usuario. Los permisos por aplicación no se ven.

3.3.5 Pruebas

Odoo no tiene una batería automática como PSP. Nuestras pruebas comprueban que los importes, las cantidades y los documentos de las capturas cuadran.

Comprobación	Valor esperado	Lo que muestran las capturas	Resultado
Importe de la venta	40 × 89,00 = 3.560,00 €	Base de 3.560,00 € (17 y 18)	Cuadra
IVA y total	3.560,00 × 0,21 = 747,60 €; total 4.307,60 €	Los mismos importes (17 y 18)	Cuadra
Factura	Igual que la venta	40 × 89,00 = 3.560,00 € (24)	Cuadra. El total con IVA no se ve
Consumo de componentes	1 par: 1 + 1 + 1; 40 pares: 40 + 40 + 40	WH/MO/00001 y WH/MO/00002 (09 y 21)	Cuadra
Stock tras la prueba	50 + 1 = 51	51 unidades (10)	Cuadra
Stock final	50 + 1 + 40 − 40 = 51	Movimientos (25); 51 en la talla 40 (26 y 27)	Cuadra
Venta y entrega	Origen S00005	WH/OUT/00005 con origen S00005 (22 y 23)	Cuadra
Venta y factura	Factura enlazada a S00005	INV/2026/00001 (S00005) (24)	Cuadra
Orden en el tiempo	Venta, fabricación, entrega	12:50, 12:51-12:52 y 12:53 (18, 21, 23 y 25)	Cuadra
Venta y fabricación	Origen S00005 en WH/MO/00002	No se ve	A demostrar en clase

El texto cuenta además pruebas del conector (p. 15): con la clave rechazada (401), el pedido `NOVA-0012` quedó pendiente y, al reintentar, se creó S00003. **No tienen captura** y no se han podido repetir desde la entrega.

Lo que hay que demostrar en vivo en el Mac de Berme

Antes de la defensa hay que comprobar en su Odoo lo que las capturas no enseñan, sin tocar los documentos de S00005:

20. La máquina en marcha, `sudo systemctl status odoo --no-pager` y las versiones de Odoo y PostgreSQL, sin enseñar las contraseñas de `odoo.conf`.
21. Los permisos de Vendedor NOVA y Almacén NOVA.
22. La ficha del Club Atletico NOVA en Contactos.
23. Qué producto muestra hoy la línea de S00005, después de crear las variantes.
24. El origen de WH/MO/00002, en la pestaña «Varios».
25. El total con IVA de INV/2026/00001 y su pago.
26. Las existencias de los componentes: el texto dice que había 99 de cada uno antes de WH/MO/00002 (p. 11), así que deberían quedar 59.

27. El precio actual de la Runner X y de sus variantes.
28. Si existen S00001 a S00004, y si S00001 tiene la referencia `NOVA-0011`.
29. La copia de seguridad: archivo, fecha, tamaño, SHA-256 y restauración.

[BERME: resultado de cada comprobación, con fecha]

3.3.6 Problemas encontrados y soluciones

El PDF cuenta varias incidencias y cómo se resolvieron. Las reunimos aquí, porque documentarlas forma parte de RA1 y RA2.

Problema	Causa	Solución	Dónde (PDF)
En la primera orden, los componentes salían «No disponible»	No había existencias de suela, tela ni cordones	Dar entrada de existencias; la orden pasa a «Disponible»	p. 8, capturas 07 y 08
Había cinco listas de materiales de la Runner X	Pruebas repetidas	Archivar cuatro y dejar «Runner X (nuevo) 4»	p. 8, capturas 16 y 15
Odoo no generaba las variantes	Una línea de atributo por talla	Un solo atributo Talla con los 9 valores en una línea	p. 16, captura 26
Referencias internas repetidas	Productos sueltos `NOVA-4-40` a 43 de las pruebas de la API	Archivarlos antes de dar esas referencias a las variantes	pp. 13 y 16 (texto)
El servidor Node no conectaba con Odoo («fetch failed»)	Node 18 resuelve `localhost` a IPv6 y Odoo escucha en IPv4	Usar `127.0.0.1` en `ODOO_URL`	p. 14 (texto)
Odoo rechazaba la clave de API (401)	Clave no válida	El pedido queda pendiente en PostgreSQL y se reintenta	p. 15 (texto)

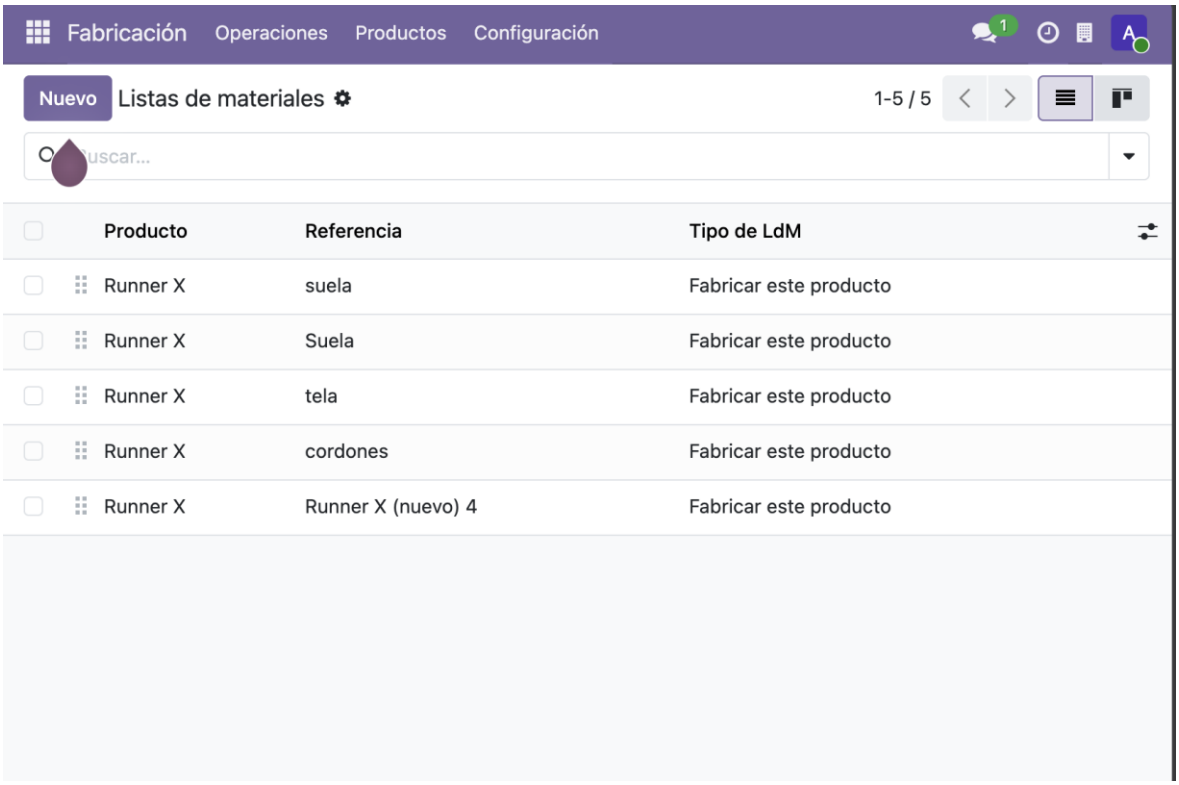


Figura 31. *Antes de la limpieza: cinco listas de materiales de la Runner X (captura 16). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 8*

Con varias listas, cada orden puede tomar una distinta. Después de archivar cuatro, la captura 15 muestra solo «Runner X (nuevo) 4», que es la que usan las dos órdenes de fabricación.

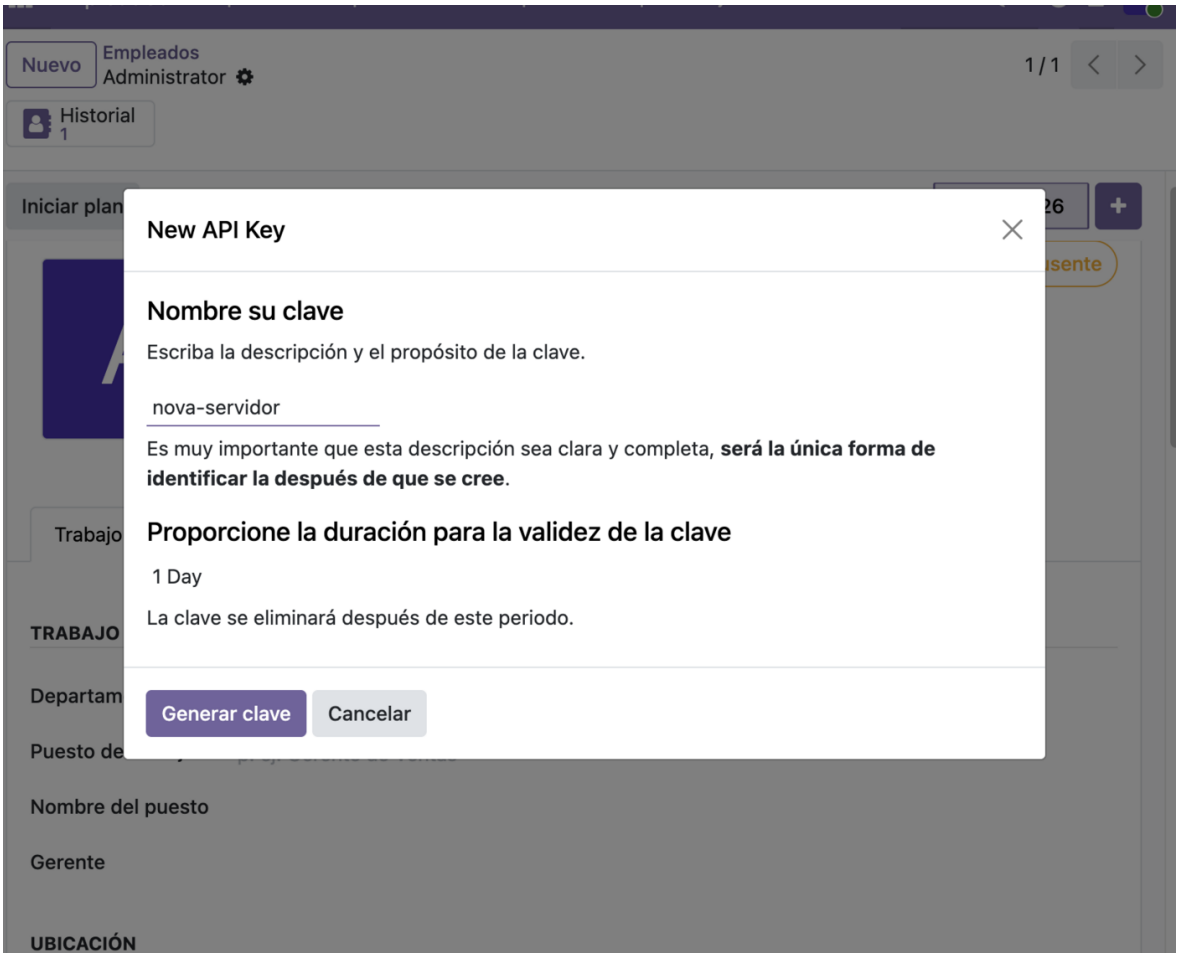


Figura 32. *Creación de la clave de API «nova-servidor», con una validez de 1 día (captura 14). Captura de Iván Bermejo, Odoo-NOVA-Sneakers.pdf, p. 14*

Es la única captura de la conexión por API. La clave caduca al día siguiente, así que no sirve para una demostración posterior, y el texto no lo advierte.

Límites

- **Precio.** El de 99,90 € solo está en el texto; las capturas muestran 89 €.
- **S00005 no es un pedido Custom completo:** una línea sin talla, sin personalización ni referencia del cliente.
- **La fabricación no nace de la venta.** La orden se creó a mano, sin ruta MTO, y la Runner X ya tenía 51 unidades, así que la entrega se podía cubrir sin fabricar.
- **Perfiles incompletos.** Los permisos solo están en el texto y faltan producción y administración. Tampoco hay CRM.
- **Adaptación limitada.** Hay un solo modelo y el almacén conserva el nombre «My Company». La compañía se llama «Nova Sneaker»; el enunciado dice «Sneaker Factory» y el resto del proyecto, «NOVA Sneakers».
- **La conexión por API no forma parte de la entrega:** rutas sin montar, clave de un día y códigos `NOVA-4-40` a `NOVA-4-44` que significan productos distintos en Odoo y en PostgreSQL. El stock de Odoo no se sincroniza con el del proyecto (p. 15).

- **La trazabilidad entre bloques no está demostrada.** S00005 solo existe en Odoo. El pedido que procesa PSP, CUSTOM-40 (NOVA Velocity Rojo, 4.796 €), es otro: solo coinciden el cliente y los 40 pares.

El PDF propone tres mejoras (p. 18): que la venta cree sola la orden de fabricación, sincronizar el stock entre Odoo y PostgreSQL y cargar el resto del catálogo como variantes.

CAPÍTULO 4. IMPLEMENTACIÓN DEL PROYECTO

4.1 Entorno de ejecución

Todo el proyecto, salvo Odoo, se ejecuta en un ordenador con Windows. Odoo está en la máquina Ubuntu del ordenador de Berme.

Pieza	Tecnología	Cómo se pone en marcha
Web de la marca	HTML, CSS y JavaScript	Se abre `01_Interfaces/cliente/index.html` o, con el servidor, `http://127.0.0.1:3000/tienda/`
Servidor (tienda, panel y portal)	Node 24 + Express, con PostgreSQL 17	`npm start` en `servidor-admin`, con un `.env` que no se sube a Git
Subsistema PSP	JDK 17	`javac` y `java -cp clases ProcesadorPedidosCustom <csv>` en `03_PSP`
Odoo	Odoo 19 Community en Ubuntu 24.04 (OrbStack)	`sudo systemctl status odoo` en la máquina de Berme; acceso por `http://ubuntu.orb.local:8069`
Dashboard (página «Análisis de datos»)	Python 3.13 + Streamlit	`streamlit run app_dashboard.py` en `04_Python`; solo lo necesita esa página del panel

El paso a paso desde un clon nuevo está en `06_Intermodular/manuales/MANUAL_INSTALACION.md`. La regresión que se hizo así, el 13/11, está en `06_Intermodular/evidencias/cierre-entrega-20261113/`. Las contraseñas las elige quien instala y nunca se guardan en el repositorio.

4.2 Demostración ante el profesor

La demostración sigue el guion del enunciado (§11). Cada paso se apoya en los requisitos de los apartados 3.1.3, 3.2.3 y 3.3.3.

30. **Empresa y problema** (presentación).
31. **Web:**

- catálogo, búsqueda y ficha;
- versión móvil (390 px);
- solicitud Custom simulada, con sus validaciones;
- recorrido con teclado.

32. **Odoo, en el ordenador de Berme:**

- estado del servicio y aplicaciones;
- usuarios y permisos;
- la venta S00005, la orden de fabricación WH/MO/00002, la entrega WH/OUT/00005, la factura INV/2026/00001 y su pago;
- en la pestaña «Varios» de WH/MO/00002, si su documento de origen es S00005 (ninguna captura lo muestra; hay que comprobarlo en vivo);

- movimientos de stock y variantes de la Runner X.

33. **PSP:**

- exportar un pedido confirmado con el script 15 de PostgreSQL;
- lanzar `ProcesadorPedidosCustom` y enseñar la consola, la ficha y el registro;
- provocar los errores con código 1 (archivo inexistente) y 2 (datos no válidos).

34. **Panel, portal y tienda:** las ampliaciones, si hay tiempo.
35. **Cierre:**

- qué parte es del ERP y qué parte es auxiliar;
- qué ampliaciones harían falta para una integración productiva: conector activo, stock sincronizado y SKU estable.

Comprobación de la trazabilidad entre bloques

No damos por hecho que un mismo pedido recorra todos los sistemas: esa trazabilidad todavía **no está demostrada**. Sin programar nada nuevo, se puede comprobar en clase así:

36. **Odoo:** abrir la venta S00001. Según el texto del PDF de Berme (p. 13), su «Referencia del cliente» es `NOVA-0011` y su importe, 4.796,00 €. **Hay que comprobarlo antes en su Odoo:** esa venta no tiene captura.
37. **PostgreSQL:** en la aplicación Java, consultar y confirmar el pedido `NOVA-0011`. Es del Club Atletico NOVA: NOVA Velocity Rojo, tallas 40 a 43 con 10 pares de cada una, 4.796,00 €.
38. **PSP:**

- exportarlo con `psql -v referencia=NOVA-0011 -f 15_exportar_fabricacion.sql`;
- procesarlo con `ProcesadorPedidosCustom`.

La ficha y el registro llevan la misma referencia que la venta de Odoo.

39. **Tabla de equivalencias:** dejar documentada la correspondencia de referencia, cliente, SKU, tallas, cantidades e importe.

Hay que explicar un detalle: hoy, en Odoo, los códigos `NOVA-4-40` a `NOVA-4-43` activos son variantes de la Runner X. Los productos de S00001 eran unos productos sueltos con esos mismos códigos, que después se archivaron.

S00005 sigue siendo el caso del flujo completo dentro de Odoo. No tiene CSV ni ficha en PSP, y no lo presentamos como integrado.

4.3 Lo que no forma parte de la entrega

- **El conector de Odoo** (`servidor-admin/odoo.js` y `pedidos.js`) está en el repositorio, pero sus rutas **no están montadas** en el servidor entregado (`/api/pedidos` responde 404). Berme documenta pruebas de ese conector en su propio entorno (pp. 13-15 de su PDF), pero no se han verificado desde la entrega.
- **La sincronización automática de stock entre Odoo y PostgreSQL** no existe. El enunciado no la exige.

CAPÍTULO 5. CONCLUSIONES

5.1 Conclusiones del trabajo

- **Los tres módulos funcionan y están probados por separado.**
- La web cumple las páginas que pide el enunciado, es responsive y obtuvo 100 en accesibilidad en Lighthouse.
- El subsistema PSP lanza y coordina tres procesos con control de errores, tiempos y códigos de salida (27 de 27 pruebas, más 8 del analizador).
- Odoo recorre un pedido realista desde la venta hasta la factura, con fabricación y entrega, documentado con capturas.
- **La integración es la parte más débil.**
- La referencia del pedido es el dato común y PSP la conserva de principio a fin.
- Todavía no hemos demostrado un mismo pedido en Odoo, PostgreSQL y PSP.
- El capítulo 4 deja preparada la comprobación con lo que ya existe.
- **Separar lo simulado de lo real ha sido tan importante como programar.** Hay que indicar con claridad:
- la solicitud Custom simulada, el pago simulado y los datos de prueba;
- que el conector de Odoo no está activado.

Así evitamos presentar como terminado algo que no lo está.

- **Trabajar con Git en equipo nos ha enseñado a conservar la autoría y a no reescribir la historia.** La reescritura del 13/11 nos obligó a documentar las equivalencias de los commits (`06_Intermodular/EQUIVALENCIAS_COMMITS.md`).
- **Mejoras futuras:**
- activar y probar el conector de Odoo con autorización y auditoría;
- un SKU estable en PostgreSQL;
- la sincronización de stock;
- la exportación directa de Odoo al formato de PSP;
- procesar varios pedidos de forma concurrente en PSP;
- la prueba de usabilidad con personas.

5.2 Conclusiones personales

Omar: **[OMAR: conclusión personal]**

Álvaro: **[ÁLVARO: conclusión personal]**

Iván Bermejo (Berme): **[BERME: conclusión personal]**

CAPÍTULO 6. ANEXOS

Anexo A. Enlaces y versión entregada

- Repositorio remoto: `https://github.com/omar1676/NOVA` (rama `main`). **[EQUIPO: confirmar el commit final que se entrega]**
- Documentación de entrega: `06_Intermodular/` (memoria común, manuales, matrices de rúbricas y guía de defensa).
- Estado final y límites: `06_Intermodular/ESTADO_ENTREGA_FINAL.md` y `06_Intermodular/CASO_COMUN_S00005.md`.

Anexo B. Documentación original de Odoo (Iván Bermejo)

Reproducimos a continuación, sin modificaciones, las 19 páginas del documento `odoo/documentacion/Odoo-NOVA-Sneakers.pdf`, escrito por Iván Bermejo. El apartado 3.3 de esta memoria se basa en él. El archivo original se conserva en el repositorio tal como lo entregó su autor.



PROYECTO INTERMODULAR · 2.º DAM · SNEAKER FACTORY

Odoo en NOVA Sneakers

Instalación, fabricación e integración con el servidor del proyecto

Autores	Ivan Bermejo, Omar y Álvaro
Tareas realizadas	OD-01 Instalación y configuración · OD-02 Variantes, referencias y usuarios · OD-03 Venta · OD-04 Fabricación · OD-05 Entrega y stock · OD-06 Factura y pago · Conexión de Odoo con el servidor. **Todas hechas.**
Versión	Odoo 19.0 Community sobre Ubuntu 24.04 (OrbStack)
Fecha	13 de noviembre de 2026

Odoo · NOVA Sneakers · 1/19

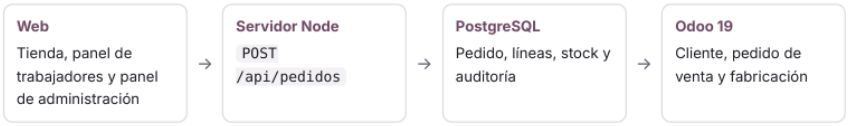
Página 1 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

Índice

1. Entorno: máquina Ubuntu en OrbStack
2. Instalación de Odoo 19 (OD-01)
3. Base de datos y aplicaciones
4. Producto y fabricación de la Runner X (OD-04)
5. Recorrido completo del pedido Custom (OD-03 a OD-06)
6. Productos y ventas creados desde la integración
7. Conexión del servidor con Odoo (API JSON-2)
8. Variantes por talla, usuarios y stock (OD-02)
9. Estado final y pruebas
10. Anexo: comandos útiles

Qué papel tiene Odoo en el proyecto

Odoo es el núcleo empresarial de NOVA Sneakers: clientes, ventas, fabricación, inventario y facturación. PostgreSQL del proyecto (`nova_sneakers`) guarda el caso académico con sus triggers y auditoría, y el servidor Node/Express une los dos mundos: cuando la web crea un pedido, se guarda en PostgreSQL y se envía a Odoo como pedido de venta.



> El enunciado no exige una conexión automática con Odoo (basta con archivos y procedimientos documentados). La conexión por API es una mejora del equipo, descrita en el apartado 7.

Página 2 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

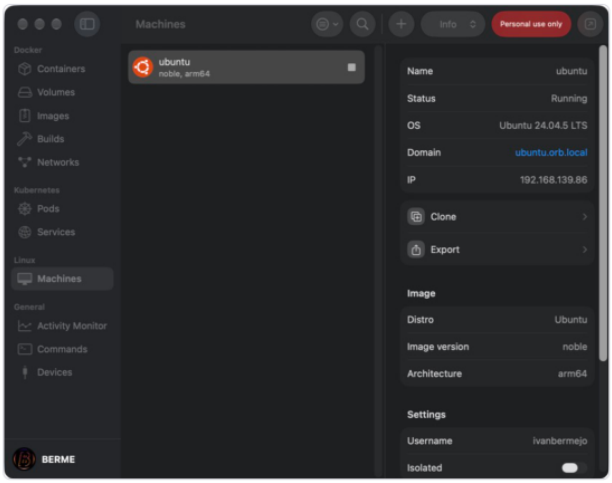
1. Entorno: máquina Ubuntu en OrbStack

Odoo corre en una máquina Linux de OrbStack dentro del Mac. OrbStack crea la máquina con un dominio local propio, así que Odoo es accesible desde el navegador del Mac sin configurar red.

Dato	Valor
Máquina	`ubuntu`
Sistema	Ubuntu 24.04.5 LTS (noble), arquitectura arm64
Dominio	`ubuntu.orb.local`
Usuario	`ivanbermejo`
Acceso a Odoo	`http://ubuntu.orb.local:8069`

Crear la máquina

1. Instalar OrbStack en el Mac.
2. En **Linux → Machines**, pulsar **+**, elegir Ubuntu 24.04 (noble) y llamarla `ubuntu`.
3. Entrar en la máquina desde la terminal del Mac con `orb -m ubuntu` (o `ssh ubuntu@orb`).



Captura 01 · Máquina ubuntu en OrbStack, en ejecución

Página 3 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

2. Instalación de Odoo 19 (OD-01)

Se usa el paquete oficial `.deb` del repositorio nightly de Odoo 19. Así Odoo queda instalado como servicio del sistema y se actualiza con `apt`.

Paso 1 · PostgreSQL

```
sudo apt update
sudo apt install -y postgresql
```

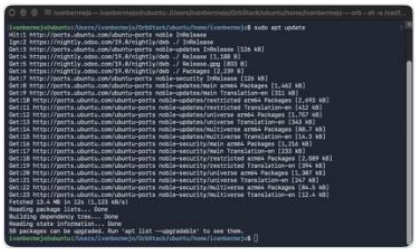
Paso 2 · Repositorio de Odoo 19

```
wget -q -O - https://nightly.odoo.com/odoo.key | sudo gpg --dearmor -o
/usr/share/keyrings/odoo-archive-keyring.gpg
echo 'deb [signed-by=/usr/share/keyrings/odoo-archive-keyring.gpg]
https://nightly.odoo.com/19.0/nightly/deb/ ./' | sudo tee /etc/apt/sources.list.d/odoo.list
sudo apt update
```

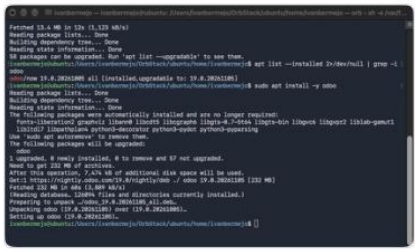
Paso 3 · Instalar Odoo

```
sudo apt install -y odoo
```

En la captura, Odoo ya estaba instalado en la versión 19.0.20261005 y se actualizó a la 19.0.20261105 con el mismo comando.



Captura 02 · `apt update` leyendo el repositorio nightly.odoo.com/19.0



Captura 03 · Instalación y actualización del paquete odoo

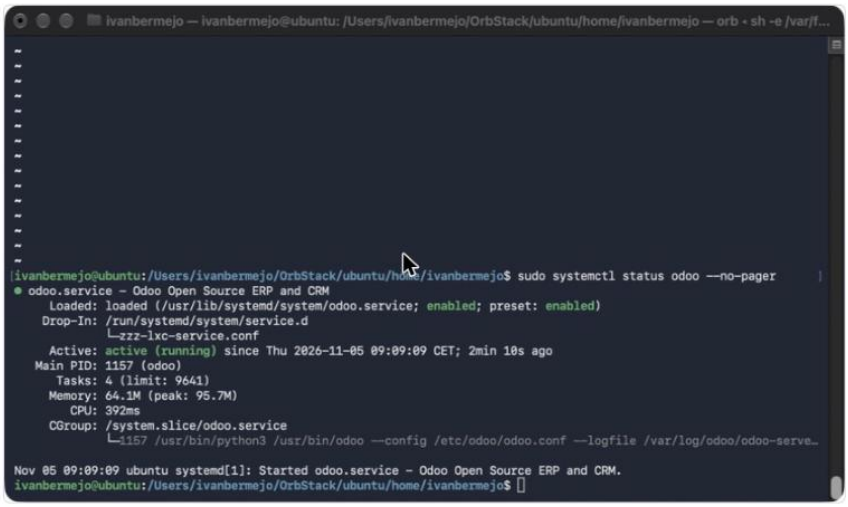
Página 4 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

2. Instalación de Odoo 19 (continuación)

Paso 4 · Comprobar el servicio

```
sudo systemctl status odoo --no-pager
```

El paquete crea el servicio `odoo.service`, activado al arrancar. Debe aparecer **active (running)**.



Captura 04 · Servicio odoo activo

Elemento	Ruta o valor
Servicio	`/usr/lib/systemd/system/odoo.service`
Configuración	`/etc/odoo/odoo.conf`
Registro (log)	`/var/log/odoo/odoo-server.log`
Puerto	8069
Usuario de PostgreSQL	`odoo` (lo crea el paquete)

Paso 5 · Primer acceso

1. Abrir `http://ubuntu.orb.local:8069` en el navegador del Mac.
2. Crear la base de datos (apartado 3) con idioma español y país España.
3. Entrar con el usuario administrador creado en ese paso.

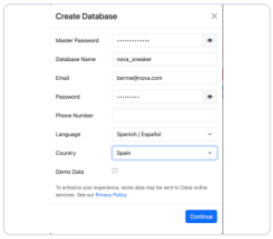
> La contraseña maestra y la del administrador no se escriben en este documento ni se suben al repositorio.

Página 5 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

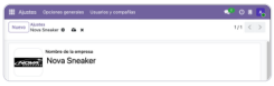
3. Base de datos y aplicaciones

Dato	Valor
Base de datos de Odoo	`nova_sneaker`
Empresa	Nova Sneaker
Idioma / moneda	Español / euro, IVA 21 %
Base del proyecto (aparte)	`nova_sneakers` , en el mismo PostgreSQL, con los scripts de Omar

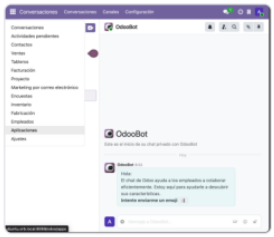
Las dos bases conviven en el mismo servidor PostgreSQL pero son independientes: Odoo no lee las tablas del proyecto ni al revés. La relación entre ellas la hace el servidor Node (apartado 7).



Captura 11 · Creación de la base nova_sneaker: español, España y sin datos de demostración



Captura 12 · Empresa Nova Sneaker con su logotipo (Ajustes → Compañías)



Captura 05 · Menú con las aplicaciones instaladas

Aplicaciones usadas

Aplicación	Para qué se usa
Contactos	Clientes de la tienda (se buscan por email)
Ventas	Pedidos de venta que llegan desde la web
Inventario	Productos, existencias y movimientos
Fabricación	Listas de materiales y órdenes de fabricación
Facturación	Facturas de los pedidos (OD-06)

Se instalan desde **Aplicaciones**, pulsando **Activar** en cada una (captura 05).

Página 6 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

4. Producto y fabricación de la Runner X (OD-04)

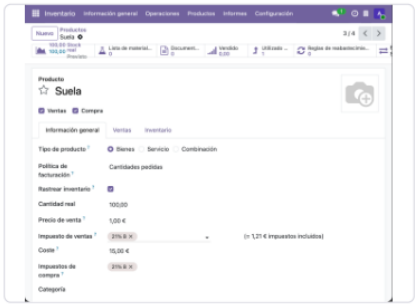
Producto terminado

En **Inventario → Productos → Nuevo** se creó la Runner X como producto fabricado y con inventario.

Campo	Valor
Nombre	Runner X
Tipo	Bienes, con **Rastrear inventario** activado
Se puede	Vender y comprar
Precio de venta	99,90 € (antes 89 €)
Coste	35,00 €
Política de facturación	Cantidades pedidas
Stock inicial	50 unidades



Captura 06 · Ficha de la Runner X con 50 unidades



Captura 13 · Componente Suela: producto con inventario, 100 unidades y coste 15 €

Lista de materiales

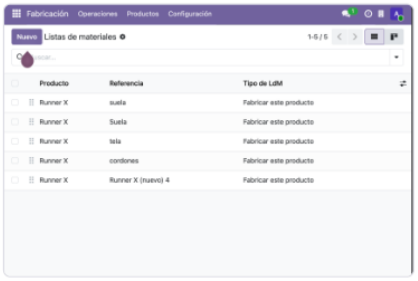
En **Fabricación → Productos → Listas de materiales → Nuevo** se define qué consume un par de Runner X:

Componente	Cantidad por par
Suela	1
Tela	1
Cordones	1

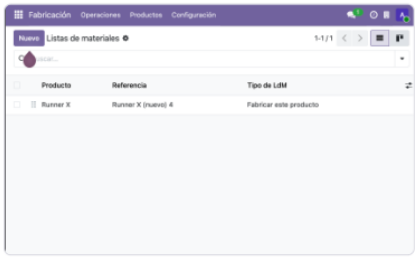
Los componentes se crean como productos de tipo Bienes con inventario, igual que la Runner X.

Página 7 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

Por las pruebas había 5 listas de materiales de la Runner X (captura 16). Se archivaron 4 y se dejó una, `Runner X (nuevo) 4`, para que cada orden use siempre la misma (captura 15).



Captura 16 · Antes: 5 listas de materiales



Captura 15 · Después: una sola lista

4. Fabricación (continuación)

Flujo de una orden de fabricación

1 · Borrador
Se crea la orden con producto, cantidad y lista de materiales

→

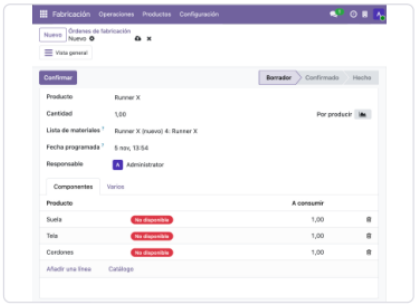
2 · Componentes
Hay existencias de suela, tela y cordones

→

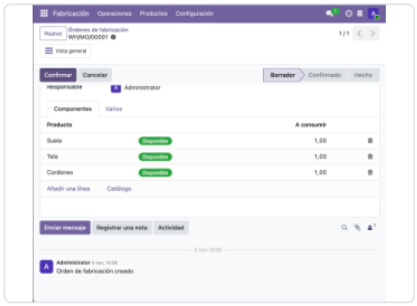
3 · Confirmado
Se reservan los componentes

→

4 · Hecho
Se consumen componentes y entra el producto



Captura 07 · Orden nueva en borrador, sin componentes en stock (5 nov)

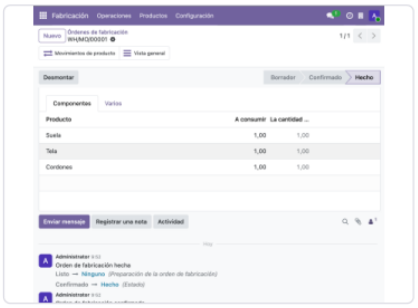


Captura 08 · WH/MO/00001 con los tres componentes disponibles

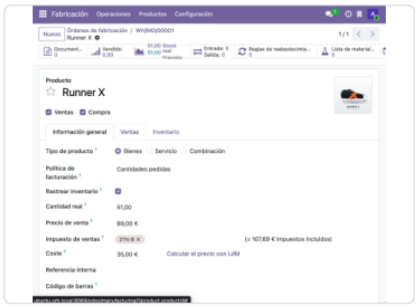
1. **Fabricación → Operaciones → Órdenes de fabricación → Nuevo**. Producto Runner X, cantidad 1. Odoo rellena la lista de materiales y los componentes.
2. Al principio los componentes salen como **No disponible** porque no hay existencias (captura 07).
3. Se da entrada de existencias de suela, tela y cordones desde Inventario. Al volver a la orden pasan a **Disponible** (captura 08).
4. Se pulsa **Confirmar** y después **Producir**. La orden `WH/MO/00001` pasa a **Hecho** (captura 09).

Página 8 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

4. Fabricación (resultado)



Captura 09 · WH/MO/00001 en estado Hecho: consumido 1 de cada componente



Captura 10 · La Runner X pasa de 50 a 51 unidades

Qué demuestra

- La lista de materiales funciona: al terminar la orden se consumen 1 suela, 1 tela y 1 par de cordones.
- El inventario se actualiza solo: la Runner X sube de 50 a 51 unidades y los movimientos quedan en **Movimientos de producto**.
- El historial (chatter) de la orden registra quién la confirmó y la terminó y cuándo.

Relación con el pedido Custom

En el caso del pedido personalizado de 40 pares, la venta se confirma en Ventas y la fabricación se lanza con una orden por la cantidad pedida. Hoy la orden se crea a mano; enlazarla con el estado `en_preparacion` del proyecto queda como mejora (apartado 9). El recorrido completo de un pedido Custom de 40 pares está en el apartado 5.

Página 9 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

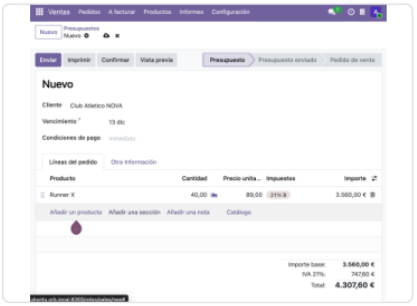
5. Recorrido completo del pedido Custom (OD-03 a OD-06)

Pedido de 40 pares de Runner X para el cliente Club Atletico NOVA, llevado de principio a fin dentro de Odoo el 13 de noviembre de 2026: venta, fabricación, entrega y factura.

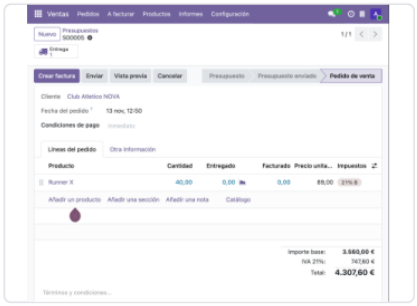
Paso	Documento	Resultado
1 · Venta (OD-03)	`S00005`	40 Runner X a 89,00 €: 3.560,00 € sin IVA, 747,60 € de IVA, 4.307,60 € en total
2 · Fabricación (OD-04)	`WH/MO/00002`	40 de 40 producidas; consume 40 suelas, 40 telas y 40 cordones
3 · Entrega (OD-05)	`WH/OUT/00005`	40 Runner X entregadas al cliente, origen S00005
4 · Factura (OD-06)	`INV/2026/00001`	3.560,00 € más IVA, estado Pagado

1 · Venta

En **Ventas → Pedidos → Presupuestos → Nuevo**: cliente, una línea con la Runner X y cantidad 40, y **Confirmar**. Al confirmar aparece el botón **Entrega** con la entrega ya creada.



Captura 17 · Presupuesto con 40 Runner X

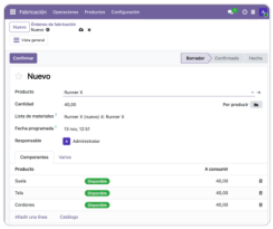


Captura 18 · Pedido S00005 confirmado, con el botón Entrega

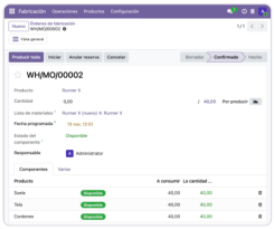
Página 10 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

2 · Fabricación

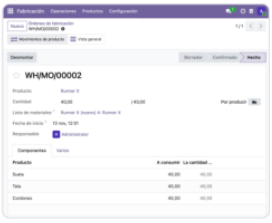
En **Fabricación → Órdenes de fabricación → Nuevo**: Runner X, cantidad 40. Odoo carga la lista de materiales `Runner X (nuevo) 4` y reserva los componentes (en verde porque hay 99 de cada uno). Se pulsa **Confirmar** y **Producir todo**. La orden se creó a mano con el pedido como origen: la venta no genera sola la fabricación, y hacerlo queda como mejora (apartado 9).



Captura 19 · Orden en borrador, componentes disponibles



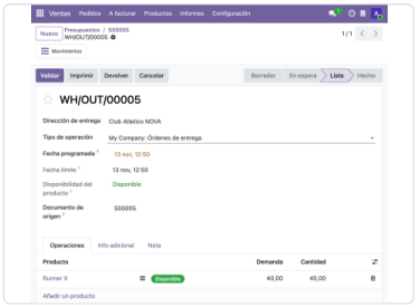
Captura 20 · WH/MO/00002 confirmada



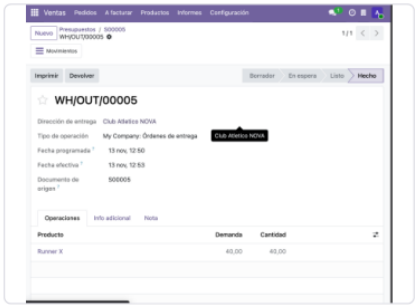
Captura 21 · WH/MO/00002 en Hecho: 40/40

3 · Entrega

Desde el pedido, botón **Entrega → Validar**. La entrega pasa de **Listo** a **Hecho** y sale el stock de las 40 Runner X.



Captura 22 · WH/OUT/00005 en Listo, lista para validar

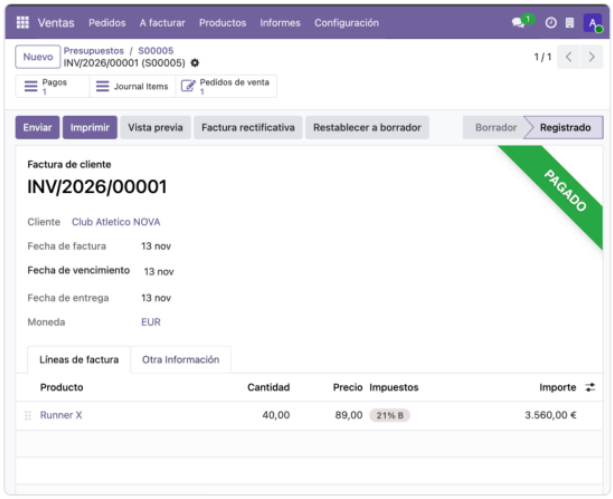


Captura 23 · WH/OUT/00005 en Hecho (13 nov, 12:53)

Página 11 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

4 · Factura

Desde el pedido, **Crear factura → Confirmar → Registrar pago**. La factura queda registrada y marcada como **Pagado**, enlazada al pedido S00005 y a su pago.



Captura 24 · Factura INV/2026/00001 del Club Atletico NOVA, pagada

Página 12 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

6. Productos y ventas creados desde la integración

Para probar la conexión se crearon en Odoo, con un script en Python (XML-RPC) y después con el servidor Node, los datos que usan los pedidos de la web. La clave que une Odoo con PostgreSQL es la **referencia interna** del producto, igual al SKU del proyecto (`NOVA-<id_variante>-<talla>`).

Qué	Dónde en Odoo	Detalle
Cliente	Contactos	Se busca por email; si no existe, se crea
4 productos	Inventario → Productos	`NOVA-4-40` a `NOVA-4-43` , 20 unidades cada uno (después archivados al crear las variantes, apartado 8)
Pedidos de venta	Ventas → Pedidos	Ver tabla siguiente

Pedido Odoo	Referencia del cliente	Origen	Importe sin IVA
S00001	NOVA-0011	Script Python con un pedido de prueba de la BD de Omar	4.796,00 € (5.803,16 € con IVA)
S00002	NOVA-PRUEBA-1	Prueba del conector (repetida: no se duplica)	—
S00003	NOVA-0012	Web → servidor, tras un fallo y reintento	119,90 €
S00004	NOVA-0013	Web → servidor, 2 líneas y personalización	599,50 €

Odoo numera sus pedidos por su cuenta (`S0000N`) y guarda la referencia del proyecto en **Referencia del cliente** (`client_order_ref`). Así cualquier pedido se encuentra en los dos sistemas con la misma referencia.

> Estos productos de prueba se sustituyeron después por las variantes de la Runner X con la misma referencia interna (apartado 8). El precio de la Runner X se ajustó después a 99,90 €, el de PostgreSQL.

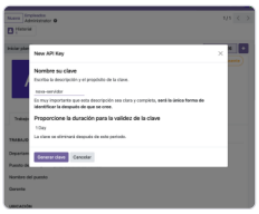
Página 13 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

7. Conexión del servidor con Odoo (API JSON-2)

Odoo 19 tiene una API JSON nueva (`/json/2`). El servidor la llama con `fetch` , sin librerías extra. El código está en `servidor/odoo.js` y `servidor/pedidos.js` .

Clave de API

1. En Odoo: avatar → **Mi perfil / Preferencias → Seguridad → Nueva clave de API**.
2. La clave solo se muestra una vez. Se guarda en el `.env` del servidor, nunca en el repositorio.



Captura 14 · Creación de la clave de API «nova-servidor»

```
ODOO_URL=http://127.0.0.1:8069
ODOO_DB=nova_sneaker
ODOO_API_KEY=(clave de API)
```

> Usar `127.0.0.1` y no `localhost` : Node 18 resuelve `localhost` a IPv6 y Odoo escucha en IPv4, y la conexión falla con `fetch failed` .

Formato de las llamadas

```
POST {ODOO_URL}/json/2/<modelo>/<método>
Authorization: bearer <clave de API>
X-Odoo-Database: nova_sneaker
Content-Type: application/json

{ "domain": [["default_code", "=", "NOVA-4-42"]], "fields": ["id", "name"] }
```


Página 14 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

7. Conexión (continuación)

Qué hace el conector al recibir un pedido

Paso	Modelo y método
1. ¿Ya existe el pedido?	`sale.order / search_read` por `client_order_ref`. Si existe, lo devuelve (`yaExistia: true`) y no crea otro
2. Cliente	`res.partner / search_read` por email, o `create`
3. Productos	`product.product / search_read` por `default_code` = SKU
4. Crear la venta	`sale.order / create` con sus líneas y la personalización en la nota
5. Confirmar	`sale.order / action_confirm`
6. Comprobar	`sale.order / read`: compara `amount_untaxed` con el total de PostgreSQL

Endpoints del servidor

Método	Ruta	Qué hace
POST	`/api/pedidos`	Crea el pedido en PostgreSQL (una transacción) y lo envía a Odoo
GET	`/api/pedidos/odoo-pendientes`	Lista los pedidos guardados que Odoo aún no tiene
POST	`/api/pedidos/:id/reintentar-odoo`	Reenvía a Odoo un pedido pendiente

Ejemplo de petición

```
curl -s -X POST http://localhost:3000/api/pedidos \
  -H "Content-Type: application/json" \
  -d '{"id_cliente":1,"lineas":[{"id_producto_sku":4,"cantidad":1}]}'
```

Si Odoo falla

- El pedido se queda guardado y confirmado en PostgreSQL. Odoo nunca bloquea una venta.
- Se registra `odoo_pendiente` en `log_eventos` con el error, y la respuesta lleva `odoo.ok = false`.
- Cuando Odoo vuelve, `reintentar-odoo` lo envía y se registra `odoo_ok`.
- Probado con NOVA-0012: con la clave rechazada (401) quedó pendiente, y al reintentar se creó S00003.

Stock

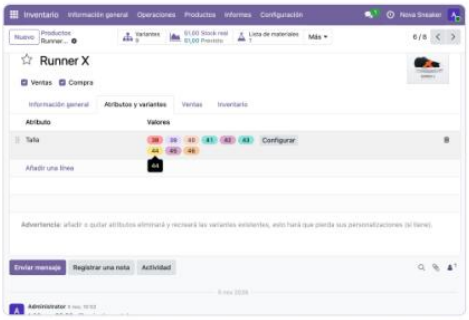
El stock de las ventas web lo controla PostgreSQL (trigger al pasar a `confirmado`). Odoo lleva su propio inventario, que se mueve con la fabricación y las entregas, pero no se sincroniza con el del proyecto.

Página 15 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

8. Variantes por talla, usuarios y stock (OD-02)

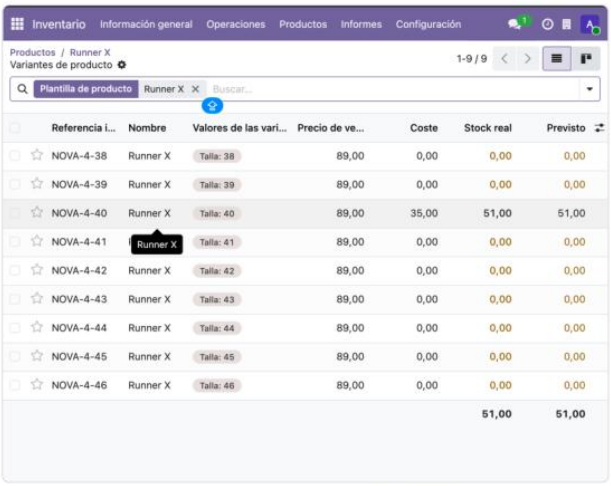
Variantes de la Runner X

Se activó **Variantes** en Inventario → Configuración → Ajustes. En la ficha de la Runner X, pestaña **Atributos y variantes**, se creó el atributo **Talla** (creación de variantes al instante) con los valores 38 a 46, en **una sola línea**. Con una línea por talla Odoo no genera variantes; con una sola línea crea 9, una por talla.



Captura 26 · Runner X: una línea Talla con 9 valores y el botón Variantes (9)

A cada variante se le puso la **referencia interna** igual al SKU del proyecto, `NOVA-4-38` a `NOVA-4-46`. Es la clave por la que el servidor encuentra el producto al crear el pedido de venta. Antes se archivaron los productos sueltos `NOVA-4-40` a `43` para que no hubiera referencias repetidas.



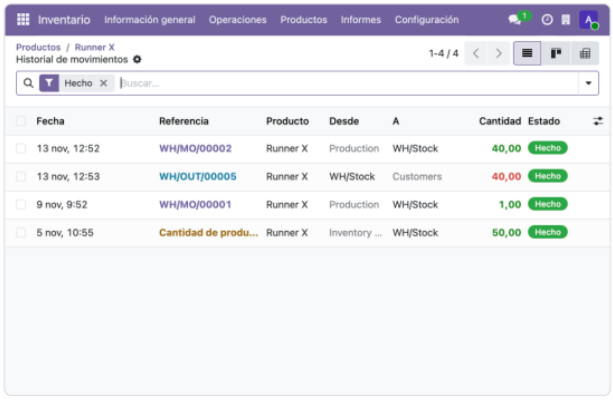
Captura 27 · Variantes de producto de la Runner X con su referencia interna

Página 16 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

> El stock original de la Runner X (51 unidades) quedó en la variante de la talla 40; las demás están a 0. El precio de venta de la Runner X se cambió de 89 € a 99,90 € para igualarlo con el de PostgreSQL. El pedido S00005 se hizo antes del cambio, a 89 €.

Trazabilidad del stock

El historial de movimientos de la Runner X recoge todo el recorrido: inventario inicial, fabricación de prueba, fabricación del pedido Custom y su entrega.

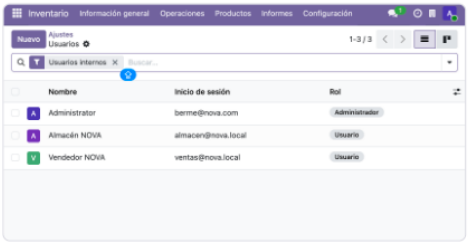


Captura 25 · Historial de movimientos: +50, +1 (WH/MO/00001), +40 (WH/MO/00002), −40 (WH/OUT/00005) = 51

Usuarios

Además del administrador se crearon dos usuarios internos con permisos mínimos para su función:

Usuario	Inicio de sesión	Permisos
Vendedor NOVA	`ventas@nova.local`	Ventas: usuario
Almacén NOVA	`almacen@nova.local`	Inventario: usuario · Fabricación: usuario



Captura 28 · Usuarios internos de Odoo

Página 17 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

9. Estado final y pruebas

Hecho y probado

- Ubuntu 24.04 en OrbStack con Odoo 19 como servicio y base `nova_sneaker`.
- Ventas, Inventario, Fabricación, Facturación y Contactos instalados.
- Runner X con lista de materiales y orden de fabricación `WH/MO/00001` terminada (stock 50 → 51).
- Pedido Custom S00005 de 40 pares recorrido entero: fabricación `WH/MO/00002`, entrega `WH/OUT/00005` y factura `INV/2026/00001` pagada.
- Lista de materiales de la Runner X limpiada: queda una sola.
- Runner X con 9 variantes por talla y referencias `NOVA-4-38` a `NOVA-4-46` (OD-02).
- Usuarios de ventas y de almacén creados.
- Pedidos de la web creados y confirmados en Odoo: S00003 (119,90 €) y S00004 (599,50 €), con importes iguales a PostgreSQL.
- Sin duplicados (búsqueda por `client_order_ref`) y reintento tras fallo.

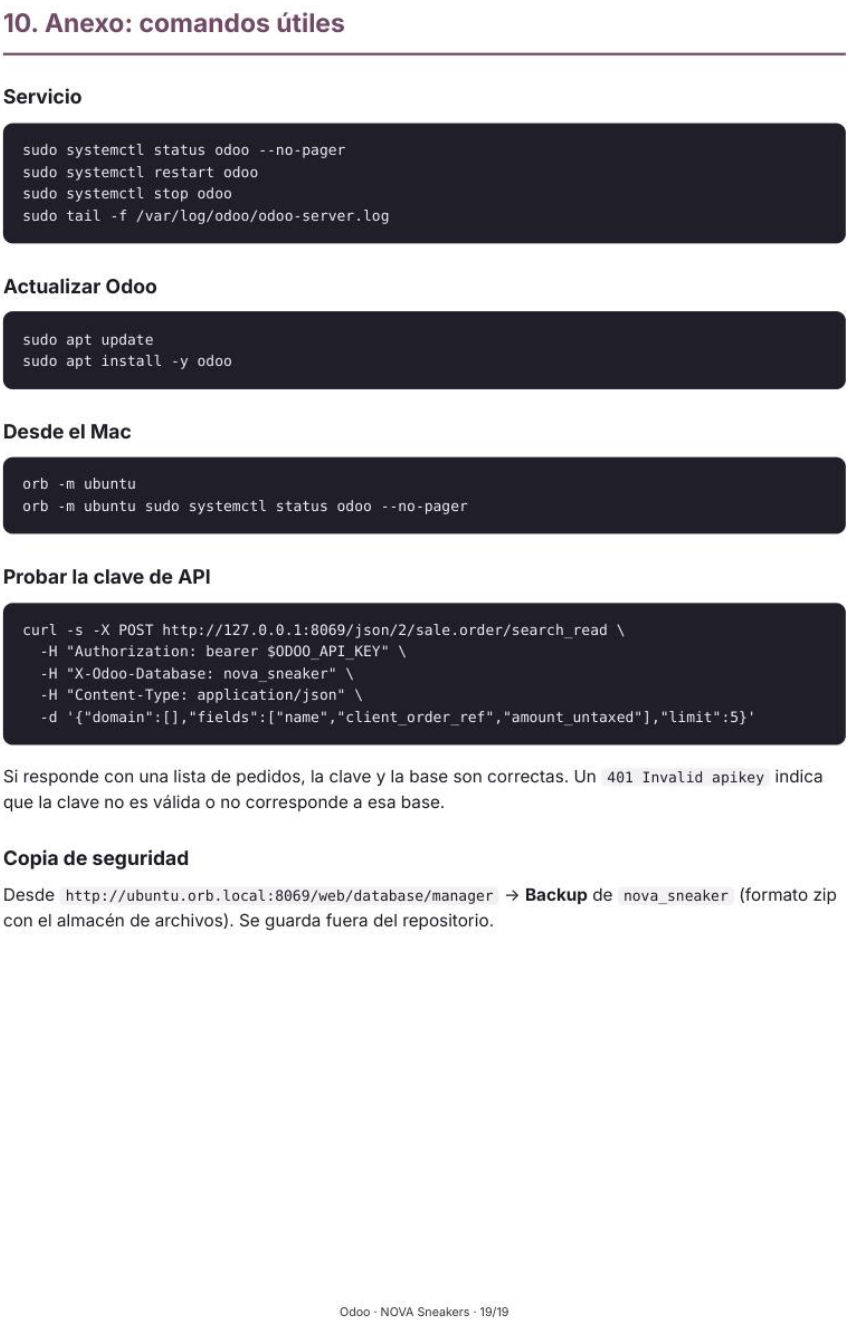
Tareas de Odoo

Tarea	Estado	Dónde está
OD-01 · Instalación de Odoo	Hecha	Apartados 1 a 3
OD-02 · Variantes, referencias y usuarios	Hecha	Apartado 8
OD-03 · Venta del pedido Custom	Hecha	Apartado 5
OD-04 · Producto y fabricación	Hecha	Apartados 4 y 5
OD-05 · Entrega y stock	Hecha	Apartados 5 y 8
OD-06 · Factura y pago	Hecha	Apartado 5

Mejoras posibles

- Que la venta cree sola la orden de fabricación, ligada al estado `en_preparacion` del proyecto. Hoy se crea a mano.
- Sincronizar el stock entre Odoo y PostgreSQL. Hoy cada uno lleva el suyo.
- Cargar el resto del catálogo (60 SKU) como variantes. Hoy está la Runner X con sus 9 tallas.

Página 18 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.



10. Anexo: comandos útiles

Servicio

```
sudo systemctl status odoo --no-pager
sudo systemctl restart odoo
sudo systemctl stop odoo
sudo tail -f /var/log/odoo/odoo-server.log
```

Actualizar Odoo

```
sudo apt update
sudo apt install -y odoo
```

Desde el Mac

```
orb -m ubuntu
orb -m ubuntu sudo systemctl status odoo --no-pager
```

Probar la clave de API

```
curl -s -X POST http://127.0.0.1:8069/json/2/sale.order/search_read \
  -H "Authorization: bearer $ODOO_API_KEY" \
  -H "X-Odoo-Database: nova_sneaker" \
  -H "Content-Type: application/json" \
  -d '{"domain":[],"fields":["name","client_order_ref","amount_untaxed"],"limit":5}'
```

Si responde con una lista de pedidos, la clave y la base son correctas. Un `401 Invalid apikey` indica que la clave no es válida o no corresponde a esa base.

Copia de seguridad

Desde `http://ubuntu.orb.local:8069/web/database/manager` → **Backup** de `nova_sneaker` (formato zip con el almacén de archivos). Se guarda fuera del repositorio.

Odoo · NOVA Sneakers · 19/19

Página 19 de 19 del documento original «Odoo-NOVA-Sneakers.pdf», de Iván Bermejo.

Anexo C. Evidencias de pruebas

Qué	Dónde (en el repositorio)
Regresión del código final (instalación desde cero, SQL, Java, PSP, Python, Node y capturas)	`06_Intermodular/evidencias/cierre-entrega-20261113/`
Regresiones anteriores (Edge, Ethereal, caso K), de árboles anteriores al cierre	`06_Intermodular/evidencias/regresion-final/` y `regresion-limpieza/`
Web: Lighthouse, axe y capturas	`01_Interfaces/evidencias/web/`
Tienda, panel, portal, reparto y correo	`01_Interfaces/evidencias/`

PSP: fichas, registros y ensayo SO024 PROVISIONAL	`03_PSP/evidencias/`
Odoo: 28 capturas	`odoo/documentacion/Odoo-NOVA-Sneakers.pdf`

Anexo D. Equivalencias de commits

El 13/11/2026 la historia publicada en GitHub se reescribió: los archivos, los autores y los mensajes se conservan, pero las fechas y los identificadores cambiaron. Las evidencias anteriores citan los identificadores antiguos. La tabla con su equivalente publicado está en `06_Intermodular/EQUIVALENCIAS_COMMITS.md`.

Anexo E. Declaración de uso de herramientas de inteligencia artificial

Persona	Herramienta	Para qué se usó	Qué revisó o cambió después
Omar	Claude Code (Anthropic)	Escribir, revisar y probar código y documentación en sus sesiones de noviembre de 2026: integración de ramas y regresiones; partes de PSP (coordinador e hijos), del servidor Node y de la tienda; limpieza y organización por asignaturas; documentación de entrega, auditorías y pruebas automáticas; borrador de esta memoria	**[OMAR: confirmar y completar]**
Omar	Codex (OpenAI)	Revisiones, auditorías y preparación del cierre del 13/11 (PR #2 y #3)	**[OMAR: confirmar y completar]**
Álvaro	**[ÁLVARO: herramienta o «ninguna»]**	**[ÁLVARO: para qué]**	**[ÁLVARO: qué revisó]**
Iván Bermejo	**[BERME: herramienta o «ninguna»]**	**[BERME: para qué]**	**[BERME: qué revisó]**

Los commits de `omar1676` incluyen código y documentación escritos con esa ayuda. Cada miembro debe poder explicar y defender su parte.

Anexo F. Requisitos y evidencias

La correspondencia completa entre cada requisito del enunciado, su estado y su evidencia está en `06_Intermodular/rubricas/MATRIZ_RUBRICAS.md` y `MATRIZ_EVIDENCIAS.md`.